

# Fast and Efficient Perfectly Secure Network-Agnostic Secure Computation

Gilad Asharov<sup>\*1</sup>, Fatima Elsheimy<sup>\*\*2</sup>, and Gilad Stern<sup>\*\*\*3,4</sup>

<sup>1</sup> Bar-Ilan University, Israel

<sup>2</sup> Yale University, Connecticut, USA

<sup>3</sup> Tel-Aviv University, Israel

<sup>4</sup> The Hebrew University of Jerusalem, Israel

**Abstract.** Secure multiparty computation (MPC) enables mutually distrustful parties to jointly compute over private data without revealing their inputs. While protocols in both synchronous and asynchronous settings have achieved impressive efficiency in either communication or round complexity, combining the two has remained challenging. Only recently, Abraham, Asharov, Patil, and Patra (Eurocrypt’23, Eurocrypt’24) achieved protocols that combine low communication complexity ( $O((Cn + Dn^2 + n^4) \log n)$ ) with fast execution ( $O(D)$  rounds) in both synchronous and asynchronous models, for circuits of size  $C$  and depth  $D$ , in the perfect setting with optimal resilience. However, both protocols crucially assume advanced knowledge of the network type and are fragile under mismatched or varying network conditions.

The fragility of protocols under mismatched network assumptions highlights the need for *network-agnostic* MPC, where security and correctness are preserved in both synchronous and asynchronous settings. Yet, all known perfect network-agnostic protocols incur  $\Omega(D + n)$  rounds and communication complexity in the order of  $n^3$  or higher, far worse than their network-specific counterparts.

---

<sup>\*</sup> [Gilad.Asharov@biu.ac.il](mailto:Gilad.Asharov@biu.ac.il). Asharov is supported by the European Union (ERC, FTRC, 101043243). Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

<sup>\*\*</sup> [Fatima.Elsheimy@yale.edu](mailto:Fatima.Elsheimy@yale.edu). Fatima Elsheimy is supported by the Center for Algorithms, Data, and Market Design at Yale (CADMY).

<sup>\*\*\*</sup> [giladstern@tauex.tau.ac.il](mailto:giladstern@tauex.tau.ac.il). Gilad Stern is supported in part by ISF 2338/23, AFOSR Award FA9550-23-1-0387, and AFOSR Award FA9550-23-1-0312. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the United States Government or AFOSR. Gilad Stern is partially supported by the Israel Science Foundation (Grant No. 1336/22) and by the European Union (ERC, FTRC, 101043243). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

In this work, we present the first network-agnostic MPC protocol in the perfect security setting, achieving expected round complexity  $O(D)$ . Our protocol has expected communication complexity  $O((Cn^2 + Dn^2 + n^4) \log n)$ , improving on the state of the art by a factor of  $n^3$  for small circuits and  $n^2$  for large circuits in communication, in addition to an additive  $O(n)$  improvement in round complexity. Our design departs from the structure of prior network-agnostic protocols and introduces several new technical ideas that enable both round and communication efficiency.

# Table of Contents

|                                                                                |          |
|--------------------------------------------------------------------------------|----------|
| <b>Fast and Efficient Perfectly Secure Network-Agnostic Secure Computation</b> | <b>1</b> |
| <i>Gilad Asharov, Fatima Elsheimy, and Gilad Stern</i>                         |          |
| 1 Introduction                                                                 | 4        |
| 1.1 Our Results                                                                | 5        |
| 1.2 Related Work                                                               | 6        |
| 2 Technical Overview                                                           | 7        |
| 2.1 The Synchronous Attempt                                                    | 9        |
| 2.2 Agreement                                                                  | 10       |
| 2.3 Multiplication Triplets with a Dealer                                      | 17       |
| 3 Preliminaries                                                                | 18       |
| 3.1 Network and Adversarial Model                                              | 19       |
| 3.2 Modeling Network Agnostic Protocols in UC                                  | 23       |
| 3.3 Reliable Broadcast                                                         | 25       |
| 4 Agreement on a Core Set                                                      | 26       |
| 5 Incomplete Verifiable Secret Sharing                                         | 28       |
| 5.1 The Functionality                                                          | 28       |
| 5.2 The Protocol                                                               | 29       |
| 5.3 Efficiency and Batching                                                    | 31       |
| 5.4 Security Analysis of Incomplete VSS                                        | 32       |
| 6 Complete Verifiable Secret Sharing                                           | 41       |
| 6.1 The Functionality                                                          | 41       |
| 6.2 The Protocol                                                               | 41       |
| 6.3 Efficiency and Batching                                                    | 43       |
| 6.4 Security Analysis of Complete VSS                                          | 43       |
| 7 Triplets Sharing and Product Relation Verification                           | 52       |
| 7.1 Verifying Using Trivariate Sharing                                         | 52       |
| 7.2 Triple Sharing construction                                                | 67       |
| 8 Perfectly Secure Network Agnostic MPC                                        | 74       |
| 8.1 Secret Reconstruction                                                      | 74       |
| 8.2 MPC Protocol                                                               | 74       |
| 8.3 Input sharing and Preparing Beaver Triples                                 | 75       |
| 8.4 Circuit Evaluation                                                         | 76       |
| 8.5 Efficiency                                                                 | 76       |

## 1 Introduction

Secure multiparty computation (MPC) is widely regarded as one of the most significant accomplishments of modern cryptography. It enables mutually distrustful parties to jointly compute a function on private inputs without revealing them. In this work, we focus on *perfectly secure computation*. Secure computation protocols fall into two main categories—synchronous and asynchronous—depending on their assumptions about the network.

In the *synchronous* model, all messages between honest parties are delivered within a fixed, known delay bound. This bound must hold for the system’s lifetime. Setting it is delicate: if too large, protocols waste time idling; if too small, messages may miss deadlines, risking non-termination or even security failures. In practice, establishing such a bound is difficult, especially under adversarial delays. The *asynchronous* model avoids this assumption: messages are guaranteed to arrive eventually, but with unknown and potentially unbounded delay. This allows protocols to adapt to adversarial conditions and ensures eventual termination even when message delivery is heavily manipulated. Since the introduction of MPC, research both in synchrony and asynchrony has followed two main tracks:

- **Linear-but-slow protocols:** Low communication complexity ( $O(n \log n)$  per multiplication gate) but high number of rounds. For example, in the synchronous model, [18] achieves  $O(Cn + n^3 \log n)$  communication and  $\Omega(D + n^2)$  rounds for a circuit with  $C$  multiplication gates and depth  $D$ . Similar results hold in the asynchronous setting [17, 19, 46–49].
- **Fast-but-not-linear protocols:** Constant-round per multiplication (total  $O(D)$ ) protocols [5, 13, 21, 31, 35, 41] but with high communication, e.g.,  $\Omega(Cn^3 + n^3 \log n)$  in the synchronous setting.

Achieving protocols that have both linear communication and are fast – remained elusive. Only recently, two protocols have emerged that achieve both efficiency and speed:  $O((Cn + Dn^2 + n^4) \log n)$  communication with  $O(D)$  rounds in expectation. This is true for the synchronous setting [2] and the asynchronous setting [3, 4].

**A unified model.** Both synchrony and asynchrony models have drawbacks. The synchronous setting is idealized, demanding round-by-round coordination and an accurate delay bound, which is hard to guarantee even with external clocks. The asynchronous setting, though more realistic, is often viewed as overly pessimistic, and it offers weaker security: perfect synchronous MPC tolerates up to  $t < n/3$  corruptions, while perfect asynchronous MPC tolerates only  $t < n/4$ , for  $n$  participants, and those bounds are tight [7, 22].

Even worse, running a synchronous protocol over an asynchronous network provides no guarantees at all: the protocol may fail to terminate, leak secrets, and even correctness can be lost. This is true even if run over a synchronous network, but with an inaccurate bound delay. In contrast, running an asynchronous protocol over a synchronous network does work but does not provide the same

level of security as synchronous protocols do. It works only with the weaker  $t < n/4$  threshold instead of  $t < n/3$ , and does not necessarily take the inputs of all honest parties into account.

This leads us to the notion of *network agnostic protocols* – the same code works on both networks without knowing the type of the network [8, 10–12, 23–25, 37]. This eliminates the need to commit to one model in advance. When the delay bound is accurate, such a protocol matches the higher resilience of synchrony. If the delay bound is inaccurate, it degrades gracefully, retaining asynchronous guarantees. In a truly asynchronous network, it naturally provides the robustness of the asynchronous model.

However, designing such a protocol is highly challenging. As opposed to the synchronous or asynchronous setting, all known network agnostic protocols are *not fast* and *not linear*. That is, no protocol is optimal in even one of the dimensions. The protocol of [10] achieves  $\Omega(Cn^4 \log n + n^7 \log n)$  communication and  $\Omega(D + n)$  rounds. On the other hand, the work of [14] shows a computational MPC and distributed key generation protocols while having  $O(n + D)$  rounds and  $\tilde{O}(Cn^2)$  communication. This motivates the central questions:

*Is it possible to construct a fast network-agnostic protocol?*  
*Is it possible to construct a fast and linear network-agnostic protocol?*

### 1.1 Our Results

We answer the first question in the affirmative, achieving the first network-agnostic fast protocol. In addition, we make significant progress towards answering the second:

**Theorem 1.1.** *Let  $n > 3t_s + t_a$  and  $t_s \geq t_a$ , and assuming  $t_s, t_a \in \Theta(n)$ . There exists a single network-agnostic multiparty computation protocol for  $n$  parties such that for computing a circuit with  $C$  multiplication gates and depth- $D$  such that: (1) Communication: The protocol has  $O((Cn^2 + Dn^2 + n^4) \log n)$  expected communication complexity; (2)  $O(D)$  expected rounds; (3) Resilient to  $t_s$  corrupted parties in synchronous networks; (4) and resilient to  $t_a$  corrupted parties in asynchronous networks.*

We note that even using many insights from previous synchronous, asynchronous, and network-agnostic protocols, the second question remains open: Is it possible to achieve linear communication per gate? If so, is it possible to do so while keeping the round complexity  $O(D)$ ? Solving this question remains interesting, and would allow us to close the gap between network-agnostic protocols and their synchronous and asynchronous counterparts.

Technically, our protocol contains a different structure compared to previous network-agnostic protocols (e.g., [10, 11]), allowing us to use cheaper alternatives for reliably broadcasting messages and agreeing on a core set of inputs asynchronously. This allows us to drastically reduce the use of consensus throughout the whole protocol.

**A limitation.** Our protocol does not provide the exact same security guarantee as the asynchronous protocol of [3, 4]. Specifically, the latter tolerates  $t < n/4$  corruptions and computes on an arbitrary set of  $n - t$  (at least  $3n/4$ ) inputs, chosen by the adversary; equivalently, the missing  $t$  inputs can be treated as default values. In contrast, in asynchronous executions our protocol may consider only  $n - t_s$  (at least  $2n/3$ ) inputs.

Previous works also faced this limitation (e.g., [10, 14, 45]). Bacho et al., [14] proved that this limitation is *inherent*: every network-agnostic MPC protocol is vulnerable to the following adversarial behavior in asynchronous networks. An adversary can mimic a synchronous execution by delivering all its own messages within the delay bound while delaying those of  $t_s$  honest parties. From the protocol’s perspective, this is indistinguishable from a synchronous execution where  $t_s$  parties are corrupted and remain silent. Consequently, the protocol must proceed with only  $n - t_s$  inputs. More formally, we define as  $(n - k)$ -asynchronous MPC functionality a functionality in which parties send their inputs to the trusted party, and the adversary is allowed to exclude the input of  $k$  parties of its choice. More formally,

**Theorem 1.2.** *Let  $t_s \geq t_a$  be two parameters. There does not exist a network-agnostic protocol that is secure against  $t_s$  corrupted parties over synchronous network, and  $t_a$  corrupted parties over asynchronous network, and computes the  $(n - t_a)$ -asynchronous MPC functionality.*

## 1.2 Related Work

**Synchronous MPC.** The first feasibility results in synchronous MPC goes back to the seminal protocol of Ben-Or, Goldwasser and Wigderson [21] and Chaum, Crépeau, and Damgård [31]. Already [21], together with the broadcast protocol of [39] gives  $O(D)$ -round protocol, but not linear communication. Linear communication was first achieved in 2008 by [18], but with high round complexity of  $\Omega(D + n^2)$ . Many improvements followed along the years (e.g., [5, 13, 35, 41, 42]). The first linear and fast protocol was achieved only recently by [2], with  $O((Cn + Dn^2 + n^4) \log n)$  expected communication complexity and  $O(D)$  round complexity.

**Asynchronous MPC.** In asynchronous MPC, the first feasibility result is given by Ben-Or, Canetti and Goldreich [19]. Here, progress has been slower. The work of [19] obtained  $\tilde{O}(n^6)$  per gate. [48, 49] improved to  $\tilde{O}(n^5)$  per gate, followed by [17] with  $\tilde{O}(n^3)$  per gate, and [46, 47] with  $\tilde{O}(n^2)$  per gate. While the known techniques in 2008 by [18] were sufficient to achieve linear communication in synchrony, the same the techniques were not suffice to achieve a similar results in asynchrony. It took 15 more years to reach linear communication compared to the synchronous setting, with the work of [3], achieving the first asynchronous MPC protocol with  $O((Cn + Dn^2 + n^5) \log n)$  expected communication complexity and  $O(D + \log n)$  expected round complexity. The added  $\log n$  term in the

round complexity was removed by [4], which also reduced the expected communication to  $O((Cn + Dn^2 + n^4) \log n)$ , matching the efficiency of the synchronous construction.

**Network agnostic.** Network-agnostic protocols have been extensively studied since their introduction by Blum, Katz, and Loss [23]. We already mentioned the works of [10], and [45] and [14]. Blum, Katz, and Loss [23] presented a network-agnostic computational consensus protocol, used as an important building block in other protocols. They followed this work by constructing a computational atomic broadcast protocol [24]. Alexandru, Blum, Katz, and Loss [8] solved the highly related task of computational state machine replication, while allowing the network to arbitrarily switch between being synchronous and asynchronous. Network-agnostic MPC with non-perfect security guarantees has also been studied [12, 25, 37].

**On optimal resilience.** While  $n \geq 3t + 1$  is the optimal resilience in the synchronous setting and  $n \geq 4t + 1$  is optimal in the asynchronous setting, the optimal resiliency for network-agnostic constructions remains unclear. When  $t_a = 0$ , Theorem 1.1 yields a synchronous protocol with  $n \geq 3t_s + 1$  participants, which is optimal for the synchronous case. Similarly, when  $t_a = t_s$ , the network-agnostic protocol of Theorem 1.1 specializes to an asynchronous protocol with  $n \geq 4t_a + 1$ , achieving optimal resilience in that regime as well. For other parameter choices of  $(t_s, t_a)$ , however, the optimal bound is unknown. The work of [45] presents a protocol secure when  $n > 2t_s + \max\{t_s, 2t_a\}$ , which also coincides with  $n \geq 3t_s + 1$  when  $t_a = 0$  and with  $n \geq 4t_a + 1$  when  $t_s = t_a$ , while offering better resilience for other values of  $(t_s, t_a)$ . Their protocol, however, requires *exponential* communication, suggesting that characterizing the optimal resiliency for *efficient* network-agnostic protocols is a delicate task.

## 2 Technical Overview

In this section, we provide a technical overview of our work. We start with clarifying the model, following specifying the structure of our protocol and highlighting our different approach compared to previous works.

**The setting and modeling.** Before proceeding, we recall the setting. Our goal is to design a single MPC protocol in which the parties are unaware of the underlying network. In other words, the same “code” (next-message function) should satisfy the following when computing a function  $(y_1, \dots, y_n) = f(x_1, \dots, x_n)$ :

- *Synchronous:* If the network is synchronous, the protocol securely computes  $f$  while tolerating up to  $t_s$  corrupted parties. In this setting, the functionality receives inputs from all honest parties, and the adversary controls the inputs of the  $t_s$  corrupted parties.
- *Asynchronous:* If the network is asynchronous, the protocol securely computes  $f$  while tolerating up to  $t_a$  corrupted parties. In this setting, the adversary may exclude up to  $t_s$  honest parties from the computation, and the

functionality computes  $f$  on (at least)  $n - t_s$  inputs (see Theorem 1.2), where the adversary can decide which parties to exclude, and with the adversary also supplying the inputs of the  $t_a$  corrupted parties.

This already implies that the functionality behaves differently depending on whether the network is synchronous or asynchronous. Hence, the functionality must be aware of the network type, in contrast to the real protocol, where parties remain agnostic. This aligns with other modelings in the UC framework, e.g., where functionalities are aware of the identities of the corrupted parties. In Section 3.1, we describe how we model network-agnostic within UC and examine such issues in greater detail.

**General recipe for network-agnostic protocols.** We begin with a general recipe for constructing network-agnostic protocols. Suppose we are given a synchronous protocol  $\Pi_s$  for some task, and an asynchronous protocol  $\Pi_a$  for the same task. Clearly,  $\Pi_a$  can also run in a synchronous network, but this often yields weaker security and resilience guarantees. The guiding principle is therefore simple: attempt to run the stronger protocol  $\Pi_s$ ; if it fails, fall back to  $\Pi_a$ . Concretely:

- **Synchrony:** Run  $\Pi_s$ , which attempts to compute the task with higher resilience (tolerating up to  $t_s$  corruptions). In fact,  $\Pi_s$  *has to be more robust than a regular synchronous protocol*, and provides some security guarantees even when running in an asynchronous network (e.g., it should guarantee privacy of the honest parties' inputs, even if it fails).
- **Agreement:** When  $\Pi_s$  is expected to terminate (this requires local clocks), parties verify whether they obtained outputs as prescribed. They then execute a network agnostic Byzantine agreement protocol to ensure consensus: either all parties confirm the successful completion of  $\Pi_s$  (and output the outputs they obtained), or they all agree to revert to the asynchronous protocol.
- **Asynchrony:** If parties revert to asynchrony, they switch to  $\Pi_a$ . Note that  $\Pi_a$  typically offers weaker guarantees, since  $t_a < t_s$ .

Indeed, previous works [10–12, 45] followed this general recipe, but with significant redundancy. They applied the recipe separately to each component of the protocol—for example, verifiable secret sharing, the broadcast primitive, Beaver triple generation, and so on. While this produces network-agnostic building blocks that can operate independently and are interesting on their own merit, such modularity is unnecessary when the goal is to construct a full MPC protocol for general functionalities. As a result, the approach introduces a large number of redundant Byzantine agreement instances, which are computationally expensive. For example, the protocol of [10] contains  $\Omega(n^3)$  such instances, each is expensive (at least  $\Omega(n^4 \log n)$ ).

**Our approach and concrete instantiations.** We follow two simple design principles:



1. **Whole over parts:** Instead of applying the general recipe separately to each building block, we step back and apply it to the *entire* MPC protocol as a whole.
2. **Async covers sync:** Any asynchronous protocol remains secure when executed in a synchronous network. In general MPC this may yield a suboptimal security threshold ( $4t+1$  and not  $3t+1$ ), but for many building blocks where stronger thresholds are attainable in asynchrony, the asynchronous protocol suffices.

Concretely, our baselines for the general recipe are as follows:  $\Pi_s$ , the synchronous protocol of [2]; and  $\Pi_a$ , the asynchronous protocol of [3], augmented with the improved verifiable secret sharing of [4]. As with most secret-sharing based MPC protocols, both follow a similar high-level structure:

1. A *preprocessing* phase, where the parties generate Beaver multiplication triples [16] independently of their inputs;
2. An *input-sharing* phase, where each party secret-shares their inputs to all others;
3. A *circuit-emulation* phase, where the circuit is computed gate-by-gate using the preprocessed triples;
4. An *output-reconstruction* phase.

Our MPC protocol, therefore, has the following high-level structure:

1. **A synchronous attempt:** We modify Steps 1 and 2 of  $\Pi_s$  so that they provide guarantees even in an asynchronous setting. This forms the crux of our technical contribution, detailed in Section 2.1.
2. **Agreement:** The parties execute an agreement protocol to determine whether they must fall back to the asynchronous setting. We discuss this step in Section 2.2.
3. **Asynchronous fallback:** If a fallback is required, the parties switch to Steps 1 and 2 of  $\Pi_a$ , operating with a lower threshold. Here, we make a direct use of  $\Pi_a$  [3, 4].
4. **Circuit evaluation and output reconstruction:** Here we apply the “async covers sync” design principle, and uniformly adopt Steps 3 and 4 of  $\Pi_a$ . Importantly, both  $\Pi_s$  and  $\Pi_a$  enter these phases only after the “commitment round” [38], at which point the output is already fixed for all parties, and this asynchronous sub-protocol actually provides the higher security threshold needed for the synchronous protocol. We can therefore unify this step and just run its asynchronous variant.

## 2.1 The Synchronous Attempt

We proceed with a brief overview of the synchronous attempt, i.e., with Steps 1 and 2 in the synchronous protocol  $\Pi_s$ . We emphasize that the security of those steps has to be enhanced in the case where we actually run it in asynchrony, and we elaborate on that in Sections 2.2 and 2.3.

**Step 1—pre-processing.** The goal of the pre-processing phase (Step 1) is to generate  $|C|$  distinct multiplication triplets, where  $|C|$  bounds the number of circuit multiplications. Each triplet consists of three degree- $t_s$  polynomials  $A(x), B(x), C(x)$  satisfying  $C(0) = A(0) \cdot B(0)$ . No party should know these polynomials outright; instead, each  $P_i$  should hold only the shares  $A(i), B(i), C(i)$ . This is achieved as follows:

1. **Triplet extraction.** To ensure no party knows the final  $A, B, C$ , each party  $P_i$  first acts as a dealer, generating  $A_i(x), B_i(x), C_i(x)$  of degree  $t_s$ , with  $C_i(0) = A_i(0) \cdot B_i(0)$ , and distributing shares to all  $P_j$ . A triplet extraction protocol then converts  $O(M)$  such dealer-based triplets into  $O(M)$  dealer-free triplets.
2. **Dealer-based triplet generation.** Each party  $P_i$  distributes  $A_i(x), B_i(x), C_i(x)$  of degree  $t_s$ , with  $C_i(0) = A_i(0) \cdot B_i(0)$  using the following two sub-components:
  - (a) **Verifiable secret sharing (VSS).** The dealer  $P_i$  shares  $A_i(x), B_i(x), C_i(x)$  of degree  $t_s$ . VSS ensures that the distributed values are consistent with degree- $t_s$  polynomials.
  - (b) **Multiplication verification with a dealer.** The dealer is supposed to pick those polynomials  $A_i(x), B_i(x), C_i(x)$  with the constraint that  $C_i(0) = A_i(0) \cdot B_i(0)$ . The parties run a distributed zero-knowledge proof to verify that indeed  $C_i(0) = A_i(0) \cdot B_i(0)$ , where the dealer (who is the only one who actually knows  $A_i, B_i, C_i$ ) is the prover. If verification fails, the parties substitute a default triplet.

**Step 2: Input sharing.** In Step 2 of  $\Pi_s$ , each party simply provides its input via VSS, and there is no reason to redesign an additional building block.

First, for triplet extraction, we can once again apply the “async covers synch” principle, as the asynchronous variant of this task suffices also in synchrony. Thus, enhancing  $\Pi_s$  in the asynchronous model reduces to strengthening two components: (1) the underlying VSS; and (2) the multiplication verification with a dealer, and we elaborate on those steps in Sections 2.2 and 2.3. Note that both of these protocols are still required to maintain privacy in asynchrony, but are not required to terminate, even for an honest dealer. That is because if we are indeed in asynchrony, there will be a fallback to  $\Pi_a$  if the number of successful dealers is not large enough.

## 2.2 Agreement

We proceed with an overview of Step 2 in our entire MPC protocol. An important goal in our construction is avoiding the many uses of Byzantine Agreement (BA) in all building blocks. The main insight that allows us to avoid multiple instances is the fact that in synchrony, each dealer should be able to eventually complete *all* of its tasks on time, including input sharing, triple sharing, and verification, as we just highlighted above. That is, we not only want to agree on whether to downgrade to asynchrony, but also agree on which dealers finished

their task successfully. This step is the turning point of the computation, as it simultaneously determines:

1. Whether or not we stay in synchrony;
2. If we stay in synchrony, which parties successfully provided and completed their input-sharing. This, in turn, fixes the set of parties for whom a default input will be substituted.

Agreeing on such a set is exactly the task of “agreeing on a core set” (ACS) in asynchronous literature [19]. In our case, each party  $P_i$  inputs a set  $S_i$ , which includes all the dealers that finished their task (by the time they were supposed to, if indeed we are in synchrony). ACS attempts to find a core subset  $K$ , of cardinality at least  $n - t_s$ , that is included in the inputs of at least  $n - t_s$  parties. If ACS succeeds in finding a set  $K$  of size exactly  $n - t_s$ , the execution proceeds along the synchronous route: we move on to circuit evaluation and output reconstruction. If, however, no such set is found (i.e., the largest core set has size  $< n - t_s$ ), the protocol falls back to the asynchronous route, which means re-running the input-sharing and preprocessing, over asynchrony, but with a lower threshold. To integrate ACS into our protocol, we require that it satisfy two properties:

- **Synchronous network:** If the network is synchronous, then  $K$  must include all honest parties. In particular, by the time ACS is invoked, every honest party must have successfully completed its tasks (sharing and generating multiplication triplets) within the synchronous time bounds.
- **Asynchronous network:** If the network is asynchronous and no core set of size  $n - t_s$  is found, then the protocol falls back to asynchrony, and we are fine.

There is, however, a subtle case: the network may be asynchronous while the adversary delivers all messages promptly, causing the parties to believe the execution is synchronous. In this case,  $K$  is allowed not to include all honest parties (as in standard asynchronous MPC). The only requirement is that  $K$  contain at least  $n - t_s$  parties, and that all parties hold valid shares and multiplication triplets corresponding to those parties in  $K$ , ensuring that the remainder of the computation proceeds successfully. This is exactly the case that complements Theorem 1.2.

**The ACS protocol.** The ACS protocol that we run in this step is actually  $n$  parallel instances of BA. Each party  $P_i$  inputs 1 in the  $j$ th instance, if  $P_j$  as a dealer completed all its tasks. We employ the asynchronous parallel BA protocol of [34], which is also secure under synchrony (the protocol of [34] is statistically-secure, but we discuss how to make it perfect; see more details in Section 4). We use [34] as it achieves an expected constant number of rounds.

**Synchronous network.** In synchrony, we make sure that each honest dealer must successfully finish its task on time (i.e., a validity property). If indeed this is guaranteed, then for every honest  $P_j$ , all honest parties would input 1 in the

$j$ th instance of the parallel BA, and thus all honest parties would include  $P_j$  in the core set, as required.

**Asynchronous network.** In asynchrony, parties agree on the output of the ACS protocol, but the core set might be either of size  $\geq n - t_s$  or not; As mentioned, if indeed  $\geq n - t_s$ , the parties continue with the computation, resulting in the function being evaluated on a large enough set of inputs, as required. On the other hand, if the set is too small to be considered valid, all parties detect the fact that the network must be asynchronous, and fall back to an asynchronous MPC protocol. (We also remark that inside the asynchronous MPC there will be another invocation of an ACS protocol, as an inside building block of an asynchronous MPC protocol.)

**Broadcast.** Each one of the VSSes, multiplication triplets with a dealer, ends with a vote whether or not to accept the instance of the dealer or not. Such a vote is a consensus. We observe that we can delay those internal decisions to this global ACS, heavily reducing the number of BAs the protocol performs.

Not only that, we also reduce the costs of broadcasts in those VSSes and multiplication triplets with a dealer. Note that in synchronous there are two options for implementing broadcast:

1. An  $O(n)$  round broadcast that we know an exact time when it ends;
2. An expected constant round protocol; here, it is unknown when exactly the broadcast would terminate (this is a random variable), and there is no simultaneous termination.

Both options complicate stuff for us; using the first option implies that we will spend  $O(n)$  rounds on the synchronous route, and not achieve a fast protocol; The second option implies that we would not know when exactly to invoke the ACS protocol.

Here we once again apply the “async covers sync” design principle. We replace these broadcasts with reliable broadcast [26]: a weaker asynchronous primitive that, for corrupted senders, only guarantees that if some party receives a message, every party receives the same message. Note that (asynchronous!) reliable broadcast works with the stronger threshold of  $n \geq 3t + 1$  (as opposed to  $n \geq 4t + 1$  in general MPC), allowing us to use it in synchrony. The important property is that, if the sender is honest, then its message is guaranteed to arrive by a constant number of rounds. This allows us to work within known time bounds without waiting  $O(n)$  rounds for synchronous broadcast to terminate.

Using reliable broadcast, all honest parties’ messages will arrive on time, allowing honest dealers to make progress on schedule. After the ACS, parties know that they can continue waiting for broadcasts to arrive for all chosen dealers without worrying that they might wait indefinitely, even if they did not hear the messages on time. We refer the reader to Section 4 for further details regarding the ACS.

### Verifiable Secret Sharing

We now move back to Steps 2 of the synchronous protocol, while focusing on the security enhancement it has to guarantee to be run over asynchrony. Our baseline for this overview is the verifiable secret sharing of BGW [21], which proceeds in synchronous steps:

1. The dealer embeds its univariate input polynomial  $h(x)$  in a bivariate polynomial  $S(x, y)$  such that  $S(x, 0) = h(x)$ . It sends each party  $P_i$  the pair  $f_i(x) = S(x, i)$ ,  $g_i(x) = S(i, y)$ .
2. The parties exchange sub-shares, i.e.,  $P_i$  sends to  $P_j$  the pair  $(f_i(j), g_i(j))$ .
3. The parties verify that they received the correct sub-shares; if  $P_i$  did not receive the expected sub-shares from  $P_j$ , then it publicly complains by broadcasting  $(\text{NOK}, i, j, f_i(j), g_i(j))$ . Note that it uses the shares it received from the dealer, and not from  $P_j$ .
4. If some party  $P_i$  falsely complains – the dealer publicly reveals the shares of that  $P_i$ .
5. The parties at the end of the protocol have to agree whether to accept the shares of that dealer: Each party should see that the dealer resolved all complaints, and whatever is publicly revealed is consistent with its private share.

**What might go wrong?** We now analyze what might go wrong when running such a protocol in an asynchronous network.

1. When the dealer is honest, the parties are never supposed to complain about one another. However, in asynchronous networks, a party  $P_i$  might not receive the sub-share from an honest  $P_k$ , and thus complain about  $P_k$  by the time it considers as “the end of the round.” In the original protocol, honest parties only complain about corrupted parties, and therefore, making these sub-shares public (as part of the complaint) does not reveal any extra information to the adversary. However, when both parties are honest, making these sub-share *public* leads to a privacy violation since this reveals sub-shares of honest parties.
2. Even when the dealer is honest, we might get non-termination. When the dealer is honest, all parties are supposed to receive messages by known given times, and proceed in the protocol accordingly. Running it in an asynchronous network, parties might receive messages too late, and not be able to proceed on time.
3. When the dealer is corrupted, we might not get the binding property (shares of the honest parties must all lie on the same polynomial). Binding is achieved by having *all* honest parties exchange subshares and having *all* complaints resolved on time. In an asynchronous execution, some honest parties might not talk to each other, and some complaints might not be heard on time.

This demonstrates why we need enhanced guarantees for this building block. Note that we do not need it to fully operate in asynchronous network. We just have to make sure that it does not completely fails.

**Low Communication is challenging.** Apart from achieving the two key properties: completeness in the synchronous setting (i.e., an honest dealer can always complete its sharing, and at a fixed time) and binding in the asynchronous setting (i.e., a malicious dealer cannot equivocate, even if the network is synchronous)—our protocol must operate under a tighter resilience threshold while minimizing communication complexity, which is our primary efficiency objective. Our target is to achieve  $O(Cn^2)$  communication. For context, in the BGW protocol, the parties exchange  $O(n^2 \log n)$  bits over point-to-point channels and an additional  $O(n^2 \log n)$  bits over the broadcast channel, resulting in a total communication cost of  $O(n^4 \log n)$  when broadcast is realized via standard techniques. Thus, BGW incurs an overhead of  $O(n^4 \log n)$  for the sharing of a single value!

To reduce this overhead, our construction relies on packing and batching. Packing means that we increase the degree of the shared polynomial, so that the same messages would encode more secrets, reducing the overall communication per secret; Batching means running multiple instances in parallel, while utilizing the same broadcast messages to resolve all instances simultaneously.

Packing is already nontrivial in asynchronous settings and becomes significantly more challenging under our stricter resilience constraints. To achieve amortization by a factor of  $n$ , we embed  $t_a \in O(n)$  secrets along the  $x$ -dimension of a bivariate polynomial, thereby reducing the per-secret complexity to  $O(n^3)$  per secret. We then show how to ensure binding for such packed values, and finally, how batching enables a further reduction to  $O(n^2)$ .

**Ensuring Binding.** To solve the issue of parties not talking to each other or complaints not being heard, we adopt techniques from asynchronous MPC [20, 27]. However, the main difficulty is that we are working here with a much tighter threshold.

- When two parties heard from one another, and they do agree with their shares, they broadcast (OK,  $i, j$ ).
- When two parties heard from one another, but disagree - they send (NOK,  $i, j, f_i(j), g_i(j)$ ), including the subshares they received from the dealer.
- If a party  $P_i$  did not hear from  $P_j$  by the end of the round, it broadcasts ( $\perp, i, j$ ).

The dealer constructs a graph on  $n$  indices corresponding to the  $n$  parties, where two indices are connected if the associated parties broadcast approval of each other's subshares. Note that if there is a large enough clique of honest parties, then there is binding. However, since finding a clique in a graph is computationally inefficient, the dealer finds instead a “star” which is some approximation of clique [20, 27], that suffices to bind a unique bivariate polynomial. Specifically, the dealer searches for a (C,D)-star, which is defined as follows:

**(C,D)-Star.**  $C, D \subseteq [n]$  is a *Star* in  $G$  if (1)  $C \subseteq D$ ; (2)  $|C| \geq n - 2t$  and  $|D| \geq n - t$ ; (3) for every  $c \in C$  and  $d \in D$ , it holds that  $(c, d) \in G$ .

In our setting,  $n > 3t_s + t_a$ . Consequently,  $|C| \geq n - 2t_s \geq t_s + t_a + 1$  and  $|D| \geq n - t_s \geq 2t_s + t_a + 1$ . These conditions are sufficient to bind a bivariate polynomial  $S(x, y)$  of degree  $t_s + t_a$  in  $x$  and  $t_s$  in  $y$ . Binding is guaranteed in the asynchronous setting (in which at most  $t_a$  parties are corrupted), since then  $|D|$  provides at least  $2t_s + 1 \geq t_s + t_a + 1$  honest parties to fix the  $x$ -dimension and  $|C|$  ensures at least  $t_s + 1$  honest parties to fix the  $y$ -dimension. By contrast, if we are in synchronous network (in which case at most  $t_s$  parties are corrupted), no binding is guaranteed.

To ensure binding in the synchronous setting, we require the dealer to *filter* the graph. Specifically, if any party sends a NOK message containing wrong information, the dealer removes that party from the graph by deleting all of its edges. Intuitively, this marks the party as faulty and excludes it from further consideration. The filtered graph is then the one in which we actually look for a star. We enforce that the dealer must find such a filtered graph of size at least  $n - t_s$ , which is guaranteed to exist in the synchronous setting when the dealer is honest. This ensures that if two honest parties receive inconsistent shares, everyone will observe that they complained about each other with differing values. Consequently, parties will reject the dealer's graph if it contains two parties with conflicting shares. Thus, under synchrony, we can be confident that any two honest parties that remain in the graph must have verified their consistency with each other. In this way, the honest parties that survive filtering collectively form a binding clique in the graph. Finally, because  $n - 2t_s \geq t_s + t_a + 1$ , these honest parties bind one another, ensuring that the bivariate is binding.

**Ensuring Completeness.** In the asynchronous setting, we abandon the requirement that every honest dealer must complete its sharing attempt. Even honest dealers may fail, in which case their inputs are excluded from the computation. Recall that to decide which inputs are used, the parties execute an agreement on a core-set protocol (see Section 2.2). If too few inputs are provided, all parties detect this and fall back to a purely asynchronous execution.

The synchronous setting is stricter: we must ensure that every honest dealer succeeds by a fixed time before running ACS. Note that only parties belonging to the star can be certain they hold correct shares. One might wonder why an honest party in the filtered graph but outside the star should not output its share. The reason is that, in synchrony, correctness hinges on timely NOK messages: honest parties issue NOK complaints on time, ensuring that no two honest parties with conflicting values coexist in the filtered graph. Thus, in synchrony, membership in the filtered graph suffices to ensure correctness. By contrast, in asynchrony, this guarantee no longer holds, and we must rely on the binding properties of the star. Since parties cannot distinguish whether the network is synchronous or asynchronous, they conservatively rely on the star to determine whether their shares are safe to output. This approach works, but there remains a subtle issue: the star has size  $n - t_s$ , which allows an honest dealer to leave up to  $t_s$  honest parties outside the star. To address this, we rely on *extended star*, which is another approximation of a clique (that can be found efficiently) that extends the set  $C$  to contain all honest parties [44].

**No one is left behind.** Additionally, we require that if an honest party outputs a share, then every honest party must output a share as well. This requirement is crucial in the circuit evaluation phase: if the dealer is malicious, we cannot allow a situation where some honest parties hold shares while others do not as reconstruction may fail. To prevent this, parties that did not receive shares during the sharing protocol must be able to recover them with the help of others. The standard approach is that parties exchange subshares, followed by error correction to reconstruct the full shares. To correct errors introduced by corrupted parties, each corrupted position requires at least one additional honest party providing subshares. However, the star structure may not contain enough honest parties to guarantee this recovery. Indeed, we can only guarantee stars of size  $n - t_s$ , which in the worst case  $n = 3t_s + t_a + 1$  gives  $2t_s + t_a + 1$  parties holding shares. Reconstruction of shares of degree  $t_s$  requires  $t_s + 1$  correct points, plus one extra honest point for every corrupted point. Thus, if up to  $t_s$  parties in the star are corrupted, we require a total of  $3t_s + 1$  points to correct  $t_s$  errors. This exceeds the  $2t_s + t_a + 1$  points guaranteed by the star, and therefore standard error correction alone does not suffice.

To avoid this complication, we force corrupted parties to either provide correct subshares or not provide subshares at all. This is done via an internal *incomplete* sharing protocol. That protocol only guarantees that the dealer provides shares on a low-degree polynomial, but does not guarantee that all parties receive shares. Parties then act as dealers in the *incomplete* sharing in order to provide subshares for the *complete* sharing protocols. Using this technique, we know that if some party is included in the star, the polynomial shared in its *incomplete* sharing must be consistent with all honest parties' shares in the star. This is because to be included in the star, a party must be connected to a large enough number of parties in the star (at least  $t_s + t_a + 1$  honest parties in our case). These parties check that the shares received via the *incomplete* sharing are consistent with the ones sent by the dealer in the *complete* sharing. Since the *incomplete* sharing guarantees that all shares are on a low-degree polynomial, any party that receives values from that corrupted party's sharing will also be consistent with the polynomial defined by the honest parties that reported their consistency. Therefore, parties can reconstruct their own shares without the use of error correction. The incomplete sharing has an identical structure to the complete sharing, except that parties outside of the star do attempt to reconstruct their shares using error correction. As stated above, in synchrony, there aren't enough parties to always successfully correct the errors introduced by the corrupted parties, but parties can detect this event and decide not to output shares, making the protocol *incomplete*. In asynchrony, the number of faults is only  $t_a$ , meaning errors can always be corrected, and thus the incomplete sharing protocol is actually complete in asynchrony.

**Batching.** Since both protocols have the same structure, each one requires  $O(n^2 \log n)$  bits of direct communication for sharing a bivariate polynomial. However, as we just saw in the complete sharing, each party acts as a dealer for a single instance of incomplete sharing, the total communication of the complete



sharing protocol is  $O(n^3 \log n)$  for sharing  $O(n)$  secrets, resulting in  $O(n^2 \log n)$  communication per secret on point-to-point communication channels.

In addition to that, parties broadcast  $n$  messages about each other OK/NOK/ $\perp$ , and the dealer broadcasts a constant number of sets of size  $O(n)$  in each sharing. Doing this naively, this would require  $n$  broadcasts of size  $O(n \log n)$ , resulting in  $O(n^4 \log n)$  total communication when using the protocol of [1] that has communication complexity  $O((n^3 + Ln) \log n)$  for broadcasting messages of size  $L$ . To reduce the broadcast costs, we batch many instances of the VSS together, and utilize the broadcast messages only once to resolve all the instances simultaneously. We rely here on the techniques of [2]. Using this idea, the total broadcast costs remain  $O(n^4 \log n)$  for any number of VSS instances, and thus when sharing  $X$  secrets, we get  $O((n^4 + n^2 X) \log n)$  complexity, meaning that the complexity becomes  $O(n^2 \log n)$  per secret when sharing  $O(n^2)$  secrets, or  $O(n)$  polynomials. We refer the reader to Sections 5, 6 for the full details.

### 2.3 Multiplication Triplets with a Dealer

The goal of the preprocessing phase of [2] is generating triples of polynomials  $A(x), B(x), C(x)$  of degree  $t$  such that  $A(0) \cdot B(0) = C(0)$ , with the adversary only learning its own shares on the polynomials. This is done by first having each party  $P_i$  act as a dealer sharing its own triples  $A_i(x), B_i(x), C_i(x)$ , and proving that for those triples  $A_i(0) \cdot B_i(0) = C_i(0)$ . Sharing the triples is done using the previously described VSS protocol (for sharing  $A_i, B_i$  and  $C_i$ ), and the proving that indeed  $C_i(0) = A_i(0) \cdot B_i(0)$ .

We adopt ideas from [2, 3], and omit the  $i$ th subscript for clarity. For a given triple  $(A(x), B(x), C(x))$  of degree- $t$  polynomials, the parties define

$$E(x) := A(x) \cdot B(x) - C(x).$$

If the dealer has shared the polynomials consistently, then the constant term of  $E(x)$  must be zero, since correctness requires  $A(0) \cdot B(0) = C(0)$ . The challenge is thus to verify, in zero knowledge, that this condition holds. The key idea is that the dealer shares the coefficients of  $E$ , letting the parties to compute linear combinations on those coefficient, and in particular, evaluate  $E$  on different points. From the sharing of  $E$ , we design a protocol in which each  $P_k$  can privately recover  $E(k)$ , and verify that indeed  $E(k) = A(k) \cdot B(k) - C(k)$ . If this relation holds for  $2t + 1$  points, then this binds  $E$ . Moreover, the parties publicly reveal  $E(0)$  and verify that  $E(0) = 0$ . This guarantees that  $A(0)B(0) = C(0)$ . However, sharing  $E$  requires shares of  $O(n)$  values, and therefore must be performed on a smaller rate than the complete VSS. The key point is that we do not need a full fledged VSS here – in particular, we do not care about reconstruction. We just have to know that there is binding.

**Trivariate sharing.** To reduce the rate, we again adopt ideas from asynchronous computation [3]. We embed the coefficients of  $E(x)$  into a family of *trivariate* polynomials.

The dealer now packs the coefficients of all such polynomials into a bounded number of trivariate polynomials  $S_1, \dots, S_\ell$ , each of degree  $t + k$  in the variables  $x, y, z$ , where  $\ell \in O(1)$ . Each trivariate polynomial can hold roughly  $(k + 1)^3$  coefficients, which suffices to accommodate all the degree- $2t$  constraints for  $k \in O(n)$ . Concretely, the trivariate would be of degree  $t_s + t_a + 1$  in all three dimensions. Verifying the product relation is then reduced to checking that all these trivariate polynomials vanish at the origin, which a party can check locally (see the full version of the paper), and certifies that the dealer’s triple satisfies  $A(0) \cdot B(0) = C(0)$ .

We are left to adapt the trivariate sharing of [3] to our tighter threshold. We proceed as follows.

**Synchrony.** In the synchronous track, recall that we have  $t_s$  corrupted participants, and a total of  $n \geq 3t_s + t_a + 1$  participants. This is a higher threshold than the case of  $n \geq 4t + 1$  with  $t$  corrupted parties as in [3]— we are left with  $2t_s + t_a + 1$  honest parties which is less than  $3t + 1$ .

We therefore combine a restarting technique of [2] which is similar to the player elimination technique (i.e., excluding parties from the computation, once found as corrupted), with excluding  $O(n)$  participants at once each time. Whenever a party  $P_k$  is excluded, its share becomes some default fixed value (say 0), and the dealer restarts the entire sharing process while choosing new polynomials under the restriction that the shares of  $P_k$  are 0s. Restarting allows to reduce the broadcast cost of “revealing” shares of parties, and keep the entire broadcast costs of the protocol low. We prove that after a constant number of restarts, an honest dealer will successfully complete the sharing.

**Asynchrony.** In the asynchronous track, recall that we have to make sure that the polynomial is binding when  $t_a$  parties are corrupted. We use the asynchronous techniques of [3] to bind the polynomial even when it has a high degree in more than one variable. Ideally, if one could find a “larger” star that includes more indices, one could bind the polynomial even though it has a high degree. A major insight of [3] is that if the graph is not “dense” enough (with many parties having very high degrees), a large star can be easily found. On the other hand, if the graph is dense, then using a *trivariate* polynomial can help bind the polynomial.

We refer the reader to Section 7 for the full details.

**Organization.** The remainder of the paper is structured as follows. Section 3 reviews the system model and cryptographic primitives used throughout the paper. Section 5 presents the incomplete sharing protocol, followed by the complete sharing protocol in Section 6. In Section 4, we introduce our new asynchronous Core Set protocol, and in Section 7, we describe the triplet sharing protocol. Finally, Section 8 presents the full network-agnostic MPC protocol.

### 3 Preliminaries

In this section, we provide an overview of the key definitions and setup essential for the paper.

### 3.1 Network and Adversarial Model

We assume a fully connected network of  $n$  parties, denoted as  $\{P_1, \dots, P_n\} = \mathcal{P}$ , communicating through pairwise, point-to-point channels. Our model is *network agnostic*, meaning that it can be either synchronous or asynchronous, without parties being aware of the specific network type in operation. All parties have access to a global clock and to an upper bound  $\Delta$  on message delays for synchronous executions. If the network is *synchronous*, the parties' clocks are synchronized and messages that are sent by an honest party at time  $t$  are delivered by time  $t + \Delta$  to all honest parties. In contrast, if the network is *asynchronous*, the messages are delivered with an arbitrary but finite delay with the only guarantee that the messages are eventually delivered.

We consider a *computationally unbounded* adversary capable of corrupting a subset of the parties and causing them to behave arbitrarily during the execution of the protocol. We assume a *static* adversary, meaning that the adversary selects which parties to corrupt at the beginning of the protocol and cannot change this set during the protocol's execution. In the synchronous setting, the adversary can corrupt up to  $t_s$  parties, whereas in the asynchronous setting, up to  $t_a$  parties can be corrupted, where  $t_a < t_s$ .<sup>5</sup> For the corruption threshold, we assume  $n > 3t_s + t_a$ , which also implies that  $t_s < n/3$  and  $t_a < n/4$ , the respective bounds for secure multi-party computation in synchronous and asynchronous networks. Finally, our protocols are defined over a finite field  $\mathbb{F}$ , with  $|\mathbb{F}| > n$ .

**Bivariate Polynomials** We consider bivariate polynomials  $S(x, y)$  of degree  $d + c$  in  $x$  and  $d$  in  $y$ , defined over a finite field  $\mathbb{F}$ . We assume that  $|\mathbb{F}| > n$ , and that, to ease notation,

These polynomials take the form:

$$S(x, y) = \sum_{i=0}^{d+c} \sum_{j=0}^d s_{ij} x^i y^j,$$

where  $s_{ij} \in \mathbb{F}$  for every  $i \in \{0, \dots, d + c\}$  and  $j \in \{0, \dots, d\}$ . Each party  $P_i$  is associated with two unique univariate polynomials derived from  $S(x, y)$ :  $f_i(x) = S(x, i)$  and  $g_i(y) = S(i, y)$ . A univariate polynomial of degree  $d + c$  over  $\mathbb{F}$  is expressed as:

$$f(x) = \sum_{i=0}^{d+c} s_i \cdot x^i,$$

where each coefficient  $s_i \in \mathbb{F}$ . In our protocols, we set the degree of the polynomials to  $d = t_s$ , where  $t_s$  is the threshold of malicious parties in the synchronous setting. Finally, our protocols rely on the following claim:

**Claim 3.1** ([28, Lemma 4.28]). *Let  $n, d_1$ , and  $d_2$  be non-negative integers. Let  $f_1, \dots, f_{d_2}$  be univariate polynomials of degree  $d_1$  over  $\mathbb{F}$  and  $g_1, \dots, g_{d_1}$  be*

<sup>5</sup> Note that this does not limit the generality of the protocols, as otherwise one could always run an asynchronous protocol with the higher corruption threshold of  $t_a$ .

univariate polynomials of degree  $d_2$  such that for every  $i, j \in [n]$ ,  $f_i(j) = g_j(i)$ . Then, for every  $s \in \mathbb{F}$ , there exists a unique bivariate polynomial  $S(x, y)$  of degree  $d_1$  in  $x$  and  $d_2$  in  $y$  such that  $S(0, 0) = s$  and  $\forall i, j \in [n]$ ,  $S(x, i) = f_i(x)$ , and  $S(j, y) = g_j(y)$ .

**Trivariate Polynomials** We consider trivariate polynomials  $S(x, y, z)$  of degree  $d + c$  in  $x, y, z$ , defined over a finite field  $\mathbb{F}$ . These polynomials take the form:

$$S(x, y, z) = \sum_{i=0}^{d+c} \sum_{j=0}^{d+c} \sum_{k=0}^{d+c} s_{ijk} x^i y^j z^k,$$

where  $s_{ijk} \in \mathbb{F}$  for every  $i \in \{0, \dots, d+c\}$ ,  $j \in \{0, \dots, d+c\}$ , and  $z \in \{0, \dots, d+c\}$ . Similarly, in our protocols, we set the degree of the polynomials to  $d = t_s$ .

**Claim 3.2** ([3, Claim 3.9]). *Let  $d$  and  $c$  be nonnegative integers, Let  $H \subset [n]$  be a set of size  $d + c + 1$ , and let  $(S_h(x, y))_{h \in H}$  be  $d + c + 1$  bivariate polynomials of degree at most  $d + c$  in both  $x, y$ . Then, there exists a unique trivariate polynomial  $S(x, y, z)$  of degree  $d + c$  such that for every  $h \in H$ :*

$$S(z, y, h) = S_h(x, y)$$

**Error Correction** We use the Reed-Solomon code for error correction. This code encodes a  $d$ -degree polynomial  $p(x) = \sum_{i=0}^d a_i x^i$  over a finite field  $\mathbb{F}$  as a vector  $(p(1), \dots, p(n))$  of  $n$  evaluations of the polynomial. The Reed-Solomon decoding algorithm takes three inputs  $S, d, e$ , where  $S$  is a set of tuples of the form  $(j, y_j) \in \mathbb{F} \times \mathbb{F}$ ,  $d \in \mathbb{N}$  is a desired degree, and  $e \in \mathbb{N}$  is the number of allowed errors. The algorithm then either outputs a polynomial  $p(x)$  of degree at most  $d$  or the value  $\perp$ . If there exists a unique polynomial  $p(x)$  of degree  $d$  or less such that for at most  $e$  tuples  $(j, y_j) \in S$ ,  $p(j) \neq y_j$ , then the decoding algorithm outputs  $p(x)$ . If no such polynomial exists (that is, either no polynomial exists, or there is more than one), then the decoding algorithm outputs  $\perp$ .

Importantly, if  $|S| < d + e + 1$ , then there is no unique polynomial that disagrees with at most  $e$  tuples in  $S$  and thus the decoding algorithm outputs  $\perp$ . One can check the size of  $S$  before running any possible Reed-Solomon decoding algorithm. In addition, if  $|S| \geq d + e + m + 1$  for some  $m \leq e$  and there exists a polynomial  $p(x)$  of degree at most  $d$  such that  $p(j) \neq y_j$  for at most  $m$  tuples  $(j, y_j) \in S$ , then the decoding algorithm outputs  $p(x)$ . That is because  $p(x)$  is consistent with all but  $m \leq e$  points and thus it is one possible polynomial for which there are at most  $e$  errors. In addition  $d + e + 1$  of the tuples are consistent with  $p(x)$ , and thus if a polynomial disagrees with at most  $e$  tuples in  $S$ , it must agree with  $d + 1$  evaluations of  $p(x)$ , meaning that it also equals  $p(x)$ .

**Finding a STAR.** For an undirected graph  $\mathbb{G} = (V, E)$ , and a vertex  $v$ , we let  $\Gamma(v)$  denote the neighbors of  $v$ , i.e.,  $\{u \mid (u, v) \in E\}$ .

**Definition 3.3** ( $(n, t)$ -Extended Star). *Let  $\mathbb{G} = (V, E)$  be an undirected graph over  $[n]$  nodes, we say  $(C, D, G, F)$  is an extended-star if the following hold:*

1.  $|C| \geq n - 2t$ ,  $C \subseteq D$ , and  $|D|, |G|, |F| \geq n - t$ .
2. For every  $c \in C$  and  $d \in D$ ,  $(c, d) \in E$ .
3. For every  $g \in G$ ,  $|\Gamma(g) \cap C| \geq n - 2t$ .
4. For every  $f \in F$ ,  $|\Gamma(f) \cap G| \geq n - t$ .

Such stars have been used extensively as relaxations of cliques in the literature on secret sharing [20, 28]. Crucially, if a clique of size  $n - t$  exists in the graph  $\mathbb{G}$ , then it is easy to efficiently find an  $(n, t)$ -star in the graph:

**Lemma 3.4** ([28, Proposition 4.21]). *If there exists a clique of size  $n - t$  in an undirected graph  $\mathbb{G} = (V = [n], E)$ , then there exists an  $(n, t)$ -star  $(C, D)$  in the graph. Furthermore, there exists an efficient algorithm outputs an  $(n, t)$ -star when run on such a graph.*

**Lemma 3.5.** *Let  $I$  be a set of cardinality of size at most  $t$ . Let  $\mathbb{G}$  be an undirected graph over  $[n]$  such that for every  $j, k \notin I$ , it holds that  $(j, k) \in \mathbb{G}$ . Then,  $C \setminus I$  contains  $n - 2t$  indices and  $C \subseteq G, F$ .*

**Almost-Surely Terminating Protocols.** Informally, a property is said to hold almost-surely if it has 0 probability of not holding. Therefore, a protocol is said to be almost-surely terminating if the probability that all honest parties terminate is 1 over all possible random tapes of the honest parties. The seminal work of Fischer, Lynch and Paterson [40] showed that any asynchronous consensus protocol resilient to even one crash must have some infinite execution. Since the Byzantine Agreement (BA) protocol used in this work is also used for asynchronous executions, it must have at least one such execution. Therefore, such a protocol can only be almost-surely terminating without any guarantee that the protocol terminates for the worst possible coin-flips of the honest parties.

**Byzantine Agreement.** A Byzantine agreement protocol allows parties to all provide inputs and output an agreed-upon value. A common requirement of such protocols is that if all parties have the same value as input, they must output that value. To agree on a set of inputs to the computation, we will use an asynchronous binary Byzantine agreement protocol, which only allows the values 0 or 1 as inputs and outputs. Note that such a protocol is guaranteed to be secure in a synchronous network as well as long as the allowed corrupted thresholds are not exceeded, as the scheduling of messages in a synchronous execution is always possible in asynchronous networks as well. Formally, a binary Byzantine Agreement protocol is defined as follows:

**Definition 3.6** (Byzantine Agreement). *Let  $\Pi_{\text{BA}}$  be a protocol. Each party  $P_i$  holds an input  $b_i \in \{0, 1\}$  and may output a value in  $\{0, 1\}$ .  $\Pi_{\text{BA}}$  is said to be a  $t$ -secure binary Byzantine agreement protocol if the following properties hold when at most  $t$  parties are corrupted:*

- *Consistency*: All honest parties that output a value from the protocol output the same value.
- *Validity*: If all honest parties have the same input  $b$ , every honest party that outputs a value from the protocol output  $b$ .
- *Termination*: If the network is asynchronous, all parties almost-surely (i.e., with probability 1) output a value.

Next, we state the property-based definitions for our secret-sharing protocols:

**Definition 3.7** (Incomplete Sharing). *Let  $\Pi_{\text{IncompleteVSS}}$  be a protocol with a designated dealer  $P_i$ . Let  $P_i$  hold an input  $S \in \mathbb{F}^{\leq d, t_s}[x, y]$ , which is a bivariate polynomial in a commonly-known finite field  $\mathbb{F}$ , and of degree  $d$  or less in  $x$  and  $t_s$  or less in  $y$ , for a commonly known  $d$ . The protocol is a perfectly-secure incomplete sharing protocol if the following properties hold when the adversary corrupts at most  $t_s$  parties in a synchronous network or at most  $t_a$  parties in an asynchronous network:*

1. *Privacy*: Irrespective of the network type, when the dealer is honest, the adversary's view is identically distributed for every two polynomials that agree on the adversary's points.  
That is, for every set of corrupted parties  $I \subseteq [n]$  of cardinality at most  $t_s$ , for every two bivariate polynomials  $S, S' \in \mathbb{F}^{\leq d, t_s}[x, y]$  such that  $S(x, i) = S'(x, i)$ ,  $S(i, y) = S'(i, y)$ , then  $\text{VIEW}_{S, I} = \text{VIEW}_{S', I}$ , where  $\text{VIEW}_{X, I}$  is a random variable denoting all messages that the corrupted parties  $I$  receive in an execution of the protocol where the honest dealer uses  $X$  as input.
2. **Synchronous Network.**
  - (a)  $t_s$ -Correctness: If the dealer  $P_1$  is honest, at a fixed time  $T_{\text{IncompleteVSS}}$ , every honest party  $P_j$  outputs  $S(x, j), S(j, y)$ .
  - (b)  $t_s$ -Weak Commitment: If the dealer is corrupted, then at a fixed time  $T_{\text{IncompleteVSS}}$ , either no honest party computes an output or there exists a polynomial  $S'(x, y)$  of degree at most  $d$  in  $x$  and degree at most  $t_s$  in  $y$  such that if an honest party  $P_j$  outputs shares  $f_j(x), g_j(y)$ , then  $f_j(x), g_j(y) = S'(x, j), S'(j, y)$ . Additionally, if some honest party outputs shares at that time, at least  $t_s + t_a + 1$  honest parties output shares up to 1 round after that.
3. **Asynchronous Network.**
  - (a)  $t_a$ -Correctness: If the dealer  $P_1$  is honest and some party outputs shares, every honest party  $P_j$  eventually outputs  $S(x, j), S(j, y)$ .
  - (b)  $t_a$ -Strong Commitment: If the dealer is corrupted, then either no honest party computes an output or there exists a polynomial  $S'(x, y)$  of degree at most  $d$  in  $x$  and degree at most  $t_s$  in  $y$  such that every honest party outputs  $f_j(x), g_j(y) = S'(x, j), S'(j, y)$ .

In our protocol, we consider a commonly known degree  $d \leq t_s + t_a$ .

**Definition 3.8** (Complete Sharing). *Let  $\Pi_{\text{CompleteSharing}}$  be a protocol with a designated dealer  $P_1$ . Let  $P_1$  hold an input  $S \in \mathbb{F}^{\leq d, t_s}[x, y]$ , which is a bivariate*

polynomial in a commonly-known finite field  $\mathbb{F}$ , and of degree at most  $d$  in  $x$  and degree at most  $t_s$  in  $y$ .  $\Pi_{\text{CompleteSharing}}$  is a perfectly-secure complete sharing protocol if the following properties hold when the adversary corrupts at most  $t_s$  parties in a synchronous network or  $t_a$  parties in an asynchronous network:

1. **Privacy:** Irrespective of the network type, when the dealer is honest, the adversary's view is identically distributed for every two polynomials that agree on the adversary's points.  
That is, for every set of corrupted parties  $I \subseteq [n]$  of cardinality at most  $t_s$ , for every two bivariate polynomials  $S, S' \in \mathbb{F}^{\leq d, t_s}[x, y]$  such that  $S(x, i) = S'(x, i)$ ,  $S(i, y) = S'(i, y)$ , then  $\text{VIEW}_{S, I} = \text{VIEW}_{S', I}$ , where  $\text{VIEW}_{X, I}$  is a random variable denoting all messages that the corrupted parties  $I$  receive in an execution of the protocol where the honest dealer uses  $X$  as input.
2. **Synchronous Network.**
  - (a)  $t_s$ -**Correctness:** If the dealer is honest and has input  $S(x, y)$ , then at a fixed time  $T_{\text{CompleteSharing}}$ , all the honest parties  $P_j$  output  $f_j(x) = S(x, j)$ ,  $g_j(y) = S(j, y)$ .
  - (b)  $t_s$ -**Commitment:** If the dealer is corrupted, then either no honest party computes an output at a fixed time  $T_{\text{CompleteSharing}}$ , or there exists a polynomial  $S'(x, y)$  of degree at most  $d$  in  $x$  and degree at most  $t_s$  in  $y$  such that if an honest party  $P_j$  outputs shares  $f_j(x)$ ,  $g_j(y)$ , then  $f_j(x), g_j(y) = S'(x, j), S'(j, y)$ . Additionally, if some honest party outputs shares at that time, every honest party  $P_j$  outputs shares up to 1 round after that.
3. **Asynchronous Network.**
  - (a)  $t_a$ -**Correctness:** If the dealer is honest, and some honest party outputs shares, every honest  $P_j$  outputs  $f_j(x) = S(x, j)$ ,  $g_j(y) = S(j, y)$ .
  - (b)  $t_a$ -**Strong Commitment:** If the dealer is corrupted, then either no honest party terminates or there exists some bivariate polynomial  $S'(x, y)$  of degree  $d$  in  $x$  and degree  $t_s$  in  $y$  all honest parties  $P_j$  output  $f_j(x) = S'(x, j)$ ,  $g_j(y) = S'(j, y)$ .

### 3.2 Modeling Network Agnostic Protocols in UC

We consider a protocol that is *network agnostic*, namely, the protocol has the same “next-message functions” when the underlying network is synchronous, or asynchronous. We now show how to model such protocols in the universal-composability (UC) framework due to Canetti [29]. We rely on several papers to simplify the exposition and the analysis.

Our formalization involves the following elements:

**The protocol.** We consider a protocol  $\Pi$  between  $n$  parties. The parties communicate using a functionality (a trusted party) called **Network**, which is described below. Moreover, the system contains a global **mode** functionality that indicates whether the network is synchronous or asynchronous.

**Global mode functionality.** To model the “mode” (either synchronous/ asynchronous), we use the formalization of a global functionality, which we denote as

$F_{\text{Mode}}$ . The mode functionality is a write-once register, which indicated whether the system is in synchronous or asynchronous. The adversary can write the network state to the register, and all functionalities can read from the register. Once read, no entity can ever write to it again. This is a global functionality that has only one instance. We emphasize that only the other functionalities in the system, and the adversary have access to this functionality. In particular, the honest parties do not have access to it. This is similar to the  $F_{\text{Lib}}$  functionality as in [30].

**The global Network functionality.** The **Network** is a functionality that has two modes: Synchronous communication mode, and asynchronous communication mode. Both modes have the same interface, and parties simply communicate by **Send** command ( $P_i$  sends a message to  $P_j$  over the communication channel) and **Fetch** (read the communication channel). In particular, asynchronous communication with eventual delivery is the  $F_{\text{ED-SMT}}$  functionality, and synchronous communication is the  $F_{\text{SYN}}$  functionality, as formalized in [43].

We emphasize that the network decides between the two functionalities (either  $F_{\text{ED-SMT}}$  or  $F_{\text{SYN}}$ ) using the global  $F_{\text{Mode}}$ .

Notice, in particular, that the network functionality has the same interface and implements synchronous or asynchronous communication accordingly.

**Clocks and rounds in synchronous functionalities.** In the synchronous mode, to implement  $F_{\text{SYNC}}$  (which is inside  $F_{\text{Network}}$ ), the functionality also uses a clock, i.e., rounds are defined. Therefore, functionalities in the **sync** might also contain instructions like “in round 3, deliver to the parties the output X”, while the communication with  $F_{\text{Clock}}$ .

**Defining functionalities with a global mode.** As mentioned, a protocol  $\Pi$  has an access to  $F_{\text{Network}}$ , and the parties do not know whether they communicate in the **synch** mode, or **asynch** mode (while the adversary might know, or even determine, the mode of the computation). The same protocol might have a different input/output behavior depending on the network. In particular, the same protocol might exhibit different behaviors under different network types, but maintain the same interface. For example, synchronous MPC is computed on all honest parties’ inputs (in addition to some corrupt parties’ inputs), while asynchronous MPC is only expected to use a “large enough” set of inputs. Each party has the same interface to the functionality: sending an input, and (eventually) retrieving an output. The functionality, however, might have different interface with the adversary, depending on the mode.

As previously, this is modeled using the global  $F_{\text{Mode}}$  functionality. The functionalities check the network state and behave accordingly. Proceeding with the example of MPC, the functionality decides whether to take all inputs into consideration, or whether to take some core of parties into consideration.

**Convention.** Throughout the paper, we use that convention that each functionality has a variable **mode** which is either **synch** or **asynch**. In general, our statements are in the form of: “Protocol  $\Pi$  implements functionality  $\mathcal{F}$  in the global  $F_{\text{Network}}-F_{\text{Mode}}$ -hybrid model”.



**Non-simultaneous termination.** In some of our synchronous protocols, termination is not guaranteed to occur simultaneously: for instance, certain parties may receive their outputs (and halt) in round  $r$ , while others may only do so in round  $r+1$ . For clarity of exposition, and to cleanly separate the “input–output” behavior of functionalities from other protocol-specific aspects, we model synchronous functionalities as if they terminate simultaneously. This simplification is justified by the work of Cohen et al. [33], which shows that broadcast without simultaneous termination can be treated as an ideal broadcast. Moreover, their compiler converts protocols that rely on simultaneous-termination hybrids into ones using non-simultaneous-termination hybrids, while preserving asymptotic round complexity. We note, however, that the compiler of [33] requires functionalities to follow the structure: (1) inputs from all parties; (2) leakage to the adversary; (3) outputs. Although we do not explicitly present our functionalities in this format, it is straightforward to see that they can be expressed in this way.

**Session-ids.** For clarity we omit the specification of sessions ids (sids). We use the convention that: (1) All functionalities are always run in the background; (2) Whenever a party calls a functionality, it provides it with a session id; (3) Each functionality works only once for the same sid, and ignore a command if a previous output to that session-id was already provided; (4) We assume that a unique sid (and the same sid) is given to all parties in the main protocol, and in all invocations of sub-protocols, the calling function’s sid is appended using an agreed-upon naming convention.

**Notations Regarding Graphs** We use the standard definition of a degree in a graph: if  $\mathbb{G} = (V, E)$  is an undirected graph, then  $\deg_{\mathbb{G}}(v)$  is defined to be  $|\{u \in V \mid (u, v) \in E\}|$ , and  $\Gamma(v)$  denote the neighbors of  $v$ , i.e.,  $\{u \mid (u, v) \in E\}$ .

In addition, for an undirected graph  $\mathbb{G} = (V, E)$  and subset of vertices  $W \subseteq V$ , we define  $\mathbb{G}[W]$  to be the subgraph induced by the vertices in  $W$ . We require that our graphs always have the same set of indices for technical reasons, and thus we only remove edges that involve indices not in  $W$ . Formally, we define  $\mathbb{G}[W] = (V, E')$  where  $E' = E \cap (W \times W)$ .

### 3.3 Reliable Broadcast

The following functionality models Bracha’s reliable broadcast. This protocol is designed for asynchronous networks, but we will run it also over synchronous networks, which raises some difficulties. If the sender is honest, all parties receive the output at the same round; If the sender is corrupted, then the parties do not necessarily receive output at the same round (yet, if some party receives an output, all parties receive it in at most one round of delay). We model this using the following functionality, while we remark that in the synchronous setting, we model the non-simultaneous termination of [33]:

---

#### Functionality 3.9: Reliable Broadcast

---

In both models, the parties obtain their outputs according to Remark 3.10.

---

- **Synchronous mode:** The sender sends a message  $m$  to the functionality. The functionality delivers to all parties the same message  $m$ .
  - **Asynchronous mode:** The sender sends a message  $m$  to the functionality. Note that in the hybrid-model, the adversary is allowed to set a separate delivery delay for each party.
- 

**Remark 3.10** (Non-simultaneous output delivery). *The functionality delivers the output in the following manner:*

- **Synchronous model, honest input provider (sender/dealer):** *when the sender is honest, all parties receive outputs simultaneously in a commonly known round.*
- **Synchronous model, corrupted input provider (sender/dealer):** *the functionality has non-simultaneous termination and therefore we use the wrapper of [33] to allow the adversary to set separate delays for each party, within known upper bounds on the delay. Specifically, in this protocol and other protocols in the paper, if some honest party outputs a value, all honest parties will output a value with a delay of at most one round.*
- **Asynchronous model:** *the adversary is allowed to add a separate unbounded delivery delay for each party.*

Recall that in both models, the parties receive their outputs using a fetch command. By that we mean that when we say that the functionality sends an output to a party, it actually stores its output and sends it after receiving an appropriate fetch command.

The reliable broadcast protocols of [1, 9, 26] are all designed for the asynchronous network, and do not guarantee simultaneous termination when execution over a synchronous network. Nevertheless, by taking the properties of the reliable broadcast as in Abraham, Asharov, and Chandramouli [1], we have:

**Theorem 3.11.** *Let  $n \geq 3t + 1$ . There exists a reliable broadcast protocol that securely realizes Functionality 3.9, tolerating at most  $t$  corrupted parties over synchronous and asynchronous network. When the sender starts with a message  $m$  of size  $|m| = L$ , then the protocol runs in  $O(nL + n^2 \log n)$ .*

In the synchronous model, the protocol works in 6 rounds of interaction, but when the sender is corrupted, some honest parties might end in one round later. Therefore, we use  $T_{BC} = 6\Delta$  to denote the optimistic termination of a broadcast, i.e., the time needed for an honest dealer.

## 4 Agreement on a Core Set

Multiparty computation protocols are often comprised of an input phase, followed by agreeing on the set of inputs to use and then performing the computation on the chosen inputs. Synchronous protocols usually don't explicitly refer

to the agreement on inputs phase, but involve either verifiable secret sharing or synchronously broadcasting inputs. These primitives are known to be equivalent to consensus in synchrony, with all parties eventually receiving parties' inputs, or default values if adversarially controlled parties choose not to provide inputs.

On the other hand, in asynchrony, if the adversary instructs parties to never provide inputs, other parties cannot differentiate between the case that no input was provided and the case that messages simply haven't arrived yet. This means that parties can only hope to perform the computation on a set of inputs of size  $n - t$  in networks with  $t$  faults. Since different parties might receive different inputs at different times, they must agree on a set of inputs to use in the later stages of the MPC protocol. The task of agreeing on such a set of inputs is called *agreement on a core set* (ACS) [4, 22].

Our goal is to construct a protocol allowing parties to output a set  $S \subseteq [n]$  of parties who provided inputs to the MPC protocol. In synchrony, we require that  $S$  include all honest parties' indices, and possibly indices of corrupted parties that chose to provide inputs via the complete VSS protocol. In asynchrony, we simply require that all parties output the same set  $S \subseteq [n]$  of parties that provided inputs to the protocol. Note that this set might be too small, including fewer than  $n - t_s$  indices, which means that some honest party's index was not included. If that is the case, all parties eventually output  $S$ , detect that the network is not synchronous, and fallback to an asynchronous MPC protocol instead. Note that this means that our "ACS" protocol implements a version of synchronous ACS, but might not implement ACS as it is usually defined in asynchrony. As such, we will say the protocol implements *detectable ACS* instead, which allows parties to detect that it failed in asynchrony.

We model every party  $P_i$  as having a binary  $x_{i,j}$  input per  $P_j$  to the protocol.  $P_i$  sets  $x_{i,j}$  to be 1 if it completed the complete VSS invocation with  $P_j$  as dealer and 0 otherwise. All parties call the ACS protocol at  $T_{\text{CompleteSharing}}$ , the time at which the VSS instance of all honest dealers is guaranteed to terminate. This means that  $x_{i,j}$  will be set to 1 for every pair of honest parties  $P_i, P_j$  in synchrony.

Formally, we define the task as follows:

**Definition 4.1.** *Let  $\Pi_{\text{ACS}}$  be a protocol in which each party  $P_i$  has an input  $(x_{i,1}, \dots, x_{i,n}) \in \{0, 1\}^n$ , and may output a vector  $(y_{i,1}, \dots, y_{i,n}) \in \{0, 1\}^n$ . Assume that in synchrony,  $x_{i,j} = 1$  for every pair of honest parties  $P_i, P_j$ . The protocol is a perfectly-secure detectable agreement on a core set protocol if the following properties hold when the adversary corrupts at most  $t_s$  parties in a synchronous network or  $t_a$  parties in an asynchronous network:*

1. *Consistency: all honest parties that complete the protocol output the same value.*
2. *Validity: if some honest party  $P_i$  outputs  $(y_{i,1}, \dots, y_{i,n})$  then for every  $k \in [n]$  such that  $y_{i,k} = 1$ , there exists an honest  $P_j$  such that  $x_{j,k} = 1$ .*
3. *Termination: all honest party almost-surely complete the protocol.*

Note that this protocol can be easily implemented by running  $n$  parallel instances of Byzantine Agreement (BA), one for each index. That is,  $P_i$  sets its

input to the  $j$ 'th instance to be  $x_{i,j}$  and sets its output to be  $y_{i,j}$ . Recalling Definition 3.6, all parties agree on their outputs and almost-surely complete all  $n$  calls to the Byzantine agreement protocol. In addition, from the validity of the BA protocol, if all honest parties have the same input to an instance, they must output that value. This implies that if some party  $P_i$  output  $y_{i,j} = 1$ , some honest party input 1 to the  $j$ 'th instance.

In order to deal with either network type, we simply run *asynchronous* BA resilient to at most  $t < n/3$  faults. We use the asynchronous parallel binary Byzantine Agreement protocol of Cohen et al. [34]. However, their protocol is only statistically secure due to using a statistically secure VSS protocol in the construction of a weak common coin. Note that the entire rest of the protocol is perfectly secure, and thus replacing the coin construction with a perfectly secure coin yields a perfectly secure parallel BA protocol. Therefore, we use the asynchronous coin construction of Bangalore et al. [15]. The coin is resilient to  $t$  faults, assuming  $n > (3+\epsilon)t$  for some  $\epsilon > 0$  and has  $O(\frac{1}{\epsilon}n^4 \log n)$  communication complexity and constant round complexity. Setting  $\epsilon = \frac{t_a}{t_s}$ , we get a communication complexity of  $O(n^4 \log n)$  as both  $t_a, t_s = \Theta(n)$ , and thus  $\epsilon = O(1)$ . Note that since  $n > 3t_s + t_a$ , the bound  $n > (3 + \frac{t_a}{t_s})t_s = (3+\epsilon)t_s > (3+\epsilon)t_a$  holds, and thus the protocol is resilient enough in either network type. In addition, the coin may fail up to  $O(\frac{1}{\epsilon}) = O(1)$  times, after which it is guaranteed to correctly work as a weak coin. This has the effect of increasing the expected round complexity of the agreement protocol by a constant without affecting the asymptotic communication complexity, as previously observed in [6, 15]. The rest of the parallel BA protocol of [34] also has  $O(n^4 \log n)$  expected communication complexity and  $O(1)$  expected round complexity, with a constant expected number of calls to the coin protocol, resulting in this complexity for the entire protocol.

## 5 Incomplete Verifiable Secret Sharing

The following primitive is a variant of verifiable secret sharing. The dealer distributes packed secrets (specifically,  $t_a + 1$  secrets). This is a weaker variant of our complete VSS as it does not ensure completeness in synchrony. Specifically, our protocol achieves the following properties:

1. In asynchronous networks, all honest parties eventually receive verified shares. I.e., this is a complete VSS.
2. In a synchronous network, it is *only* guaranteed that at least  $t_s + t_a + 1$  honest parties would have verified shares. We model this in the UC, by having the dealer sending to the incomplete VSS functionality (Func 5.1) which parties receive the output. Moreover, it can later decide to deliver to other parties shares on the same polynomial.

### 5.1 The Functionality

---

#### Functionality 5.1: Incomplete VSS

---

**1. Honest dealer:**

- (a) The dealer sends to the functionality the command  $(\text{input}, S)$ , where  $S$  is a bivariate polynomial  $S(x, y)$  of degree  $d$  in  $x$  and  $t_s$  in  $y$ . The functionality sets  $K = [n] \setminus I$ . The functionality stores  $(S, K)$ .
- (b) The functionality sends  $S(x, i), S(i, y)$  to the adversary for every  $i \in I$ .

**2. Dishonest dealer:**

- (a) The dealer sends to the functionality the command  $(\text{input}, S, K)$ , where  $S$  is a bivariate polynomial  $S(x, y)$ , and  $K \subseteq [n] \setminus I$ .
- (b) The functionality verifies that  $S(x, y)$  is of degree  $d$  in  $x$  and  $t_s$  in  $y$ , and that  $|K| \geq t_s + t_a + 1$  and otherwise, replaces it with  $S(x, y) = K = \perp$ . The functionality stores  $(S, K)$ .

**3. Output:**

- (a) **Synchronous model:** the functionality sends  $S(x, j), S(j, y)$  for every  $j \in K$  (non-simultaneously, see Remark 3.10). The adversary can additionally send a command  $(\text{allowDelivery}, j)$ . In this case, if  $(S, K)$  are stored, and  $j \notin K$ , then  $j$  is added to  $K$ .
- (b) **Asynchronous model:** The functionality ignores  $K$  (the VSS is complete in asynchrony), and sends  $S(x, j), S(j, y)$  to  $P_j$  for every  $j \in [n] \setminus I$ . Recall that the adversary is allowed to set a separate delivery delay for each party, see Remark 3.10.

The following protocol implements this functionality both in the synchronous and asynchronous models. In the asynchronous model (note that in this case, the number of corruptions is at most  $t_a$ ), the protocol implements a stronger variant of the functionality, in which the adversary must send  $K = [n] \setminus I$ , i.e., all parties receive valid shares.

**5.2 The Protocol**

The protocol is implemented in the Reliable-broadcast-hybrid model (as described in Section 3.3). Recall that by our convention, we simply write “the parties broadcast a message  $m$ ” and do not use directly the functionality. Moreover, we assume that all parties pull broadcast messages at the beginning of each round, and we omit this instruction as well from the protocol. Instead, we can just think as parties receiving broadcast messages in different schedule (with limitations described in the reliable-broadcast functionality).

**Protocol 5.2:**  $\Pi_{\text{IncompleteVSS}}$ 

**Input.** The dealer  $P_1$  has input  $(\text{input}, S)$  for  $S \in \mathbb{F}^{\leq d, t_s}[x, y]$ . All parties have a commonly known degree  $d \leq t_s + t_a$ .

**The protocol.**

1. **Round I – Share dissemination (Time 0):** Dealer  $P_1$ :
  - (a) Send  $f_i(x) = S(x, i), g_i(y) = S(i, y)$  to each party  $P_i$ .

2. **Round II – Pair-wise checks (Time  $\Delta$ ):** Each party  $P_i$ :
    - (a) If received  $f_i(x), g_i(y)$  from the dealer such that  $f_i(x)$  is of degree at most  $d$  and  $g_i(y)$  is of degree of at most  $t_s$ , send  $a_{ij} = f_i(j), b_{ij} = g_i(j)$  to each party  $P_j$ .
  3. **Broadcast Round I – Results of pair-wise checks (Time  $2\Delta$ ):** Each party  $P_i$ :
    - (a) For every party  $P_j$ :
      - i. If no message was received from  $P_j$ , set  $m_{i,j} = \perp$
      - ii. If a message  $(a_{ji}, b_{ji})$  was received from  $P_j$  such that  $(a_{ji}, b_{ji}) = (g_i(j), f_i(j))$ , set  $m_{i,j} = (\text{OK}, i, j)$
      - iii. If a message was received from  $P_j$  such that the above does not hold, set  $m_{i,j} = (\text{NOK}, f_i(j), g_i(j))$ .
    - (b) Broadcast  $(m_{i,1}, \dots, m_{i,n})$ .
  4. **Broadcast Round II – Extended Star Computation (Time  $2\Delta + T_{\text{BC}}$ ):** Dealer  $P_1$ :
    - (a) Construct a graph  $\mathbb{G} = (V = [n], E = \phi)$ .
    - (b) Add an edge  $(i, j)$  to  $E$ , if  $P_i$  and  $P_j$ 's  $(m_{i,1}, \dots, m_{i,n})$  and  $(m_{j,1}, \dots, m_{j,n})$  broadcasts were received by the end of the previous round such that  $m_{i,j} = m_{j,i} = \text{OK}$ .
    - (c) Remove all edges incident with  $P_i$  if  $P_i$  broadcast  $m_{i,j} = (\text{NOK}, a_{ij}, b_{ij})$  such that  $(a_{ij}, b_{ij}) \neq (S(j, i), S(i, j))$  for any  $j \in [n]$ .
    - (d) Define  $W = V$ . Remove  $i$  from  $W$  if  $\deg_{\mathbb{G}[W]}(i) < n - t_s$ . Repeat this until no more parties can be removed from  $W$ .
    - (e) If an  $(n, t_s)$  extended star  $(C, D, G, F)$  exists in the graph  $\mathbb{G}[W]$ , then broadcast  $(W, C, D, G, F)$ .
  5. **Intrusion Detection (Time  $2\Delta + 2T_{\text{BC}}$ ):** Each  $P_i$ :
    - (a) Construct the graph  $\mathbb{G}_i = (V = [n], E)$  as specified in Step 4b.
    - (b) If  $(W, C, D, G, F)$  is broadcast by the dealer by the end of the previous round, then check if the following holds:
      - i. There exist no  $j, k \in W$ , such that  $P_j, P_k$  broadcast  $m_{j,k} = (\text{NOK}, a_{jk}, b_{jk}), m_{k,j} = (\text{NOK}, a_{kj}, b_{kj})$  respectively by the end of the pair-wise checks round, where  $(a_{jk}, b_{jk}) \neq (b_{kj}, a_{kj})$ .
      - ii. For each  $j \in W$ ,  $\deg_{\mathbb{G}_i[W]}(j) \geq n - t_s$ .
      - iii.  $(C, D, G, F)$  is a  $(n, t_s)$  extended star in  $\mathbb{G}_i[W]$ .
    - (c) If the checks pass, accept the graph.
- 
6. **Asynchronous output computation.** Each party  $P_i$  does the following *asynchronously upon* (a) time  $2\Delta + 2T_{\text{BC}}$  passing; and (b) receiving a  $(W, C, D, G, F)$  broadcast from the dealer that passes the checks in Step 5:
    - (a) If  $i \in G$ , record the value  $g_i(y)$  received from the dealer. If  $i \in F$ , record the value  $f_i(x)$  received from the dealer.
    - (b) If  $i \notin G$ , consider the values  $(a_{ji}, b_{ji})$  received from  $P_j$  such that  $j \in F$  and apply Reed-Solomon decoding to compute a unique degree  $t_s$  polynomial  $g_i(y)$  such that  $g_i(j) = a_{ji}$  for all but  $t_s$  parties  $j \in F$ . If

- no such polynomial exists, wait for more messages. If such a polynomial exists, record  $g_i(y)$ , and send  $(\text{reconstruct}, i, k, g_i(k))$  for each  $k \notin F$ .
- (c) If  $i \notin F$ , consider the points  $(j, b_{ji})$  received from parties in  $j \in G$ , or points  $(j, b_{ji})$  received from messages  $(\text{reconstruct}, j, i, b_{ji})$  from  $j \notin G$  and use Reed Solomon decoding to compute a unique degree  $d$  polynomial  $f_i(x)$ . If no such polynomial exists, wait for more messages. If such a polynomial exists, record  $f_i(x)$ .
- (d) If both  $f_i(x)$  and  $g_i(y)$  have been recorded, output  $(f_i(x), g_i(y))$  upon receiving a  $(\text{fetch})$  command.
- 

**The round complexity in synchronous network  $T_{\text{IncompleteVSS}}$ .** Over synchronous networks, the protocol terminates after 3 rounds + 2 broadcast rounds if the dealer is honest. As we will use this protocol as a sub-protocol in other protocols (looking ahead, Section 6), we denote by  $T_{\text{IncompleteVSS}} = 3\Delta + 2T_{\text{BC}}$ , where  $\Delta$  is a bound on a synchronous round, and  $T_{\text{BC}}$  is a bound on the time needed for broadcast.

### 5.3 Efficiency and Batching

When considering a single instance of  $\Pi_{\text{IncompleteSharing}}$ , we can see that the dealer sends  $O(n)$  polynomials over the field, each of size field elements of size  $O(n \log n)$  in share dissemination round, each party sends  $O(n)$  field elements when performing the pair-wise checks, and then both the dealer and participants each broadcast a constant number of vectors of size  $O(n \log n)$ . Using a broadcast protocol with complexity  $O(n^3 \log n + nL)$  for messages of size  $L$ , we get a total complexity of  $O(n^4 \log n)$  for sharing  $O(n)$  secrets.

However, parties can also batch many simultaneous calls to the protocol with the same dealer. When calling  $\ell$  instances to the protocol for sharing  $\ell n$  secrets, the dealer will send  $O(\ell n)$  field elements of size  $O(\log n)$ , and each party will send  $O(\ell n)$  field elements while performing the consistency checks. Then, instead of sending a message  $m_{i,j}$  for each of the sharings separately, parties can send a single message. This is done in an analogous manner to the way OK messages are sent in the full description of the protocol and following the logic of [2].

- If no message is received from  $P_j$  in any of the  $\ell$  instances,  $P_i$  sets  $m_{i,j} = \perp$ .
- If values  $(a_{j,i}, b_{j,i})$  are received from  $P_j$  in all  $\ell$  instances such that  $(a_{j,i}, b_{j,i}) = (g_i(j), f_i(j))$ , then  $P_i$  sets  $m_{i,j} = \text{OK}$ .
- If values  $(a_{j,i}, b_{j,i})$  are received from  $P_j$  in all  $\ell$  and for at least one of these pairs  $(a_{j,i}, b_{j,i}) \neq (g_i(j), f_i(j))$ ,  $P_i$  sets  $m_{i,j} = (\text{NOK}, f_i(j), g_i(j))$  for the instance with the minimal index with inconsistent values.

Note that exactly like in the regular protocol, for any pair of honest parties, parties send OK if all of their received shares are consistent, and send NOK for the same minimal index with inconsistent values. This means that for an honest dealer in synchrony, all honest parties are guaranteed to send OK about

each other. On the other hand, if a dishonest dealer chooses to send inconsistent values to honest parties in any of the instances, these parties will send a NOK message about each with inconsistent values in synchrony and either send such a message about each other or  $\perp$  if they do not hear from each other in asynchrony. In both cases, the protocol above remains secure, with no edge existing between parties if they claim they are inconsistent even in one instance. Now, instead of constructing a separate graph for each instance and broadcasting it, the dealer constructs a single graph and broadcasts it.

In total, this means that for sharing  $O(Xn)$  secrets, the complexity of the batched protocol is  $O((n^4 + Xn^2) \log n)$ , or  $O(n \log n)$  per secret.

#### 5.4 Security Analysis of Incomplete VSS

In this section, we provide the security proof of protocol 5.2, following Definition 3.7 and the simulation proof with respect to Functionality 5.1.

**Theorem 5.3.** *Assume that  $n > 3t_s + t_a$ , Protocol 5.2 perfectly implements Functionality 5.1 in the presence of at most  $t_s$  corrupted parties over a synchronous network, and  $t_a$  corrupted parties over an asynchronous network.*

*Proof.* The theorem is proven using the following for claims:

1. The case of a synchronous network,  $t_s$ -corruption, and an honest dealer (Claim 5.4).
2. The case of a synchronous network,  $t_s$ -corruption, and a corrupted dealer (Claim 5.7).
3. The case of an asynchronous network,  $t_a$ -corruption, and an honest dealer (Claim 5.10).
4. The case of an asynchronous network,  $t_a$ -corruption, and a corrupted dealer (Claim 5.12).

Note that  $\mathcal{S}$  also simulates the reliable-broadcast functionality; this means that whenever an honest party broadcasts a message, then this message is delivered by the end of the round in the synchronous model. Moreover, whenever a corrupted party broadcasts a message, the simulator receives from the adversary, together with the message to broadcast, also the delay vector (as formalized in the reliable-broadcast functionality). It delivers the messages to the honest parties according to the delays (again, in the synchronous model). In the asynchronous model, it behaves the same, but recall that the adversary can further delay messages via the network functionality.  $\square$

#### Synchrony: The Case of an Honest Dealer

**Claim 5.4** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 5.2 perfectly implements Functionality 5.1 in the presence of at most  $t_s$  corrupted parties over a synchronous network, assuming that the dealer is honest.*



*Proof.* In this case, the honest dealer sends its polynomial  $S(x, y)$  to the functionality. The functionality delivers to the simulator the shares of the corrupted parties  $S(x, i), S(i, y)$ . The simulator  $\mathcal{S}$  works as follows.

1. Receive from the trusted party the shares  $(f_i(x), g_i(y)) = (S(x, i), S(i, y))$  of every corrupted party  $P_i$ .
2. Choose an arbitrarily polynomial  $S'(x, y)$  of degree at most  $d$  in  $x$  and degree at most  $t_s$  in  $y$ , such that for every  $i \in I$ ,  $S'(x, i) = f_i(x)$  and  $S'(i, y) = g_i(y)$ .
3. Run the protocol honestly with the adversary  $\mathcal{A}$ , simulating the honest parties as described in the protocol, while the dealer uses  $S'(x, y)$  as input (and the other honest parties have no input).
4. Output the output of the adversary.

We claim that in the synchronous setting, the ideal execution is identical to the real execution. We show that using a hybrid argument:

- **Hyb<sub>0</sub>:** This is the ideal execution. The environment sees the output of the ideal honest parties and the simulator  $\mathcal{S}$ . Note that the environment chooses the input  $S(x, y)$  of the honest dealer, and the output of all parties is determined by  $S$ .
- **Hyb<sub>1</sub>:** In this execution, we assume that the simulator also receives  $S(x, y)$  from the functionality. When simulating the protocol, it uses  $S(x, y)$  as the input of the dealer. Note that the output of the honest parties in the ideal execution is determined by the trusted party.
- **Hyb<sub>2</sub>:** This is the real execution. In particular, the output of the honest parties is determined by the protocol, and not by the functionality.

**Hyb<sub>0</sub> and Hyb<sub>1</sub> are identically-distributed.** We claim that the view of the environment in Hyb<sub>0</sub> and Hyb<sub>1</sub> is identical. The only difference between the two hybrids is the view of the adversary. All parties are deterministic. In Hyb<sub>0</sub> it runs in an execution where the dealer uses  $S(x, y)$  as input. In Hyb<sub>1</sub> it runs in an execution where the dealer uses  $S'(x, y)$ . However, for every  $i \in I$ , it holds that  $S(x, i) = S'(x, i)$  and  $S(i, y) = S'(i, y)$ . As follows from the privacy property in Definition 3.7, the view of the adversary is distributed identically in those two executions (see Lemma 5.6).

**Hyb<sub>1</sub> and Hyb<sub>2</sub> are identically-distributed.** The difference is that in Hyb<sub>1</sub>, the outputs of the honest parties are determined by the functionality, whereas in the real execution, they are determined by the protocol. In both executions, the protocol is executed using the same input for the honest dealer – a polynomial  $S(x, y)$ . As follows from the correctness property in Definition 3.7 (see Lemma 5.5), the output of all honest parties in the real protocol is exactly  $f_j(x) = S(x, j)$  and  $g_j(y) = S(j, y)$ , which is exactly the output as determined by the functionality.  $\square$

In what follows, we prove each one of the properties of the protocol (5.2), following Definition 3.7.

**Lemma 5.5** ( $t_s$ -Correctness). *If the dealer is honest and the network is synchronous, at a fixed time  $T = 3\Delta + 2T_{\text{BC}}$ , all the honest parties  $P_j$  output  $f_j(x) = S(x, j), g_j(y) = S(j, y)$ .*

*Proof.* Let  $I$  denote the set of malicious parties. Since the network is synchronous and the dealer is honest, then:

1. By the end of round 1, each honest party  $P_j$  receives polynomials  $f_j(x) = S(x, j)$  and  $g_j(y) = S(j, y)$ .
2. In round 2, each honest party  $P_j$  sends the points  $(a_{ji}, b_{ji}) = (S(j, i), S(i, j))$  to each party  $P_i$ , which honest parties receive by the end of the round.
3. Each honest party  $P_j$  receives consistent points from honest parties  $P_k$ , i.e.  $P_k$  sends  $(a_{kj}, b_{kj}) = (S(j, k), S(k, j)) = (b_{jk}, a_{jk})$ . Following that,  $P_j$  sets  $m_{j,k} = \text{OK}$  for every honest  $P_k$  and broadcasts  $(m_{j,1}, \dots, m_{j,n})$ . All honest parties, including the dealer, receive these broadcasts by the end of the broadcast round. Furthermore, for each honest  $P_j$ , if  $m_{j,k} = (\text{NOK}, a_{ij}, b_{ij})$  then  $(a_{ij}, b_{ij}) = (S(j, i), S(i, j))$ .

We now have that the honest parties form a clique in the graph  $\mathbb{G}$  constructed by the honest dealer in round 4. Since there are at least  $n - t_s$  honest parties, this clique is of size  $n - t_s$ , and  $\deg_{\mathbb{G}}(i) \geq n - t_s$  holds for every honest  $P_i$ . This means that no honest party's index is removed from  $W$  in the first iteration. Following similar logic, the honest parties' indices form a clique in  $\mathbb{G}[W]$  in each subsequent iteration, meaning that no honest parties are removed from  $W$ . Hence, the honest parties form a clique even in the graph  $\mathbb{G}[W]$ , and a valid extended star  $(C, D, G, F)$  must be found. From Lemma 3.5, the set  $C$  contains at least  $n - 2t_s$  indices of honest parties from the clique of honest parties, and the indices of all honest parties are included in  $G, F$ . The rest of the rounds proceed as follows:

4. The dealer broadcasts the found star at the beginning of round 4, and all honest parties receive the message by the end of the round.
5. Since the dealer is honest, each party in  $W$  has sufficiently large degree (otherwise they would have been removed) and  $(C, D, G, F)$  must be a valid  $(n, t_s)$  extended star in each honest party  $P_j$ 's graph  $\mathbb{G}_j[W]$ , as they receive the same OK messages as the dealer up to a round later and thus have the same edges in  $E$ . Note that if some vertex  $j \in [n]$  has a non-zero degree in the dealer's graph, then the dealer received its  $(m_{j,1}, \dots, m_{j,n})$  broadcast, including all OK and NOK messages. This means that no honest party will see two indices  $j, k \in W$  such that  $P_j$  and  $P_k$  broadcast conflicting  $m_{j,k} = (\text{NOK}, a_{jk}, b_{jk})$  and  $m_{k,j} = (\text{NOK}, a_{kj}, b_{kj})$ , as the dealer must will have received the same messages and removed at least one in round 4.
6. Each honest party  $P_j$  will see that  $j \in G \cap F$ , record  $g_j$  and  $f_j$  and output  $(f_j(x), g_j(y))$  at time  $2\Delta + 2T_{\text{BC}}$ .

□

**Lemma 5.6** ( $t_s$ -Privacy). *If the dealer is honest, then irrespective of the network type, then for any  $S, S' \in \mathbb{F}^{\leq d}[x]$  such that  $(S(x, i), S(i, y)) = (S'(x, i), S'(i, y))$  for every corrupt  $P_i$ ,  $\text{VIEW}_S = \text{VIEW}_{S'}$ .*

*Proof.* In the beginning of the protocol, the dealer sends every party  $P_i$  its shares  $f_i(x), g_i(y)$ . Every corrupt party receives the same shares, assuming  $f_i(x) = S(x, i) = S'(x, i)$ , and  $g_i(y) = S(i, y) = S'(i, y)$ . Following that, every honest  $P_j$  sends the values  $f_j(k), g_j(k)$  to every party  $P_k$ . The adversary receives these values for every corrupt  $P_i$ , and by assumption  $f_j(i) = S(i, j) = S'(i, j)$ ,  $g_j(i) = S(j, i) = S'(j, i)$ . If an honest party broadcasts NOK, then it uses its own shares. An honest party  $P_j$  broadcasts NOK on some  $P_k$  only after receiving a wrong message. Thus, honest parties never complain on one another. All possible complaints of honest parties would be on corrupted parties, and thus the adversary already has those sub-shares.

We conclude that all the messages that the adversary sees depend on the values it sends to the honest parties in the exchange phase, and its complaints. All of those are identical between the executions of  $S$  and  $S'$ , and thus the view of the adversary is exactly the same in both executions.  $\square$

### Synchrony: The Case of a Corrupted Dealer

**Claim 5.7** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 5.2 perfectly implements Functionality 5.1 in the presence of at most  $t_s$  corrupted parties over a synchronous network, assuming that the dealer is corrupted.*

*Proof.* We describe the simulator  $\mathcal{S}$ :

1. The simulator simulates an execution of the honest parties (who have no input), with the adversary  $\mathcal{A}$ . Since the honest parties have no input, the view of the adversary is identical to the real.
2. Once the first honest party outputs a value, let  $K \subseteq [n] \setminus I$  be the set of honest parties guaranteed to output values, as defined in Lemma 5.9. Note that this is the set  $F \cap G$  received from the dealer's broadcast. In addition, set  $S$  to be the polynomial defined in Lemma 5.9, computed from the views of the honest parties at that time.
3. Continue to run the protocol. Whenever an additional simulated honest party  $P_j$  such that  $j \notin K$  stores shares  $f_j, g_j$ , the simulator sends  $\text{allowDelivery}(j)$  to the functionality.

The view of the adversary is clearly identical between the real and ideal, as the simulator simulates honest parties that have no inputs. We now show that the output of the environment in the ideal execution is identical to its output in the real. This follows directly from Lemma 5.9.  $\square$

**Lemma 5.8** (Binding). *Assume the network is synchronous and an honest party accepts the dealer's graph  $(W, C, D, G, F)$  in round 5. Then, there exists a unique*

bivariate polynomial  $S'$  with degree  $d$  in  $x$  and degree  $t_s$  in  $y$  such that for every honest  $P_j$  such that  $j \in W$ ,  $P_j$  received the polynomials  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$  from the dealer and stored them.

*Proof.* Let  $P_i$  be an honest party that accepts the dealer's graph  $(W, C, D, G, F)$  in round 5, then from Step 5, the following must hold for  $P_i$ :

1. There are no  $j, k \in W$ , such that  $P_i$  received NOK messages with differing values from  $P_j$  and  $P_k$  by round 4.
2. The graph is consistent with the  $\mathbb{G}_i$  graph constructed using the OK/NOK messages received by round 4.
3. Each party whose index is in  $W$  has sufficiently large degree ( $\geq n - t_s$ ).
4.  $(C, D, G, F)$  is a valid  $(n, t_s)$  extended star in graph  $\mathbb{G}[W]$ .

Assume, by contradiction, that there exist some honest parties  $P_j, P_k$  such that  $j, k \in W$  that are not pairwise consistent, meaning  $f_j(k) \neq g_k(j)$  or  $f_k(j) \neq g_j(k)$ . First note that every  $j \in W$  has nonzero degree, and thus  $P_j$  must have sent a vector  $(m_{j,1}, \dots, m_{j,n})$ , which are only sent if  $P_j$  receives polynomials  $f_j(x), g_j(y)$  from the dealer at the end of round 1, of degrees  $d$  and  $g_j(y)$  respectively. After receiving their polynomials,  $P_j$  and  $P_k$  would have received each other's messages at the end of round 2, seen that they are inconsistent, and would have set  $m_{j,k}, m_{k,j}$  to be NOK messages with differing values in round 3. Since  $P_i$  accepted a graph with  $j, k$  having non-zero degree, it received the broadcasts sent by  $P_j, P_k$ . This means that it would have seen the contradicting NOK messages sent by  $P_j, P_k$ , contradicting the fact that it accepted the graph.

Next, we demonstrate that the shares of all honest parties whose indices are in  $W$  lie on a unique bivariate polynomial. From 3, every honest  $P_j$  such that  $j \in W$  has at least  $n - 2t_s \geq t_s + t_a + 1 \geq d + 1$  honest neighbors in  $W$ . These neighbors are pairwise consistent. According to Lemma 3.1, the polynomials  $f_j(x)$  and  $g_k(y)$  for every honest  $P_j, P_k$  such that  $j, k \in W$  lie on a unique bivariate polynomial  $S'(x, y)$ , where  $f_j(k) = g_k(j) = S'(k, j)$  and  $f_k(j) = g_j(k) = S'(j, k)$ .  $\square$

**Lemma 5.9** ( $t_s$ -Weak Commitment). *Let  $I$  denote the set of corrupted parties. If the dealer is corrupted and the network is synchronous, then at the time  $T_{\text{IncompleteVSS}}$ , either (a) no honest party computes an output; or (b) there exists a set  $K \subseteq [n] \setminus I$  of cardinality at least  $t_s + t_a + 1$ , and a unique polynomial  $S'(x, y)$  of degree at most  $d$  in  $x$  and  $t_s$  in  $y$ , such that:*

1. For every  $j, l \in K$ ,  $P_j$  outputs  $(f_j(x), g_j(y)) = (S'(x, j), S'(j, y))$  at time  $T_{\text{IncompleteVSS}}$ ;
2. For every  $j \notin K \cup I$ , if an honest  $P_j$  outputs  $(f_j(x), g_j(y))$  after time  $T_{\text{IncompleteVSS}}$ , then  $(f_j(x), g_j(y)) = (S'(x, j), S'(j, y))$ .

*Proof.* If no honest party computes an output, then the weak commitment holds trivially. Therefore, we consider the scenario where there exists an honest party  $P_j$  that outputs shares  $f_j(x), g_j(y)$  at time  $T_{\text{IncompleteVSS}}$ . Party  $P_j$  outputs its shares after receiving and accepting the dealer's  $(W, C, D, G, F)$ . Therefore,  $P_j$  must have verified that it received a set  $W$  and an extended star  $(C, D, G, F)$

from the dealer by the end of round 4, and constructed a graph  $\mathbb{G}_j$  using the OK/NOK messages received from all parties 3. Furthermore,  $P_j$  verifies that each party in  $W$  has sufficiently large degree ( $\geq n - t_s$ ) in  $\mathbb{G}_k[W]$  and that  $(C, D, G, F)$  is a valid  $(n, t_s)$ -extended star in  $\mathbb{G}_j[W]$ .

All honest parties receive the same OK/NOK messages as party  $P_j$  at most  $\Delta$  time after  $P_j$  did, as well as the message  $(W, C, D, G, F)$  sent by the dealer. Note that for any  $k \in W$ ,  $P_j$  saw that it has non-zero degree, and thus it received a broadcast  $(m_{k,1}, \dots, m_{k,n})$  from  $P_k$ . This means that there is no pair  $k, l \in W$  such that  $P_k$  and  $P_l$  sent conflicting NOK messages, as  $P_j$  would have rejected the graph sent by the dealer. Following that, every honest party will accept the dealer's graph as well, seeing as the same conditions will hold.

Let  $S'(x, y)$  be the polynomial defined in Lemma 5.8. We will show that every honest  $P_j$  that outputs shares from  $\Pi_{\text{IncompleteSharing}}$  outputs  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$ . From Lemma 5.8, every honest  $P_j$  such that  $j \in W$  receives the polynomials  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$  from the dealer and stores these polynomials. Every honest party  $P_j$  such that  $j \in G \cap F \subseteq W$  simply records outputs  $f_j(x) = S'(x, j)$ ,  $g_j(y) = S'(j, y)$  after receiving the aforementioned broadcasts in at most  $T_{\text{BC}}$  time and outputs these values, as required. Note that  $|G \cap F| \geq n - t_s$ , and thus the number of honest indices in the set is at least  $n - 2t_s \geq t_s + t_a + 1$ , and thus a large enough number of parties output shares consistent with  $S'$ . It is left to show that any honest party  $P_j$  such that  $j \notin G \cap F$  either doesn't output anything, or outputs  $f_j(x) = S'(x, j)$ ,  $g_j(y) = S'(j, y)$  as well.

1. For  $g_j(y)$ : Since  $j \notin G \cap F$ ,  $P_j$  constructs  $g_j(y)$  from received points  $f_k(j)$  for each party  $P_k$  such that  $k \in F$ . At that time, it had already received all messages sent by parties whose indices are in  $G \cap F$ . As shown above, there are at least  $t_s + t_a + 1$  honest indices in  $G \cap F$ .  $P_j$  either doesn't output anything, or finds a unique polynomial  $g_j(y)$  of degree  $t_s$  or less such that  $g_j(k) = f_k(j)$  for all but  $t_s$  parties  $j \in F$  from whom it received messages. If  $P_j$  doesn't output anything, the property holds. Now, assume that it outputs  $g_j(y)$ .  $P_j$  received at least  $t_s + t_a + 1$  values  $f_k(j) = S'(j, k)$  from the honest parties whose indices are in  $G \cap F$ , and no honest party sends a value that is not on  $S'(j, y)$ . Since  $P_j$  receives at most  $t_s$  points that are not on  $S'(j, y)$  from the corrupt parties,  $S'(j, y)$  is one possible polynomial that agrees with all but at most  $t_s$  points. By assumption,  $P_j$  outputs the unique  $g_j(y)$  for which this holds, and thus  $g_j(y)$  must be  $S'(j, y)$ , as required.
2. For  $f_j(x)$ : As shown above, every honest party  $P_j$  such that  $j \in G$  has  $g_j(y) = S'(j, y)$ , and there are  $t_s + t_a + 1$  such parties. In addition, every party  $P_j$  such that  $j \notin G$  that sent a reconstruct message reconstructed  $g_j(y)$  as described in the previous bullet, and thus also has  $g_j(y) = S'(j, y)$ . If  $P_j$  doesn't output anything, the property holds. Now, assume that it outputs  $f_j(x)$ . As before,  $P_j$  received at least  $t_s + t_a + 1$  values  $g_k(j) = S'(k, j)$  from the honest parties whose indices are in  $G \cap F$ , and no honest party sends a value that is not on  $S'(x, j)$ .  $P_j$  receives at most  $t_s$  points that are not on  $S'(x, j)$  from the corrupt parties, and thus  $S'(x, j)$  is one possible polynomial

of degree  $d \leq t_s + t_a$  or less that agrees with all but at most  $t_s$  points. By assumption,  $P_j$  output the unique  $f_j(x)$  for which this holds, and thus  $f_j(x)$  must be  $S'(x, j)$ .

□

**Proofs for Asynchronous Execution** We now analyze the protocol when it runs over an asynchronous network. We show that if some honest party decides to terminate, then it must have been that the adversary delivers sufficient messages so that it behaves close to synchronous network, and the protocol satisfies some “good” properties.

**The case of an honest dealer.** For the case of an honest dealer, we have the following claim:

**Claim 5.10** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 5.2 perfectly implements Functionality 5.1 in the presence of at most  $t_a$  corrupted parties over an asynchronous network, assuming that the dealer is honest.*

*Proof.* When the dealer is honest, then it sends to the functionality its polynomial  $S(x, y)$ . The functionality sends the shares  $(S(x, i), S(i, y))$  to the adversary for every corrupted party  $P_i$ , and also sends each honest party  $P_j$  its output share  $S(x, j), S(j, y)$ . It then runs the protocol with the adversary, while the simulated honest dealer uses the polynomial  $S'(x, y)$  satisfying  $S'(x, i) = S(x, i)$  and  $S'(i, y) = S(i, y)$  for every corrupted parties  $P_i$ . Whenever an honest party  $P_j$  writes to its output tape in this simulated execution, the simulator allows the delivery of the output  $S(x, j), S(j, y)$  to the honest party  $P_j$  in the ideal execution.

To show that the executions are perfectly distributed, we relied in the synchronous setting on privacy (Lemma 5.6), which allows the simulator to work with alternative input  $S'(x, y)$  that is consistent with the shares of the corrupted parties and not with  $S(x, y)$  that the dealer actually used in the execution. We rely on the same Lemma here as well. In the synchronous case, we also relied on Lemma 5.5 for correctness, showing that if the dealer is honest and uses some polynomial  $S(x, y)$  then each honest party outputs  $S(x, j), S(j, y)$ . Here, we rely here on the correctness for the asynchronous setting, i.e., Lemma 5.11, which just shows that each party eventually outputs those values. □

**Lemma 5.11** ( $t_a$ -Correctness). *If the dealer was honest and some honest party outputs shares, all honest parties  $P_j$  almost-surely output  $f_j(x) = S(x, j), g_j(y) = S(j, y)$ .*

*Proof.* Assume that the dealer is honest and that some honest party outputs shares. In that case, it receives a broadcast  $(W, C, D, G, F)$  from the dealer by time  $2\Delta + 2T_{BC}$ . The dealer sends such a message after constructing a graph  $\mathbb{G}$  from the OK and NOK messages, repeatedly removing indices with low degree from  $W$  and finding an  $(n, t_s)$ -extended star  $(C, D, G, F)$  in  $\mathbb{G}[W]$ . Every honest party  $P_j$  such that  $j \in G \cup F \subseteq W$  sent OK messages, and thus received shares

$f_j(x), g_j(y) = S(x, j), S(j, y)$  from the dealer. Every honest party  $P_j$  eventually receives the same broadcast, as well as  $(a_{ij}, b_{ij}) = (f_i(j), g_i(j)) = (S(j, i), S(i, j))$  from every honest  $P_i$  that received shares from the dealer. This means that every honest  $P_j$  receives such messages from every honest  $P_i$  such that  $i \in F$ . Since  $|F| \geq n - t_s \geq 2t_s + t_a + 1$ , and there are at most  $t_a$  corrupt parties, every honest  $P_i$  eventually receives at least  $2t_s + 1$  values  $a_{ji} = S(j, i)$  from honest parties  $P_j$  such that  $j \in F$  and at most  $t_a \leq t_s$  values  $a_{ji} \neq S(j, i)$ .  $P_i$  will then successfully interpolate the points to  $g_j(y) = S(j, y)$ , the unique polynomial of degree  $t_s$  or less that disagrees with at most  $t_s$  points  $a_{ji}$ . In addition, note that for there to be a unique interpolation,  $P_i$  must have received at least  $2t_s + 1$  values, out of which at least  $2t_s + 1 - t_a \geq t_s + 1$  were sent by honest parties. This means that if  $P_i$  ever succeeds in interpolating the polynomial, it does so after receiving at least  $t_s + 1$  points on the polynomial  $S(i, y)$  and at most  $t_a \leq t_s$  other points. In that case,  $S(i, y)$  is one possible polynomial that disagrees with at most  $t_s$  points, and thus must be the unique polynomial for which this holds.

Following that, every party  $P_j$  such that  $j \notin G$  will record  $g_j(y) = S(j, y)$  and send  $(\text{reconstruct}, j, i, g_j(i))$  to every party  $P_i$ . Similarly to above, every honest  $P_i$  receives either  $(a_{ji}, b_{ji})$  from every honest  $P_j$  such that  $j \in G$ , or a **reconstruct** message from every honest  $P_j$  such that  $j \notin G$ . In either case, the message includes the value  $g_j(i) = S(j, i)$ . This means that every honest  $P_i$  receives the value  $S(j, i)$  from at least  $n - t_a \geq 3t_s + 1$  honest parties, and up to  $t_a$  additional arbitrary values from the corrupted parties.  $P_i$  will then attempt to interpolate the value, and find that the unique polynomial of degree  $d \leq t_s + t_a$  that agrees with all but  $t_s$  values must be  $S(x, i)$ . As before, note that for there to be a unique interpolation,  $P_i$  must have received at least  $2t_s + t_a + 1$  values, out of which at least  $2t_s + t_a + 1 - t_a \geq t_s + t_a + 1$  were sent by honest parties. This means that if  $P_i$  succeeds in interpolating the polynomial, it did so after receiving at least  $t_s + t_a + 1$  points on the polynomial  $S(x, i)$  and at most  $t_a \leq t_s$  other points. Therefore,  $S(i, y)$  is one possible polynomial that disagrees with at most  $t_s$  points, and thus must be the unique polynomial for which this holds.

This means that every party  $P_j$  eventually interpolates  $f_j$  if  $j \notin F$  and  $g_j$  if  $j \notin G$ , records both, and outputs  $(f_j(x), g_j(y)) = (S(x, j), S(j, y))$ .  $\square$

**The case of a corrupted dealer.** For a corrupted dealer, we have the following claim:

**Claim 5.12** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 5.2 perfectly implements Functionality 5.1 in the presence of at most  $t_a$  corrupted parties over an asynchronous network, assuming that the dealer is corrupted.*

*Proof.* The simulator runs the protocol while simulating the honest parties. Note that the honest parties have no inputs in the protocol, and therefore the simulation is perfect.

When the first honest party  $P_j$  writes an output to its output tape,  $f_j(x), g_j(y)$ , then it must be that all honest parties will eventually also have outputs. As follows from the proof of Lemma 5.13, from the message  $(W, C, D, G, F)$  that the



dealer has broadcasted, it can reconstruct a unique bivariate polynomial  $S(x, y)$  from the shares of the honest parties in  $D$ . It sends  $S(x, y)$  to the trusted party, together with the set  $K$  of all honest parties. The trusted party then sends  $S(x, k), S(k, y)$  to each honest party  $P_k$ , yet those messages are not delivered yet. The simulator continues to simulate the protocol with the adversary. Whenever an honest party  $P_j$  in the simulated execution writes to its output tape, the simulator allows the delivery of the output of  $P_j$  (which is  $S(x, j), S(j, y)$ ) in the ideal model.

As we will see in Lemma 5.13, if one honest party receives an output, then eventually all honest parties receive an output, and all the outputs of the honest parties must lie on the same bivariate polynomial.  $\square$

**Lemma 5.13** ( $t_a$ -Strong Commitment). *If the dealer is corrupted, then either no honest party computes an output or there exists a polynomial  $S'(x, y)$  of degree at most  $d$  in  $x$  and degree at most  $t_s$  in  $y$  such that every honest party outputs  $f_j(x), g_j(y) = S'(x, j), S'(j, y)$ .*

*Proof.* If no honest party computes an output, the claim trivially holds. Assume some honest party does compute an output. Before doing so, it received and accepted a graph  $(W, C, D, G, F)$  from the dealer, and thus  $(C, D, G, F)$  is an  $(n, t_s)$ -extended star in the graph constructed using the OK and NOK messages. Since  $(C, D, G, F)$  is an  $(n, t_s)$ -extended star in the graph,  $|C| \geq n - 2t_s \geq t_s + t_a + 1$ ,  $|D| \geq n - t_s \geq 2t_s + t_a + 1$ , and there is an edge between every  $j \in C$  and  $k \in D$ . This means that every honest  $P_j, P_k$  such that  $j \in C$  and  $k \in D$  received shares  $f_j(x), g_j(y)$  and  $f_k(x), g_k(y)$  of degrees  $t_s$  and  $d + 1$  from the dealer respectively such that  $(f_j(k), g_j(k)) = (g_k(j), f_k(j))$ , and  $f_j(j) = g_j(j)$ ,  $f_k(k) = g_k(k)$ . From Lemma 3.1, there exists a unique bivariate polynomial  $S'(x, y)$  of degree at most  $d$  in  $x$  and  $t_s$  in  $y$  such that  $f_j(x) = S'(x, j)$  for every honest  $P_j$  such that  $j \in C$  and  $g_j(y) = S'(j, y)$  for every honest  $P_j$  such that  $j \in D$ .

From the fact that  $(C, D, G, F)$  is an  $(n, t_s)$ -extended star, every  $j \in G$  has at least  $n - 2t_s \geq t_s + t_a + 1$  neighbors in  $C$ , and thus  $P_j$  sent OK messages about at least  $t_s + t_a + 1$  parties  $P_k$  such that  $k \in C$ . Out of those, at least  $t_s + t_a + 1 - t_a = t_s + 1$  are honest, and thus  $P_j$  found that  $g_j(k) = f_k(j) = S'(j, k)$  for these honest parties. Since both  $g_j(y)$  and  $S'(j, y)$  are polynomials of degree  $t_s$  or less,  $g_j(y) = S'(j, y)$ . In other words, every honest party  $P_j$  such that  $j \in G$  received  $g_j(y) = S'(j, y)$  from the dealer. Similarly, every  $j \in F$  has at least  $n - t_s \geq 2t_s + t_a + 1$  neighbors in  $C$ , and thus  $P_j$  sent OK messages about at least  $2t_s + t_a + 1$  parties  $P_k$  such that  $k \in C$ . Out of those, at least  $2t_s + t_a + 1 - t_a = 2t_s + 1 \geq d + 1$  are honest, and thus  $P_j$  found that  $f_j(k) = g_k(j) = S'(k, j)$  for these honest parties. Since both  $g_j(y)$  and  $S'(j, y)$  are polynomials of degree  $d$  or less,  $f_j(y) = S'(x, j)$ . This means that every honest party  $P_j$  such that  $j \in F$  received  $f_j(x) = S'(x, j)$  from the dealer.

From this point on, the proof that every party  $P_j$  eventually interpolates and outputs  $f_j(x), g_j(y) = S'(x, j), S'(j, y)$  is identical to the proof of Lemma 5.11.  $\square$



## 6 Complete Verifiable Secret Sharing

The following primitive is a stronger variant of verifiable secret sharing, where now all parties are guaranteed to receive a share. Recall that in the previous primitive (Section 5), this was not guaranteed in the synchronous setting, and some parties might not output a share.

### 6.1 The Functionality

The functionality is similar to Functionality 5.1, where now there is no set  $K$  indicating which parties will receive outputs, as all parties eventually receive output. (Alternatively, this can be viewed as exactly Functionality 5.1, where  $K = [n] \setminus I$ ).

---

#### Functionality 6.1: Complete VSS

---

1. **Honest dealer:**
    - (a) The dealer sends to the functionality the command  $(\text{input}, S)$ , where  $S$  is a bivariate polynomial  $S(x, y)$  of degree  $d$  in  $x$  and  $t_s$  in  $y$ .
    - (b) The functionality sends  $S(x, i), S(i, y)$  to the adversary for every  $i \in I$ .
  2. **Dishonest dealer:**
    - (a) The dealer sends to the functionality the command  $(\text{input}, S)$ , where  $S$  is a bivariate polynomial  $S(x, y)$ .
    - (b) The functionality verifies that  $S(x, y)$  is of degree  $d$  in  $x$  and  $t_s$  in  $y$  and otherwise, replaces it with  $S(x, y) = \perp$ . The functionality stores  $S$ .
  3. **Output:** The functionality sends  $S(x, j), S(j, y)$  for every  $j \in [n] \setminus I$ . Recall that in the synchronous model, this is non-simultaneous, while in asynchrony, the adversary is allowed to set a separate delivery delay (see Remark 3.10).
- 

Next, we describe our Complete VSS protocol. We start by giving the protocol construction and then give the security proofs per Definition 3.8 and the simulation proof to realize the Functionality 6.1.

### 6.2 The Protocol

Throughout the protocol all of a party's edges might be removed from a certain graph  $\mathbb{G}$  over the indices  $[n]$ . For simplicity, we say that an index  $i$  is in the graph  $\mathbb{G}$  if it has any edges and we say that it is not in the graph if it has degree 0 in  $\mathbb{G}$ . With slight abuse of notation, we say that  $i \in \mathbb{G}$  means that  $i \in V$  and that  $\deg_{\mathbb{G}}(i) \geq n - t_s$ . The protocol is almost similar to that of the incomplete VSS protocol; here, each party needs to secret-share the share it receives from the dealer via  $\Pi_{\text{IncompleteSharing}}$ . We highlight the differences in gray.

---

#### Protocol 6.2: $\Pi_{\text{CompleteSync}}$

---

**Input.**

The dealer  $P_1$  has input  $S \in \mathbb{F}^{\leq d, t_s}[x, y]$ . All parties have common knowledge of a degree  $d \in \mathbb{N}$

**The protocol.**

1. **Round I – Share dissemination (Time 0):** Dealer  $P_1$ :
  - (a) Send  $f_i(x) = S(x, i), g_i(y) = S(i, y)$  to each party  $P_i$ .
2. **Round II – Pair-wise checks (Time  $\Delta$ ):** Each party  $P_i$ :
 

If  $P_i$  received  $f_i(x), g_i(y)$  from the dealer such that  $f_i(x)$  is of degree at most  $d$ ,  $g_i(y)$  is of degree of at most  $t_s$ , and  $g_i(i) = f_i(i)$ ,  $P_i$  uniformly samples polynomials  $S_{f,i}(x, y) \in \mathbb{F}^{\leq d, t_s}[x, y], S_{g,i}(x, y) \in \mathbb{F}^{\leq t_s, t_s}[x, y]$  such that  $S_{f,i}(x, 0) = f_i(x)$  and  $S_{g,i}(x, 0) = g_i(x)$  and initiates two instances of  $\Pi_{\text{IncompleteSharing}}$ , say  $\Pi_{\text{IncompleteSharing}}^{(i,f)}$  and  $\Pi_{\text{IncompleteSharing}}^{(i,g)}$ , with inputs  $S_{f,i}$  and  $S_{g,i}$ , respectively.
3. **Broadcast Round I – Results of pair-wise checks (Time  $T_{\text{IncompleteSharing}} + \Delta$ ):** Each party  $P_i$ :
 

Let  $p_{ji}^f(x), q_{ji}^f(y)$  and  $p_{ji}^g(x), q_{ji}^g(y)$  be the outputs of  $\Pi_{\text{IncompleteSharing}}^{(j,g)}$  and  $\Pi_{\text{IncompleteSharing}}^{(j,f)}$ , respectively. For every  $j \in [n]$ :

  - If no such outputs were received, set  $m_{i,j} = \perp$ .
  - Else, if  $(f_i(j), g_i(j)) = (q_{ji}^g(0), q_{ji}^f(0))$ , set  $m_{i,j} = \text{OK}$ .
  - Else, set  $m_{i,j} = (\text{NOK}, f_i(j), g_i(j))$ .

  - (b) Broadcast  $(m_{i,1}, \dots, m_{i,n})$
4. **Broadcast Round II – Extended Star Computation (Time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ ):** Dealer  $P_1$ :
  - (a) Construct a graph  $\mathbb{G} = (V = [n], E = \phi)$ .
  - (b) Add an edge  $(i, j)$  to  $E$ , if  $P_i$  and  $P_j$ 's  $(m_{i,1}, \dots, m_{i,n})$  and  $(m_{j,1}, \dots, m_{j,n})$  broadcasts were received by the end of the previous round such that  $m_{i,j} = m_{j,i} = (\text{OK}, i, j)$ .
  - (c) Remove all edges incident with  $P_i$  if  $P_i$  broadcast  $m_{i,j} = (\text{NOK}, a_{ij}, b_{ij})$  such that  $(a_{ij}, b_{ij}) \neq (S(j, i), S(i, j))$  for any  $j \in [n]$ .
  - (d) Define  $W = V$ . Remove  $i$  from  $W$  if  $\deg_{\mathbb{G}[W]}(i) < n - t_s$ . Repeat this until no more parties can be removed from  $W$ .
  - (e) If an  $(n, t_s)$  extended star  $(C, D, G, F)$  exists in the graph  $\mathbb{G}[W]$ , then broadcast  $(W, C, D, G, F)$ .
5. **Intrusion Detection (Time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$ ):** Each party  $P_i$ :
  - (a) Construct the graph  $\mathbb{G}_i = (V = [n], E)$  as specified in Step 4b.
  - (b) If  $(W, C, D, G, F)$  is broadcast by the dealer by the end of the previous round, then check if the following holds:
    - i. There exist no  $j, k \in W$ , such that  $P_j, P_k$  broadcast  $m_{j,k} = (\text{NOK}, a_{jk}, b_{jk}), m_{k,j} = (\text{NOK}, a_{kj}, b_{kj})$  respectively by the end of the pair-wise checks round, where  $(a_{jk}, b_{jk}) \neq (b_{kj}, a_{kj})$ .
    - ii. For each  $j \in W$ ,  $\deg_{\mathbb{G}_i[W]}(j) \geq n - t_s$ .

- iii.  $(C, D, G, F)$  is a  $(n, t_s)$  extended star in  $\mathbb{G}_i[W]$ .
  - (c) If the checks pass, accept the graph.
- 
6. **Asynchronous output computation.** Each party  $P_i$  does the following *asynchronously* **upon** (a) time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$  passing; and (b) receiving a  $(W, C, D, G, F)$  broadcast from the dealer that passes the checks in Step 5:
- (a) If  $i \in G$ , record the value  $g_i(y)$  received from the dealer and if  $i \in F$ , record the value  $f_i(x)$  received from the dealer.
  - (b) If  $i \notin G$ , consider the values  $(p_{ji}^f(x), q_{ji}^f(y))$  output from  $\Pi_{\text{IncompleteSync}}^{j,f}$  for  $j \in F$ . Once  $t_s + 1$  such values are output from the  $\Pi_{\text{IncompleteSync}}$  calls, interpolate the polynomial  $g_i(y)$  with degree  $t_s$  such that  $g_i(j) = q_{ji}^f(0)$  for  $t_s + 1$  values  $j \in F$  and record  $g_i(y)$ .
  - (c) If  $i \notin F$ , consider the values  $(p_{ji}^g(x), q_{ji}^g(y))$  output from  $\Pi_{\text{IncompleteSync}}^{j,g}$  for  $j \in G$ . Once  $d + 1$  such values are output from the  $\Pi_{\text{IncompleteSync}}$  calls, interpolate the polynomial  $f_i(y)$  with degree  $d$  such that  $f_i(j) = q_{ji}^g(0)$  for  $d + 1$  values  $j \in G$  and record  $f_i(y)$ .
  - (d) Once  $f_i$  and  $g_i$  have been recorded, output  $f_i(x), g_i(y)$  upon receiving a (fetch) command.
- 

Note that the synchronous timeout of  $\Pi_{\text{CompleteSharing}}$  is  $T_{\text{CompleteSharing}} = 2\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$ , that is, the parties expect to complete the protocol by time  $T_{\text{CompleteSharing}}$  if the network was synchronous.

### 6.3 Efficiency and Batching

Similarly to  $\Pi_{\text{IncompleteSharing}}$ , in a single instance of the protocol, the dealer sends  $O(n)$  polynomials of size  $O(n \log n)$ , and every party broadcasts vectors of size  $O(n \log n)$ . In addition, each party calls the  $\Pi_{\text{IncompleteSharing}}$  protocol sharing its polynomials, each requiring  $O(n^3 \log n)$  communication. This means that in total, the protocol requires  $O(n^4 \log n)$  communication for sharing  $O(n)$  secrets.

To batch many calls to the protocol at the same time with the same dealer, parties can batch their OK/NOK messages and the star sent by the dealer in the same way as described in Section 5.3, resulting in  $O(n)$  total broadcasts of messages of size  $O(n \log n)$  for any number of secrets. In addition, parties can batch the different calls to  $\Pi_{\text{IncompleteSharing}}$ , each requiring  $O((n^3 + \ell n^2) \log n)$  communication for participating in  $\ell$  instances of the  $\Pi_{\text{CompleteSync}}$  protocol. In total, this means that the protocol requires  $O((n^4 + Xn^3) \log n)$  for  $X$  instances of the  $\Pi_{\text{CompleteSync}}$  protocol, i.e., for sharing  $O(Xn)$  secrets, or  $O(n^2 \log n)$  per secret.

### 6.4 Security Analysis of Complete VSS

In what follows, we prove each one of the properties of the protocol (6.2), following Definition 3.8 and simulation proof with respect to Functionality 6.1.

**Theorem 6.3.** *Assume that  $n > 3t_s + t_a$ , Protocol 6.2 perfectly implements Functionality 6.1 in the presence of at most  $t_s$  corrupted parties over a synchronous network, and  $t_a$  corrupted parties over an asynchronous network.*

*Proof.* The simulation proof is similar to that of the simulators provided in Claims 5.4, 5.7, 5.10, 5.12, when now the simulator does not provide  $K$  to the trusted party. Moreover, we use the underlying Lemmas of 6.4, 6.7, 6.10, and 6.11, instead of the corresponding lemmas in the incomplete case.  $\square$

Much of the analysis follows a similar intuition to the one in section 5, with consistency checks being done using  $\Pi_{\text{IncompleteSharing}}$  instead of by direct messages. Since no error correction takes place in this protocol (in order to guarantee completeness), all parties' shared polynomials must be consistent with each. This is achieved by filtering out parties with low degree in synchronous executions and by more carefully binding the corrupted parties' shares in the asynchronous case.

### Proofs for Synchronous Execution

**Lemma 6.4** ( $t_s$ -Correctness). *If the dealer is honest and the network is synchronous, then each honest party  $P_j$  outputs  $f_j(x) = S(x, j)$  and  $g_j(y) = S(j, y)$  at time  $2\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$ .*

*Proof.* Since the network is synchronous and the dealer is honest, every honest  $P_i$  receives  $f_i(x), g_i(x)$  by time  $\Delta$  such that  $f_j(x) = S(x, j), g_j(y) = S(j, y)$ . Every honest  $P_i$  samples a pair of bivariate polynomials  $S_{f,i}(x, y)$  and  $S_{g,i}(x, y)$  such that  $S_{f,i}(x, 0) = f_i(x)$  and  $S_{g,i}(x, 0) = g_i(x)$  and initiates two instances of sharing the polynomials. From  $t_s$ -correctness of  $\Pi_{\text{IncompletSync}}$ , every honest party  $P_i$  outputs  $p_{ji}^f(x), q_{ji}^f(y)$  and  $p_{ji}^g(x), q_{ji}^g(y)$  respectively from the calls  $\Pi_{\text{IncompleteSharing}}^{(j,f)}$  and  $\Pi_{\text{IncompleteSharing}}^{(j,g)}$ , respectively for every honest  $P_j$  by time  $\Delta + T_{\text{IncompleteSharing}}$ . For these polynomials,  $q_{ji}^f(y) = S_{f,j}(i, y)$  and  $q_{ji}^g(y) = S_{g,j}(i, y)$ , and thus  $q_{ji}^f(0) = S_{f,j}(i, 0) = f_j(i)$  and  $q_{ji}^g(0) = S_{g,j}(i, 0) = g_j(i)$ . Therefore, every honest party  $P_i$  see that  $f_i(j) = S(j, i) = g_j(i) = q_{ji}^g(0)$  and  $g_i(j) = S(i, j) = f_j(i) = q_{ji}^f(0)$  for every honest  $P_j$ , and broadcasts  $(m_{i,1}, \dots, m_{i,n})$  with  $m_{i,j} = \text{OK}$ . The dealer will receive this broadcast by time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ . In addition, if some honest party sets  $m_{i,j} = (\text{NOK}, f_i(j), g_i(j))$  for any  $j$ , it does so with  $f_i(j) = S(i, j)$  and  $g_i(j) = S(j, i)$ . Consequently, the dealer adds an edge  $(i, j)$  to  $E$  for every pair of honest parties  $P_i, P_j$ .

We now have that the honest parties form a clique in the consistency graph  $\mathbb{G}$  constructed by the honest dealer at time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ . Additionally, it must hold that each honest  $j \notin I$  is included in  $W$ . Hence, the honest parties form a clique even in the graph  $\mathbb{G}[W]$ , and a valid extended star  $(C, D, G, F)$  will be found. From Lemma 3.5, the set  $C$  contains at least  $n - 2t_s$  honest parties from the clique of honest parties and the indices of all honest parties are in  $G$  and  $F$ . By time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$ , the dealer's broadcast of  $(W, C, D, G, F)$

will be received by all the honest parties. In addition, any broadcast of the form  $(m_{i,1}, \dots, m_{i,n})$  received by the dealer by time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$  and used for the construction of the graph will be received by all parties by time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$  as well.

Since the dealer is honest, no two parties in  $W$  will have conflicting NOK messages, as the dealer only includes an index  $i$  in  $W$  if it has a large enough degree, and thus it must have received its  $(m_{i,1}, \dots, m_{i,n})$  broadcast. As described above, all parties receive these broadcasts as well and see that this condition holds. In addition, each party in  $W$  has sufficiently and  $(C, D, G, F)$  must be a valid  $(n, t_s)$ -extended star in the dealer's graph by construction. Again, since all parties receive the same broadcasts in time, this condition holds for each honest party  $P_j$ 's consistency graph  $\mathbb{G}_j[W]$ . Hence, each honest party accepts the graph, and as  $j \in G \cap F$  for every honest  $P_j$ , every honest party records  $f_j(x) = S(x, j)$ ,  $g_j(y) = S(j, y)$  and outputs these shares.

□

**Lemma 6.5.** *Assume the network is synchronous and an honest party  $P_i$  accepts the dealer's graph  $(W, C, D, G, F)$  at time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$ . Then, all honest parties accept the graph at most  $\Delta$  time later and there exists a unique bivariate polynomial  $S'$  with degree  $d \leq t_s + t_a$  in  $x$  and degree  $t_s$  in  $y$  such that for every honest  $P_j$  such that  $j \in W$ ,  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$ . In addition, if the dealer is honest,  $S' = S$ .*

*Proof.* The proof is analogous to the proof of Lemma 5.8, with only small differences. Let  $P_i$  be an honest party that accepts the dealer's graph  $(W, C, D, G, F)$  at time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$ , then from the protocol steps, the following must hold for party  $P_i$ :

1. There are no  $j, k \in W$ , such that  $P_i$  received NOK messages with differing values from  $P_j$  and  $P_k$  at time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ .
2. The graph is consistent with the  $\mathbb{G}_i$  graph constructed using the OK/NOK messages received at time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ .
3. Each party in  $W$  has sufficiently large degree ( $\geq n - t_s$ ).
4.  $(C, D, G, F)$  is a valid  $(n, t_s)$ -extended star in graph  $W$ .

All honest parties receive the same broadcasts received by  $P_i$  at most  $\Delta$  time later and construct the same graph. This means that every honest party also sees that conditions 2, 3 and 4 from above hold. Note that every  $j \in W$  has non-zero degree, and thus  $P_i$  received a  $(m_{j,1}, \dots, m_{j,n})$  broadcast from it by that time. Since  $P_i$  accepted the graph, for every  $j, k \in W$ ,  $m_{j,k}, m_{k,j}$  are not conflicting NOK messages and thus all honest will see that condition 1 holds as well and accept the graph.

Assume, by contradiction, that there exist some honest parties  $P_j, P_k$  such that  $j, k \in W$  that are not pairwise consistent, meaning  $f_j(k) \neq g_k(j)$  or  $f_k(j) \neq g_j(k)$ . According to the protocol, both parties received shares directly from the dealer by time  $\Delta$  and invoked  $\Pi_{\text{IncompleteSharing}}$  with the polynomials  $S_{f,j}, S_{g,j}$  and  $S_{f,k}, S_{g,k}$  as input. In such a case, by  $t_s$ -correctness of  $\Pi_{\text{IncompleteSharing}}$ ,  $P_j$  and

$P_k$  would output shares for  $\Pi_{\text{IncompleteSharing}}^{k,f}$ ,  $\Pi_{\text{IncompleteSharing}}^{k,g}$  and  $\Pi_{\text{IncompleteSharing}}^{j,f}$ ,  $\Pi_{\text{IncompleteSharing}}^{j,g}$ , respectively, by time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ .  $P_j$  would then check if  $f_j(k) = q_{kj}^g(0) = S_{g,k}(j, 0) = g_k(j)$  and if  $g_j(k) = q_{kj}^f(0) = S_{f,k}(j, 0) = f_k(j)$ , and would see that at least one of these equalities doesn't hold. Similarly,  $P_k$  would perform the same checks and see that one of the equalities doesn't hold.  $P_j$  and  $P_k$  would then broadcast NOK messages with differing values at time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ .

Due to the properties of  $\Pi_{\text{BC}}$  in a synchronous setting, every honest party  $P_i$  would receive these NOK messages at time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ . Consequently, if the dealer has  $j, k \in W$  in its  $(W, C, D, G, F)$  message, party  $P_i$  would reject the dealer's graph received at time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$ . This leads to a contradiction. Therefore, every honest party  $j \in W$  must have received its shares  $f_j(x)$  and  $g_j(y)$  from the dealer at time  $\Delta$ , where  $f_j(x)$  is a polynomial of degree  $d$  and  $g_j(y)$  is a polynomial of degree  $t_s$ . Additionally, every pair of honest parties  $P_j, P_k$  such that  $j, k \in W$  are pairwise consistent, meaning  $f_j(k) = g_k(j)$  and  $f_k(j) = g_j(k)$ , meaning that all parties in  $\mathbb{G}[W]$  have shares that are consistent with each other. As  $|D| \geq n - t_s$ , and thus  $|\mathbb{G}[W]| \geq n - t_s$ , there are at least  $n - 2t_s \geq t_s + t_a + 1 \geq d + 1$  indices  $j \in \mathbb{G}[W]$  such that  $P_j$  is honest. According to Lemma 3.1, there is a unique polynomial  $S'$  of degree at most  $d$  in  $x$  and  $t_s$  in  $y$  such that  $f_j(x) = S'(x, j)$ ,  $g_j(y) = S'(j, y)$  for every honest  $P_j$  such that  $j \in W$ . In addition, if the dealer is honest, then the shares  $f_i(x), g_i(y)$  received from the dealer are  $f_i(x) = S(x, i)$ ,  $g_i(y) = S(i, y)$  with  $S$  being the polynomial sampled by the dealer. This means that the unique polynomial  $S'$  must equal  $S$ .  $\square$

**Lemma 6.6.** *Assume the network is synchronous, and an honest party  $P_i$  accepts the dealer's graph  $(W, C, D, G, F)$  at time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$ . Then, there exists a unique polynomial  $S'$  of degree  $d$  in  $x$  and  $t_s$  in  $y$ , such that for every honest party  $P_i$  that outputs shares  $p_{ji}^f(x), q_{ji}^f(y)$  and  $p_{ji}^g(x), q_{ji}^g(y)$  from  $\Pi_{\text{IncompleteSharing}}^{j,f}$ ,  $\Pi_{\text{IncompleteSharing}}^{j,g}$  respectively for  $j \in W$ , it holds that  $q_{ji}^f(0) = S'(i, j)$  and  $q_{ji}^g(0) = S'(j, i)$ . In addition, if the dealer is honest,  $S'(x, y) = S(x, y)$ , where  $S$  is the polynomial sampled by the dealer.*

*Proof.* By Lemma 6.5, there exists a unique bivariate polynomial  $S'(x, y)$  such that, for every honest  $P_j$  with  $j \in W$ , the shares  $f_j(x)$  and  $g_j(y)$  received from the dealer satisfy  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$ . Let  $S'$  be that polynomial. If for some  $j \in W$ ,  $P_j$  is honest, then it invokes  $\Pi_{\text{IncompletSync}}$  with the polynomials  $S_{f,j}, S_{g,j}$ . From  $t_s$ -correctness of  $\Pi_{\text{IncompleteSharing}}$ , every honest party  $P_i$  that outputs shares  $p_{ji}^f(x), q_{ji}^f(y)$  and  $p_{ji}^g(x), q_{ji}^g(y)$  from  $\Pi_{\text{IncompleteSharing}}^{j,g}$ ,  $\Pi_{\text{IncompleteSharing}}^{j,f}$  for  $j \in W$ , it holds that  $q_{ji}^f(0) = f_j(i) = S'(i, j)$  and  $q_{ji}^g(0) = g_j(i) = S'(j, i)$ .

Now, consider a malicious party  $P_j$ . As the honest party  $P_i$  accepts the dealer's graph  $(W, C, D, G, F)$  at time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$  by assumption, then from the protocol steps, the following must hold for party  $P_i$ : (1)  $C \subseteq D \subseteq [n]$ ,  $|C| \geq n - 2t_s$ ,  $|D| \geq n - t_s$ , (2) For every  $j \in C, k \in D$ , there exists an edge  $(j, k) \in E$ , (3) there exist no  $j, k \in \mathbb{G}[W]$ , such that  $P_j, P_k$  broadcast

$m_{j,k} = (\text{NOK}, f_j(k), g_j(k))$  and  $m_{k,j} = (\text{NOK}, f_k(j), g_k(j))$  respectively by time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ , and (4) each  $j \in W$  has sufficiently large degree ( $\geq n - t_s$ ) in  $\mathbb{G}[W]$  and  $(C, D, G, F)$  is a valid  $(n, t_s)$ -extended star.

From point (4), each  $j \in W$  has degree  $n - t_s \geq 2t_s + t_a + 1$ , and thus has edges to at least  $t_s + t_a + 1$  nodes  $i \in W$  such that  $P_i$  is honest and sent an  $(\text{OK}, i, j)$  message. This means that every such honest party  $P_i$  confirmed that for the output values  $p_{ji}^f(x), q_{ji}^f(y)$  and  $p_{ji}^g(x), q_{ji}^g(y)$  from  $\Pi_{\text{IncompleteSharing}}^{j,g}$  and  $\Pi_{\text{IncompleteSharing}}^{j,f}$ , respectively it holds that  $q_{ji}^f(0) = g_i(j)$  and  $q_{ji}^g(0) = f_i(j)$ . From the  $t_s$ -Weak Commitment property of the  $\Pi_{\text{IncompleteSharing}}$  protocol, there exist two bivariate polynomials:  $S_{j,f}(x, y)$  of degree  $d$  in  $x$  and  $t_s$  in  $y$  and  $S_{j,g}(x, y)$  of degree  $t_s$  in both  $x$  and  $y$ , such that every honest party  $P_k$  that outputs a non- $\perp$  share from  $\Pi_{\text{IncompleteSharing}}^{j,f}$  and  $\Pi_{\text{IncompleteSharing}}^{j,g}$  receives  $S_{j,f}(x, k), S_{j,f}(k, y)$  and  $S_{j,g}(x, k), S_{j,g}(k, y)$ , respectively. In addition, for these polynomials  $S_{f,j}(i, 0) = q_{ji}^f(0) = g_i(j) = S'(i, j)$  and  $S_{g,j}(i, 0) = q_{ji}^g(0) = f_i(j) = S'(j, i)$  for  $t_s + t_a + 1 \geq d + 1$  honest parties  $P_i$  such that  $i \in W$  and  $S_{f,j}(x, 0)$  and  $S_{g,j}(x, 0)$  are polynomials of degree  $t_s + t_a$  or less, it must be the case that  $S_{f,j}(x, 0) = S'(x, j), S_{g,j}(x, 0) = S'(j, x)$ . Therefore, for every honest  $P_i$  that outputs  $p_{ji}^f(x), q_{ji}^f(y)$  and  $p_{ji}^g(x), q_{ji}^g(y)$  from  $\Pi_{\text{IncompleteSharing}}^{j,f}$  and  $\Pi_{\text{IncompleteSharing}}^{j,g}$  respectively has  $q_{ji}^f(0) = S'(i, j)$  and  $q_{ji}^g(0) = S'(j, i)$ , as required. In addition, if the dealer is honest, then by Lemma 6.5,  $S'(0, 0) = s$ .  $\square$

**Lemma 6.7** ( $t_s$ -Strong Commitment). *Assume the network is synchronous and the dealer is corrupted, then either no honest party terminates or there exists a bivariate polynomial  $S'(x, y)$  of degree  $t_s + t_a$  in  $x$  and degree  $t_s$  in  $y$  such that every honest party  $P_j$  outputs  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$ . In addition, if some honest party outputs shares at time  $T_{\text{CompleteSync}}$ , every honest party outputs shares by time  $T_{\text{CompleteSync}} + \Delta$ .*

*Proof.* If no honest party computes an output, then the strong commitment holds trivially. Therefore, we consider the scenario where there exists an honest party  $P_j$  that outputs shares  $f_j(x)$  and  $g_j(y)$ . Let  $P_j$  be such a party. Before doing so,  $P_j$  received  $(W, C, D, G, F)$ , and verified it against the received  $\text{OK}/\text{NOK}$  messages. Furthermore,  $P_j$  verifies that (1)  $C \subseteq D \subseteq [n], |C| \geq n - 2t_s, |D| \geq n - t_s$ , (2) For every  $j \in C, k \in D$ , there exists an edge  $(j, k) \in E$ , (3) there exist no  $j, k \in \mathbb{G}$ , such that  $P_j, P_k$  broadcast  $(\text{NOK}, f_j(k), g_j(k))$  and  $(\text{NOK}, f_k(j), g_k(j))$  respectively with  $(f_j(k), g_j(k)) \neq (g_k(j), f_k(j))$  by time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ , and (5) each vertex in  $W$  has sufficiently large degree in  $\mathbb{G}[W]$  ( $\geq n - t_s$ ) and  $(C, D, G, F)$  is a valid  $(n, t_s)$  extended star.

All honest parties receive the same  $\text{OK}/\text{NOK}$  messages and the same graph  $(W, C, D, G, F)$  from the dealer at most  $\Delta$  time after  $P_j$  and accept the graph, as shown in Lemma 6.5. Let  $S'(x, y)$  be the polynomial defined in Lemma 5.8. We will show that every honest  $P_j$  outputs  $f_j(x) = S'(x, j), g_j(y) = S'(j, y)$ . From Lemma 6.5, every honest  $P_j$  such that  $j \in W$  receives the polynomials  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$  from the dealer and stores these polynomials. Parties compute their output as follows:



1. For  $g_j(y)$ : If  $j \in G \subseteq W$ , then  $P_j$  records  $g_j(y) = S'(j, y)$ . Else, if  $j \notin G$ ,  $P_j$  interpolates its  $g_j(y)$  share using at least  $t_s + 1$  values  $q_{ij}^f(0)$  with  $i \in F$ , which are obtained from  $\Pi_{\text{IncompleteSharing}}^{(i,f)}$  at time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ . From Lemma 6.6, for every honest party  $P_j$  that outputs shares  $p_{ij}^f(x), q_{ij}^f(y)$  from  $\Pi_{\text{IncompleteSharing}}^{i,f}$  for  $i \in W$ ,  $q_{ij}^f(0) = S'(j, i)$ . Note that  $|F| \geq n - t_s$ . Therefore, there are at least  $n - 2t_s \geq t_s + t_a + 1$  honest parties  $P_i$  such that  $i \in F$ . By the  $t_s$ -correctness property of  $\Pi_{\text{IncompleteSharing}}$ , every honest party  $j \notin G$  outputs shares from  $\Pi_{\text{IncompleteSharing}}^{i,f}$  for each honest  $P_i$  such that  $i \in F$ . Thus, party  $P_j$  has enough points to interpolate the polynomial share  $g_j(y)$  of degree  $t_s$ . In addition, interpolating at least  $t_s + 1$  points  $S'(j, i)$  on the polynomial  $S'(j, y)$  of degree  $t_s$  or less, results in  $g_j(y) = S'(j, y)$ . After interpolating  $g_j$ ,  $P_j$  records the polynomial.
2. For  $f_j(x)$ : The proof is almost identical to 1. If  $j \in F \subseteq W$ , then  $P_j$  records  $f_j(y) = S'(x, j)$ . Else, if  $j \notin F$ ,  $P_j$  interpolates its  $f_j(y)$  share using at least  $d+1$  values  $q_{ij}^g(0)$  with  $i \in G$ , which are obtained from  $\Pi_{\text{IncompleteSharing}}^{(i,g)}$  at time  $\Delta + T_{\text{BC}} + T_{\text{IncompleteSharing}}$ . From Lemma 6.6, for every honest party  $P_j$  that outputs shares  $p_{ij}^g(x), q_{ij}^g(y)$  from  $\Pi_{\text{IncompleteSharing}}^{i,g}$  for  $i \in W$ ,  $q_{ij}^g(0) = S'(i, j)$ . Note that  $|G| \geq n - t_s$ . Therefore, there are at least  $n - 2t_s \geq t_s + t_a + 1$  honest parties  $P_i$  such that  $i \in G$ . By the  $t_s$ -correctness property of  $\Pi_{\text{IncompleteSharing}}$ , every honest party  $j \notin F$  outputs shares from  $\Pi_{\text{IncompleteSharing}}^{i,g}$  for each honest  $P_i$  such that  $i \in G$ . Thus, party  $P_j$  has enough points to interpolate the polynomial share  $f_j(y)$  of degree  $d \leq t_s + t - a$ . In addition, interpolating at least  $d + 1$  points  $S'(i, j)$  on the polynomial  $S'(x, j)$  of degree  $d$  or less, results in  $f_j(y) = S'(j, y)$ . After interpolating  $f_j$ ,  $P_j$  records the polynomial.

Therefore, every party  $P_j$  records both  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$  and outputs these shares. In addition, if the dealer is honest, from Lemma 6.5,  $S' = S$ . Note that as stated above, if some honest party outputs shares at time  $T_{\text{CompleteSync}}$ , it accepted the dealer's graph by that time and thus all honest parties accept the graph at most  $\Delta$  time later. In addition, every honest party receives all points shared by honest parties before time  $T_{\text{CompleteSync}}$ , and thus they interpolate their shares and output them by time  $T_{\text{CompleteSync}} + \Delta$ .  $\square$

## Proofs for Asynchronous Execution

**Lemma 6.8.** *Assume the network is asynchronous, and upon receiving the dealer's broadcast  $(W, C, D, G, F)$ , an honest party accepts the graph. Then, there exists a unique polynomial  $S'$  of degree  $d$  in  $x$  and  $t_s$  in  $y$ , such that for every honest  $j \in F \cup C$ , it holds that  $f_j(x) = S'(x, j)$ , and for every  $j \in G \cup D$ , it holds that  $g_j(y) = S'(j, y)$ .*

*Proof.* Given that the network is asynchronous, there can be at most  $t_a$  malicious parties. Let  $P_i$  be an honest party that accepts the dealer's graph  $(W, C, D, G, F)$ . Based on the protocol steps, the following conditions must hold for party  $P_i$ : (1)  $C \subseteq D \subseteq [n]$ ,  $|C| \geq n - 2t_s$ ,  $|D| \geq n - t_s$ , (2) For every  $j \in C, k \in D$ , there



exists an edge  $(j, k) \in E$ , (3) there exist no  $j, k \in \mathbb{G}$ , such that  $P_j, P_k$  broadcast  $m_{j,k} = (\text{NOK}, f_j(k), g_j(k))$  and  $m_{k,j} = (\text{NOK}, f_k(j), g_k(j))$  respectively, and (4) for each party  $P_j$  such that  $j \in W$ ,  $j$  has sufficiently large degree in  $\mathbb{G}[W]$  ( $\geq n - t_s$ ) and  $(C, D, G, F)$  is a valid  $(n, t_s)$  extended star. By construction, an edge  $(j, k)$  for two honest parties  $P_j, P_k$  is formed if they send OK messages about each other. From the properties of  $(n, t_s)$ -extended stars, we know that  $|C| \geq n - 2t_s \geq t_s + t_a + 1$ . Therefore, there are at least  $t_s + 1$  honest parties in  $C$ , which we denote as set  $H_C$ . Further, every  $j \in C$  is connected to every  $k \in D$ , and thus, every pair of parties  $P_j, P_k$  such that  $j \in C$  and  $k \in D$  sent OK messages about each other. As shown in Lemma 6.5, this means that for every such pair  $(f_j(k), g_j(k)) = (g_k(j), f_k(j))$ . The set  $D$  contains at least  $n - t_s$  indices, out of which at least  $2t_s + 1 \geq d + 1$  parties are honest, denoted by  $H_D$ . From Lemma 3.1, there exists a unique polynomial  $S'(x, y)$  of degree at most  $d$  in  $x$  and  $t_s$  in  $y$  such that  $f_j(x) = S'(x, j)$  for every honest  $P_j$  such that  $j \in C$  and  $g_k(y) = S'(k, y)$  for every honest  $P_k$  such that  $k \in D$ .

Consider an honest party  $P_k$  such that  $k \in G$ . From the properties of  $(n, t_s)$ -extended stars,  $k$  has at least  $n - 2t_s$  edges into  $C$ , out of which at least  $n - 2t_s - t_a \geq t_s + 1$  are indices of honest parties. Thus,  $g_k(j) = f_j(k) = S'(k, j)$  for at least  $t_s + 1$  indices  $j \in C$  of honest parties  $P_j$ . Since both  $g_k(y)$  and  $S'(k, y)$  are polynomials of degree  $t_s$  or less,  $g_k(y) = S'(k, y)$ . Similarly, consider an honest party  $P_k$  such that  $k \in F$ . From the properties of  $(n, t_s)$ -extended stars,  $k$  has at least  $n - t_s$  edges into  $G$ , out of which at least  $n - t_s - t_a \geq 2t_s + 1 \geq d + 1$  are indices of honest parties. In other words,  $f_k(j) = g_j(k) = S'(k, j)$  for at least  $d + 1$  indices  $j \in G$  of honest parties  $P_j$ . Since both  $f_k(x)$  and  $S'(x, k)$  are polynomials of degree  $d$  or less,  $f_k(x) = S'(x, k)$ .  $\square$

**Lemma 6.9.** *Assume the network is asynchronous, and upon receiving the dealer's broadcast  $(W, C, D, G, F)$ , an honest party accepts the graph. Then, there exists a unique polynomial  $S'$  of degree at most  $d$  in  $x$  and  $t_s$  in  $y$ , such that for every honest party  $P_i$  that outputs shares  $p_{ji}^g(x), q_{ji}^g(y)$  from  $\Pi_{\text{IncompleteSharing}}^{j,g}$  for  $j \in G$  or shares  $p_{ji}^f(x), q_{ji}^f(y)$  from  $\Pi_{\text{IncompleteSharing}}^{j,f}$  for  $j \in F$ , it holds that  $q_{ji}^g(0) = S'(j, i)$  and  $q_{ji}^f(0) = S'(i, j)$ . In addition, if the dealer is honest,  $S'(x, y) = S(x, y)$ .*

*Proof.* Given that the network is asynchronous, there can be at most  $t_a$  malicious parties. Let  $P_i$  be an honest party that verifies the dealer's graph  $(W, C, D, G, F)$  at time  $\Delta + 2T_{\text{BC}} + T_{\text{IncompleteSharing}}$ . Based on the protocol steps, the following conditions must hold for party  $P_i$ : (1)  $C \subseteq D \subseteq [n]$ ,  $|C| \geq n - 2t_s$ ,  $|D| \geq n - t_s$ , (2) For every  $j \in C, k \in D$ , there exists an edge  $(j, k) \in E$ , (3) there exist no  $j, k \in \mathbb{G}$ , such that  $P_j, P_k$  broadcast  $(\text{NOK}, f_j(k), g_j(k))$  and  $(\text{NOK}, f_k(j), g_k(j))$ , and (4) for each party  $P_j$  such that  $j \in W$ ,  $j$  has sufficiently large degree in  $\mathbb{G}[W]$  ( $\geq n - t_s$ ) and  $(C, D, G, F)$  is a valid  $(n, t_s)$  extended star. By construction, an edge  $(i, j)$  is formed due to  $P_i$  and  $P_j$  broadcasting OK messages about each other. An honest party  $P_i$  sends such an OK message after it confirmed that for the shares  $p_{ji}^f(x), q_{ji}^f(y)$  and  $p_{ji}^g(x), q_{ji}^g(y)$  output from  $\Pi_{\text{IncompleteSharing}}^{j,f}$  and  $\Pi_{\text{IncompleteSharing}}^{j,g}$ , respectively it holds that  $q_{ji}^f(0) = g_i(j)$  and  $q_{ji}^g(0) = f_i(j)$ .

From Lemma 6.8, there exists a unique polynomial  $S'$  of degree at most  $d$  in  $x$  and  $t_s$  in  $y$ , such that for every honest  $P_j$  such that  $j \in F$ , it holds that  $f_j(x) = S'(x, j)$ , and for every honest  $P_j$  such that  $j \in G$ , it holds that  $g_j(y) = S'(j, y)$ . Observe some  $P_j$  such that  $j \in W$ , and note that  $\deg_{\mathbb{G}[W]}(j) \geq n - t_s$ . At least some honest party sent an OK message about  $P_j$  and thus must have output shares from the  $\Pi_{\text{IncompletSync}}$  calls with  $P_j$  as dealer. Since some honest party output shares from  $\Pi_{\text{IncompletSharing}}^{j,f}$ , there exists a unique bivariate polynomial  $S_{f,j}(x, y)$  of degrees  $d$  in  $x$  and  $t_s$  in  $y$  such that every party  $P_k$  that outputs  $p_{jk}^f(x), q_{jk}^f(y)$  from  $\Pi_{\text{IncompletSync}}^{j,f}$  has  $p_{jk}^f(x) = S_{f,j}(x, k)$  and  $q_{jk}^f(y) = S_{f,j}(k, y)$ . Similarly, there exists a unique bivariate polynomial  $S_{g,j}(x, y)$  of degree  $t_s$  in both  $x$  and  $y$  such that every party  $P_k$  that outputs  $p_{jk}^g(x), q_{jk}^g(y)$  from  $\Pi_{\text{IncompletSync}}^{j,g}$  has  $p_{jk}^g(x) = S_{g,j}(x, k)$  and  $q_{jk}^g(y) = S_{g,j}(k, y)$ . From the properties of  $(n, t_s)$ -extended stars, every party  $k \in G$  has an edge with at least  $n - 2t_s$  vertices in  $C$ , out of which at least  $n - 2t_s - t_a \geq t_s + 1$  correspond to honest parties.

As  $C \subseteq F$ , each honest  $P_k$  such that  $k \in C$  holds a share  $f_k(x) = S'(x, k)$ . Thus, every party  $P_j$  such that  $j \in G$  is pairwise consistent with at least  $t_s + 1$  honest parties  $P_k$  such that  $k \in C$ . In other words,  $S_{f,j}(k, 0) = q_{jk}^f(0) = g_k(j) = S'(k, j)$  and  $S_{g,j}(k, 0) = q_{jk}^g(0) = f_k(j) = S'(j, k)$ . Both  $S_{g,j}(x, 0)$  and  $S'(j, x)$  are polynomials of degree  $t_s$  or less and thus  $S_{g,j}(x, 0) = S'(j, x)$ . Therefore, every honest party  $P_k$  that outputs shares  $p_{jk}^g(x), q_{jk}^g(y)$  from  $\Pi_{\text{IncompletSync}}^{j,g}$  has  $q_{jk}^g(0) = S_{g,j}(k, 0) = S'(j, k)$ . Similarly, every party  $P_j$  such that  $j \in F$  has edges with at least  $n - t_s$   $k \in G$ , out of which at least  $2t_s + 1$  are indices of honest parties. Since  $d + 1 \leq t_s + t_a + 1$  points define the polynomial  $S_{f,j}(x, 0)$ , it must be the case that  $S_{f,j}(x, 0) = S'(x, j)$ . Therefore, every honest  $P_k$  that outputs  $p_{jk}^f(x), q_{jk}^f(y)$  from  $\Pi_{\text{IncompletSync}}^{j,f}$  has  $q_{jk}^f(0) = S_{f,j}(k, 0) = S'(k, j)$ . Finally, if the dealer is honest, it sends shares that lie on  $S' = S$ , from Lemma 6.8.  $\square$

**Lemma 6.10** (Strong Commitment). *Assume the network is asynchronous, and upon receiving the dealer's broadcast  $(W, C, D, G, F)$ , an honest party accepts the graph. Then, there exists a unique bivariate polynomial  $S'(x, y)$  of degree at most  $d$  in  $x$  and  $t_s$  in  $y$  such that every honest party  $P_j$  outputs  $(f_j(x), g_j(y))$ , such that  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$ . Additionally, if the dealer is honest, then  $S' = S$ .*

*Proof.* Given that the network is asynchronous, there can be at most  $t_a$  malicious parties. Let  $P_i$  be an honest party that accepts the dealer's graph  $(W, C, D, G, F)$ . Based on the protocol steps, the following conditions must hold for party  $P_i$ : (1)  $C \subseteq D \subseteq [n]$ ,  $|C| \geq n - 2t_s$ ,  $|D| \geq n - t_s$ , (2) For every  $j \in C, k \in D$ , there exists an edge  $(j, k) \in E$ , (3) there exist no  $j, k \in \mathbb{G}$ , such that  $P_j, P_k$  broadcast  $(\text{NOK}, f_j(k), g_j(k))$  and  $(\text{NOK}, f_k(j), g_k(j))$  respectively by time  $\Delta + T_{\text{BC}} + T_{\text{IncompletSharing}}$ , and (4) for each party  $P_j$  such that  $j \in W$ ,  $j$  has sufficiently large degree in  $\mathbb{G}[W]$  ( $\geq n - t_s$ ) and  $(C, D, G, F)$  is a valid  $(n, t_s)$  extended star. By construction, an edge between two honest parties  $P_j, P_k$  is formed due to them sending OK messages about each other.

Let  $S'$  be the polynomial defined in Lemmas 6.8 and 6.9, and note that if the dealer is honest,  $S' = S$ . From Lemma 6.8, every honest party  $P_j$  has  $f_j(x) = S'(x, j)$  if  $j \in F$  and  $g_j(y) = S'(j, y)$  if  $j \in G$ . On the other hand,  $P_j$  interpolates  $f_j$  if  $j \notin F$  and interpolates  $g_j$  if  $j \notin G$ . From Lemma 6.9, for every  $k \in G$ , if  $P_j$  outputs  $p_{kj}^g(x), q_{kj}^g(y)$  from  $\Pi_{\text{IncompleteSharing}}^{k,g}$ , then  $q_{kj}^g(0) = S'(k, j)$ . Similarly, for every  $k \in F$ , if  $P_j$  outputs  $p_{kj}^f(x), q_{kj}^f(y)$  from  $\Pi_{\text{IncompleteSharing}}^{k,f}$ , then  $q_{kj}^f(0) = S'(j, k)$ . That is, for every  $k \in G$ ,  $q_{kj}^g(0) = S'(k, j)$  lies on the polynomial  $S'(x, j)$ , and for every  $k \in G$ ,  $q_{kj}^f(0) = S'(j, k)$  lies on the polynomial  $S'(j, y)$ . Note that  $F, G \subseteq W$ , and thus they include indices of parties with nonzero degrees, meaning that they must have sent OK messages. Every honest party that sent such a message received shares from the dealer and called the  $\Pi_{\text{IncompleteSharing}}$  protocol. In addition,  $|F|, |G| \geq n - t_s$ , meaning that they include indices of at least  $n - t_s - t_a \geq 2t_s + 1$  honest parties. From the  $t_s$ -correctness of the  $\Pi_{\text{IncompleteSharing}}$  protocol,  $P_j$  eventually outputs  $q_{kj}^f(y)$  for every honest  $P_k$  such that  $k \in F$  and  $q_{kj}^g(y)$  for every honest  $P_k$  such that  $k \in G$ . This means that if  $j \notin G$ ,  $P_j$  eventually outputs  $2t_s + 1 \geq t_s + 1$  shares  $q_{kj}^f(y)$  and interpolates the points  $q_{kj}^f(0) = S'(j, k)$  to the polynomial  $g_j(y) = S'(j, y)$ . Similarly, if  $j \notin F$ ,  $P_j$  eventually outputs  $2t_s + 1 \geq d + 1$  shares  $q_{kj}^g(y)$  and interpolates the points  $q_{kj}^g(0) = S'(k, j)$  to the polynomial  $f_j(y) = S'(x, j)$ . Following that, every honest party will have either received or interpolated both  $f_j(x) = S'(x, j)$  and  $g_j(y) = S'(j, y)$ , and output the shares, as required. Importantly, if  $P_j$  outputs  $q_{kj}^f(y)$  or  $q_{kj}^g(y)$  for any corrupt  $k$  such that  $k \in G$  or  $k \in F$ , then  $q_{kj}^f(0) = S'(j, k)$  and  $q_{kj}^g(0) = S'(k, j)$  as well, so  $P_j$  only uses correct points on the polynomials when interpolating.  $\square$

**Lemma 6.11** (Privacy). *Irrespective of the network type, the adversary's view is identically distributed for every two polynomials that agree on the adversary's points. More formally, for any polynomial  $S \in \mathbb{F}^{\leq d, t_s}[x, y]$ , define  $\text{VIEW}_S$  to be the distribution of the adversary's view over executions of the protocol with an honest dealer with input  $S$ .*

*Proof.* The proof follows very similar logic to that of Lemma 5.6, taking into account the fact that parties don't directly check pairwise consistency, but do so using an incomplete sharing protocol instead. At the beginning of the protocol, the dealer sends every party  $P_i$  its shares  $f_i(x), g_i(y)$ . Every corrupt party receives the same shares, assuming  $f_i(x) = S(x, i) = S'(x, i)$ , and  $g_i(y) = S(j, y) = S'(j, y)$ . Following that, every honest  $P_j$  samples  $S_{f,j}(x, y), S_{g,j}(x, y)$  such that  $S_{f,j}(x, 0) = f_j(x), S_{g,j}(x, 0) = g_j(x)$  and shares these polynomials using  $\Pi_{\text{IncompleteSharing}}$ . The adversary outputs  $p_{ji}^f(x), q_{ji}^f(y) = S_{f,j}(x, i), S_{f,j}(i, y)$  and  $p_{ji}^g(x), q_{ji}^g(y) = S_{g,j}(x, i), S_{g,j}(i, y)$  from these sharings. Out of these values, only  $q_{ji}^f(0) = S_{f,j}(i, 0) = f_j(i)$  and  $q_{ji}^g(0) = S_{g,j}(i, 0) = g_j(i)$  depend on the dealer's input, and the rest of the values are sampled identically. As stated above, by assumption  $f_j(i) = g_i(j) = S(i, j) = S'(i, j), g_j(k) = f_i(j) = S(j, k) = S'(j, k)$ .

If the dealer's input is  $S$ , every pair of honest parties  $P_j, P_k$  will receive each other's messages, see that  $f_j(k) = S(k, j) = g_k(j)$  and  $g_j(k) = S(j, k) = f_k(j)$ , and not send NOK messages about each other. For identical reasons, honest parties do not send NOK if the dealer's input is  $S'$ . This means that an honest  $P_j$  only sends NOK messages after receiving a message with incorrect shares from a corrupt  $P_k$ . In that case, it sends the message with the values  $f_j(k) = S(k, j) = S'(k, j)$ ,  $g_j(k) = S(j, k) = S'(j, k)$  after receiving any value other than  $f_j(k), g_j(k)$  from  $P_k$ 's share instances. As shown, all honest parties send the exact same messages throughout the protocol, and thus the adversary's view is identical in both executions.  $\square$

## 7 Triplets Sharing and Product Relation Verification

In this section, we show how to realize the product-relation verification functionality. Assume that a dealer owns and preshares  $O(n^2)$  Shamir-sharings of triples, such that each of the triples are supposed to satisfy a product relation. That is, for a triple  $(a, b, c)$ ,  $c$  must be equal to  $ab$ . The parties input shares of those shared triples to the functionality, and the functionality checks that shares define triples satisfying product relations. For the most part, we follow the construction in [3], and implement some changes to maintain the security properties in the network agnostic setting.

First, we introduce the primary building block underlying our product relation verification functionality 7.19, namely trivariate secret sharing (Functionality 7.1). Throughout this section, we set the degree of the trivariate polynomials to  $d = t_s + \min(t_a, \frac{t_a + t_s}{3})$ .

### 7.1 Verifying Using Trivariate Sharing

We follow the definition of [3] for trivariate polynomial verification. The functionality (and protocol) is parameterized by linear circuits  $L_1, \dots, L_n$  for each party. These circuits are functions taking polynomials in either 2 or 3 variables, and outputting a polynomial in 1 or 2 variables, respectively. This is done by evaluating the polynomials in one of the variables, and then computing a known linear combination of the results. Specifically, each circuit  $L$  is of the form:

$$L(\{F_i(x, y, z)\}_{i \in [r]}) = \sum_{i=1}^r \sum_{j=1}^k \lambda_j F_i(x, \alpha_j, z)$$

for some parameter  $k$  and for known values in the field  $(\lambda_j, \alpha_j)_{j \in [k]}$ . With slight abuse of notation, the same circuits are also used with bivariate polynomials as input:

$$L(\{F_i(x, y)\}_{i \in [r]}) = \sum_{i=1}^r \sum_{j=1}^k \lambda_j F_i(x, \alpha_j) .$$

In addition, each party  $P_i$  receives a predicate  $\text{ValidityPred}_i$  as input to the protocol. We assume a restricted environment such that when the dealer is honest, for every honest  $P_i$ ,  $\text{ValidityPred}_i(\text{share}_i) = 1$  for the  $\text{share}_i$  as defined in the functionality below (as a function of  $S$ ).

---

**Functionality 7.1: Trivariate Polynomial Verification**

---

The functionality is parameterized by: (1) The set of corrupted parties  $I \subseteq [n]$ ; (2) Linear circuits  $L_1, \dots, L_n$ . The functionality works as follows:

1. The dealer sends  $\ell$  trivariate polynomials  $S_1(\mathbf{x}, \mathbf{y}, \mathbf{z}), \dots, S_\ell(\mathbf{x}, \mathbf{y}, \mathbf{z})$ , and a set  $\mathbf{zeros} \subseteq [n]$ .
2. Define the share of each party  $P_i$  to be

$$\text{share}_i = ((S_r(\mathbf{x}, \mathbf{y}, i), S_r(\mathbf{x}, i, \mathbf{z}), S_r(\mathbf{x}, \mathbf{y}, i))_{r \in [\ell]}, L_i(S_1, \dots, S_\ell)) .$$

If the dealer is honest, then send the adversary the set  $\mathbf{zeros}$  and for every  $i \in I$  send  $P_i$  the share  $\text{share}_i$ .

3. The honest parties send their external validity predicate  $\text{ValidityPred}_j$  to the functionality. Each  $\text{ValidityPred}_j$  takes as input  $\text{share}_j$  and outputs 0 or 1.
  4. If the dealer is corrupted, then send  $(\text{ValidityPred}_j)_{j \notin I}$  to the adversary.
  5. If the dealer is honest, and the adversary sends a set  $\mathbf{conflicts}$ , then verify that  $|\mathbf{zeros}| \leq t_s - t_a$ , and  $|\mathbf{conflicts} \setminus \mathbf{zeros}| \geq t_a$ . If this holds, store the set  $\mathbf{conflicts}$ .
  6. If each one of the  $\ell$  trivariate polynomials  $S_1, \dots, S_\ell$  is of degree  $d$  in each one of the three variables, and  $\text{ValidityPred}_j(\text{share}_j) = 1$  holds for at least  $2t_s + 1$  honest parties, then move to the next step. Otherwise, the functionality doesn't have an output.
  7. **Output:** If a set  $\mathbf{conflicts}$  has been stored and the dealer is honest, send  $\mathbf{zeros} \cup \mathbf{conflicts}$ . Otherwise, send OK to every party  $P_j$  such that  $j \in [n] \setminus I$ . In synchronous networks, this is performed according to Remark 3.10.
- 

Next, we give the property based definition, followed by the protocol construction that realizes Functionality 7.1.

**Definition 7.2.** A Relation Verification Protocol has a designated party acting as the dealer, which has as input  $S_1, \dots, S_\ell \in \mathbb{F}^{\leq d, d, d}[x, y, z]$ , which are  $\ell$  trivariate polynomials in a commonly-known finite field  $\mathbb{F}$ , and of degree  $d$  or less in  $x, y, z$ , for a commonly known  $d$ , and a set  $\mathbf{zeros}$ , where  $\mathbf{zeros} \subseteq [n]$ . Each party  $P_i$  has an input  $\text{ValidityPred}_i$ , where  $\text{ValidityPred}_i$  is a predefined predicate. The protocol is parameterized by linear functions  $L_1, \dots, L_n$ . We say  $\Pi_{\text{RelationVerification}}$  is a Relation Verification protocol over the finite field  $\mathbb{F}$  against an adversary corrupting  $t_s$  parties in a synchronous network and  $t_a$  parties in an asynchronous network if the following holds:

1. *Privacy:* Irrespective of the network type, when the dealer is honest, the adversary's view is identically distributed for every two polynomials that agree on the adversary's points.

That is, for every set of corrupted parties  $I \subseteq [n]$  of cardinality at most  $t_s$ , for every set of trivariate polynomials  $S_1, \dots, S_\ell, S'_1, \dots, S'_\ell \in \mathbb{F}^{\leq d, t_s}[x, y]$  such that for every  $i \in I$ ,  $S_r(x, y, i) = S'_j(x, y, i)$ ,  $S_r(x, i, z) = S'_j(x, i, z)$ ,  $S_r(i, y, z) = S'_j(i, y, z)$  for every  $r \in [\ell]$  and  $L_i(S_1, \dots, S_\ell) = L_i(S'_1, \dots, S'_\ell)$ , then  $\text{VIEW}_{S, I} = \text{VIEW}_{S', I}$ , where  $\text{VIEW}_{X, I}$  is a random variable denoting all messages that the corrupted parties  $I$  receive in an execution of the protocol where the honest dealer uses  $X$  as input.

## 2. Synchronous Network:

- (a) **Validity:** If the dealer is honest and has inputs  $S_1, \dots, S_\ell$ , such that  $\text{ValidityPred}_i(\text{share}_i) = 1$  for every honest  $i \in [n]$ , then either the dealer outputs an updated set **zeros** by time  $T_{\text{RelationVerification}}$ , such that additional  $t_a$  corrupted parties are added or each honest party outputs **OK** by a known time  $T_{\text{RelationVerification}}$ . Furthermore, if  $|\text{zeros}| \geq t_s - t_a$ , then every honest party outputs **OK** by a known time  $T_{\text{RelationVerification}}$ .
- (b) **Termination:** If one honest party terminates by time  $T$ , then every honest party does so as well.
- (c) **Binding:** If an honest party outputs **OK** by time  $T$ , then all honest parties output **OK** at most one round later, and there exist unique trivariate polynomials  $S_1, \dots, S_\ell$  of degree at most  $d$  in each of the variables  $x, y, z$  that can be extracted from the views of honest parties. Furthermore, for at least  $2t_s + 1$  honest parties, it holds that  $\text{ValidityPred}_i(\text{share}_i) = 1$ .

## 3. Asynchronous Network:

- (a) **Validity:** If the dealer is honest and has inputs  $S_1, \dots, S_\ell$ , such that  $\text{ValidityPred}_i(\text{share}_i) = 1$  for every honest  $i \in [n]$ , and some party outputs shares, then each honest party eventually outputs **OK**.
- (b) **Termination:** If one honest party terminates, then every honest party eventually does so as well.
- (c) **Binding:** If an honest party outputs **OK**, then all parties eventually output **OK** and there exist unique trivariate polynomials  $S_1, \dots, S_\ell$  of degree at most  $d$  in each of the variables  $x, y, z$  that can be extracted from the views of honest parties. Furthermore, for at least  $2t_s + 1$  honest parties, it holds that  $\text{ValidityPred}_i(\text{share}_i) = 1$ .

For ease of notation, we define a predicate that checks the consistency of the shares between two parties as follows:

**Definition 7.3** (Shares Consistency Check [3]). We define the shares  $\text{share}_i^r$  for  $r \in [\ell]$  received by party  $P_i$  from the dealer as follows:

$$\text{share}_i^r = \{f_i^r(x, y) = S_r(x, y, i), \quad g_i^r(x, z) = S_r(x, i, z), \quad h_i^r(y, z) = S_r(i, y, z), \\ q_i(x, z) = L_i(S_1, \dots, S_r)\}.$$

A share  $\text{share}_i^r$  is said to be consistent with  $\text{share}_{j,i}^r$  for  $i, j \in [n]$ , where

$$\text{share}_{j,i}^r = \{f_j^r(i, y), f_j^r(x, i), g_j^r(i, z), g_j^r(x, i), h_j^r(i, z), h_j^r(y, i), L_i(\{f_j^r(x, y)\}_{r \in [\ell]})\},$$

denoted as  $\text{consistent}(\text{share}_{i,j}^r, \text{share}_{j,i}^r) = 1$ , if the following hold:

- $f_j^r(i, y) = h_i^r(y, j)$ ,
- $f_j^r(x, i) = g_i^r(x, j)$ ,
- $g_j^r(i, z) = h_i^r(j, z)$ ,
- $g_j^r(x, i) = f_i^r(x, j)$ ,
- $h_j^r(i, z) = g_i^r(j, z)$ ,
- $h_j^r(y, i) = f_i^r(j, y)$ , and
- $L_i(\{f_j^r(x, y)\}_{r \in [\ell]}) = q_i(x, j)$ .

---

**Protocol 7.4:**  $\Pi_{\text{RelationVerification}}$ 


---

**Input.** The dealer, denoted as  $P_1$ , has as input trivariate polynomials  $S_1(x, y, z), \dots, S_r(x, y, z)$  over the finite field  $\mathbb{F}$ , and a set of parties  $\text{zeros} \subseteq [n]$ . Each party  $P_i$  has input predicate  $\text{ValidityPred}_i$ .

**The protocol.**

1. **Round I – Share dissemination (Time 0):** Dealer  $P_1$ :
  - (a) The dealer chooses an arbitrary subset of  $\text{zeros}$ , such that  $\text{zeros}^* = \min\{|\text{zeros}|, t_s - t_a\}$ .
  - (b) Send  $(f_i^r(x, y) = S_r(x, y, i), g_i^r(x, z) = S_r(x, i, z), h_i^r(y, z) = S_r(i, y, z), q_i(x, z) = L_i(S_1, \dots, S_r), \text{zeros}^*)$  to each party  $P_i$ .
2. **Round II – Pair-wise checks (Time  $\Delta$ )** Each party  $P_i$ :
  - (a) Upon receiving  $(\{f_i^r(x, y), g_i^r(x, z), h_i^r(y, z)\}, q_i(x, z))$ , check if each of the following holds for every  $r \in [\ell]$ :
    - i.  $\text{ValidityPred}_i(\{f_i^r(x, y), g_i^r(x, z), h_i^r(y, z)\}, q_i(x, z))$
    - ii.  $\text{consistent}(\text{share}_{i,i}^r, \text{share}_{i,i}^r)$
    - iii. For every  $j \in \text{zeros}^*$ , set  $\text{share}_{j,i}^r = \{f_j^r(i, y) = 0, f_j^r(x, i) = 0, g_j^r(i, z) = 0, g_j^r(x, i) = 0, h_j^r(i, z) = 0, h_j^r(y, i) = 0, L_j(\{f_i^r(x, y)\}_{r \in [\ell]})\}$ , and check if  $\text{consistent}(\text{share}_i^r, \text{share}_{j,i}^r)$  holds for every  $r \in [\ell]$ .
  - (b) If the above checks hold, send  $(\text{received}, \{\text{share}_{i,j}^1, \dots, \text{share}_{i,j}^r\})$  to each party  $P_j \notin \text{zeros}^*$ . Else, stop participating in the protocol.
3. **Broadcast Round I – Results of pair-wise checks (Time  $2\Delta$ ):** Each party  $P_i$ :
  - (a) Upon receiving  $(\text{received}, \{\text{share}_{i,j}^1, \dots, \text{share}_{i,j}^r\})$  from party  $P_j$ , check if  $\text{consistent}(\text{share}_i^r, \text{share}_{j,i}^r) = 1$  for all  $r$ .
    - i. If the check holds, set  $m_{i,j} = \text{OK}$ .
    - ii. Else, if a messages was received from  $P_j$ , set  $m_{i,j} = (\text{NOK}, f_i^r(j, y), f_i^r(x, j), g_i^r(j, z), g_i^r(x, j), h_i^r(j, z), h_i^r(y, j), q_i(x, j), L_j(\{f_i^r(x, y)\}_{r \in [\ell]}))$  for the minimal  $r$  for which the check failed.
    - iii. Else, if no message was received from  $P_j$ , set  $m_{i,j} = \perp$ .
  - (b) Broadcast  $(m_{i,1}, \dots, m_{i,n})$
4. **Broadcast Round II – Extended Star or Dense Graph Computation (Time  $2\Delta + T_{\text{BC}}$ ):** Dealer  $P_1$ :



- (a) Construct a graph  $\mathbb{G} = (V = [n], E = \phi)$ .
- (b) Add an edge  $(i, j)$  to  $E$ , if  $P_i$  and  $P_j$ 's  $(m_{i,1}, \dots, m_{i,n})$  and  $(m_{j,1}, \dots, m_{j,n})$  broadcasts were received by the end of the previous round such that  $m_{i,j} = m_{j,i} = \text{OK}$ .
- (c) Remove all edges incident with  $P_i$  if  $P_i$  broadcast  $m_{i,j} = (\text{NOK}, f_i^r(j, y), f_i^r(x, j), g_i^r(j, z), g_i^r(x, j), h_i^r(j, z), h_i^r(y, j), q_i(x, j))$  for some value  $r$ , such that  $(f_i^r(j, y), f_i^r(x, j), g_i^r(j, z), g_i^r(x, j), h_i^r(j, z), h_i^r(y, j), q_i(x, j)) \neq (S_r(j, y, i), S_r(x, j, i), S_r(j, i, z), S_r(x, i, j), S_r(i, j, z), S_r(i, y, j), L_i(\{S_r(x, y, j)\}_{r \in [\ell]}))$ .
- (d) Define  $V' = \{i \mid \deg_{\mathbb{G}}(i) \geq n - t_s\}$ , where  $\deg_{\mathbb{G}}(i)$  denotes the degree of  $i$  in  $\mathbb{G}$ .
- (e) Remove  $i$  from  $V'$  if  $\deg_{\mathbb{G}[V']}(i) < n - t_s$ . Repeat this until, no more parties can be removed from  $V'$ .
- (f) If  $|V' \cup \text{zeros}^*| < n - t_a$ , pick a set of  $t_a$  parties outside  $V' \cup \text{zeros}^*$ , and add them to set  $\text{zeros}$ . Then, broadcast (Restart), and restart the protocol. Else, if  $|V' \cup \text{zeros}^*| \geq n - t_a$ , add an edge between each  $i \in \text{zeros}^*$  and  $j \in V' \cup \text{zeros}^*$ , add  $\text{zeros}^*$  to  $V'$  and proceed to Step 4g.
- (g) Define  $W = \{i \in [n] \mid \deg_{\mathbb{G}[V']}(i) \geq n - \min(t_a, \frac{t_s + t_a}{3}) + 1\}$ . If  $|W| \geq n - t_s$ , then broadcast (dense,  $V', W$ ).
- (h) If  $|W| < n - t_s$ , then for each  $i \in V' \setminus W$ , define  $\mathbb{G}_i = (V_i, E_i)$  to be the subgraph induced by  $i$ 's neighbors in  $V'$ . That is:
  - $V_i = \{j \in [n] \mid (i, j) \in E\}$ ,
  - $E_i = \{(j, k) \in E \mid j, k \in V_i\}$ .

For every  $i \in \mathbb{G}_i \setminus W$ , define  $n_i = |V_i|$  and  $t_i = t_s - \min(t_a, \frac{t_s + t_a}{3})$  and attempt to find an  $(n_i, t_i)$ -star in  $\mathbb{G}_i$ . If such a star is found for any  $i$ , choose one such index arbitrarily and broadcast (star,  $V', C_i, D_i$ ), where  $(C_i, D_i)$  is the corresponding  $(n_i, t_i)$ -star.

**5. Intrusion Detection (Time  $2\Delta + 2T_{\text{BC}}$ ):** Each  $P_i$ :

- (a) If (Restart) is broadcast by the dealer by the end of the previous round, go to step 6. Else, proceed to the next step.
- (b) Construct the graph  $\mathbb{G}_i = (V = [n], E)$  as specified in Step 4b.
- (c) If (dense,  $V', W$ ) is broadcast by the dealer by the end of the previous round, then check if the following holds:
  - i.  $|W| \geq n - t_s, |V'| \geq n - t_a, W \subseteq V'$
  - ii. For each  $j \in V'$ ,  $\deg_{\mathbb{G}_i[V']}(j) \geq n - t_a$ .
  - iii. For each  $j \in W$ ,  $\deg_{\mathbb{G}_i[V']}(j) \geq n - \min(t_a, \frac{t_s + t_a}{3}) + 1$ .
  - iv. There exist no  $i, j \in V'$ , such that  $P_i, P_j$  broadcast  $m_{i,j} = (\text{NOK}, f_i^r(j, y), f_i^r(x, j), g_i^r(j, z), g_i^r(x, j), h_i^r(j, z), h_i^r(y, j), q_i(x, j), L_j(\{f_i^r(x, y)\}_{r \in [\ell]}))$  and  $m_{j,i} = (\text{NOK}, f_j^r(i, y), f_j^r(x, i), g_j^r(i, z), g_j^r(x, i), h_j^r(i, z), h_j^r(y, i), q_j(x, i), L_i(\{f_j^r(x, y)\}_{r \in [\ell]}))$ , respectively by the end of the pairwise checks round such that  $(f_j^r(i, y), f_j^r(x, i), g_j^r(i, z), g_j^r(x, i), h_j^r(i, z), h_j^r(y, i), q_j(x, i), L_i(\{f_j^r(x, y)\}_{r \in [\ell]})) \neq (h_i^r(y, j), g_i^r(x, j), h_i^r(j, z), f_i^r(x, j), g_i^r(j, z), f_i^r(j, y), L_j(\{f_i^r(x, y)\}_{r \in [\ell]}), q_i(x, j))$ .

If all the checks pass, then accept the graph.



- (d) If  $(\text{star}, V', C, D)$  is broadcast by the dealer by the end of the previous round, then check if the following holds:
- i. For each  $j \in V'$ ,  $\deg_{G_i[V']}(j) \geq n - t_a$ .
  - ii.  $C \subseteq D \subseteq V'$ ,  $|C| \geq n' - 2t'$ ,  $|D| \geq n' - t'$ , where  $|n'| = n - \min(t_a, \frac{t_s+t_a}{3})$  and  $t' = t_s - \min(t_a, \frac{t_s+t_a}{3})$ .
  - iii. For every  $j \in C, k \in D$ , there exists an edge  $(j, k) \in E$ .
  - iv. There exist no  $i, j \in V'$ , such that  $P_i, P_j$  broadcast  $m_{i,j} = (\text{NOK}, f_i^r(j, y), f_i^r(x, j), g_i^r(j, z), g_i^r(x, j), h_i^r(j, z), h_i^r(y, j), q_i(x, j), L_j(\{f_i^r(x, y)\}_{r \in [\ell]}))$  and  $m_{j,i} = (\text{NOK}, f_j^r(i, y), f_j^r(x, i), g_j^r(i, z), g_j^r(x, i), h_j^r(i, z), h_j^r(y, i), q_j(x, i), L_i(\{f_j^r(x, y)\}_{r \in [\ell]}))$ , respectively by the end of the pairwise checks round such that  $(f_j^r(i, y), f_j^r(x, i), g_j^r(i, z), g_j^r(x, i), h_j^r(i, z), h_j^r(y, i), q_j(x, i), L_i(\{f_j^r(x, y)\}_{r \in [\ell]})) \neq (h_i^r(y, j), g_i^r(x, j), h_i^r(j, z), f_i^r(x, j), g_i^r(j, z), f_i^r(j, y), L_j(\{f_i^r(x, y)\}_{r \in [\ell]}), q_i(x, j))$ .
- (e) If the checks pass, accept the graph.
6. **Synchronous output computation (Time  $2\Delta + 2T_{\text{BC}}$ ):** Each  $P_i$ :
- (a) If the graph was accepted in the previous round, output OK. Else, output zeros and restart the protocol.
- 

## Proofs for Synchronous Execution

### Synchrony: The Case of an Honest Dealer

**Claim 7.5** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 7.4 perfectly implements Functionality 7.1 in the presence of at most  $t_s$  corrupted parties over a synchronous network, assuming that the dealer is honest.*

*Proof.* The proof follows a similar logic to that of Claim 5.4. The honest dealer sends its polynomials  $S_1(x, y, z), \dots, S_\ell(x, y, z)$  and a set **zeros** to the functionality. The functionality delivers to the simulator the shares of the corrupted parties  $(S_r(x, y, i), S_r(x, i, z), S_r(i, y, z))_{r \in [\ell]}, L_i(S_1, \dots, S_\ell)$ . The simulator  $\mathcal{S}$  works as follows.

1. Receive from the trusted party the shares of the corrupted parties  $(S_r(x, y, i), S_r(x, i, z), S_r(i, y, z))_{r \in [\ell]}, L_i(S_1, \dots, S_\ell)$ .
2. Choose arbitrarily polynomials  $S'_1(x, y, z), \dots, S'_\ell(x, y, z)$  of degree at most  $d$  all three polynomials, such that for every  $i \in I, r \in [\ell]$ ,  $S'_r(x, y, i) = S_r(x, y, i)$ ,  $S'_r(x, i, z) = S_r(x, i, z)$ ,  $S'_r(i, y, z) = S_r(i, y, z)$ , and  $L_i(S_1, \dots, S_\ell) = L_i(S'_1, \dots, S'_\ell)$ .
3. Run the protocol honestly with the adversary  $\mathcal{A}$ , simulating the honest parties as described in the protocol, while the dealer uses  $S'_1, \dots, S'_\ell$  and **zeros** as input (and the other honest parties have no input).
4. If the dealer outputs a set **zeros'** and the honest parties do not output OK by the end of the protocol, send the functionality the set **zeros'**.

## 5. Output the output of the adversary.

We claim that in the synchronous setting, the ideal execution is identical to the real execution. We show that using a hybrid argument:

- **Hyb<sub>0</sub>**: This is the ideal execution. The environment sees the output of the ideal honest parties and the simulator  $\mathcal{S}$ . Note that the environment chooses the input  $S(x, y)$  of the honest dealer, and the output of all parties is determined by  $S$ .
- **Hyb<sub>1</sub>**: In this execution, we assume that the simulator also receives  $S_1(x, y, z), \dots, S_\ell(x, y, z)$  from the functionality. When simulating the protocol, it uses  $S_1, \dots, S_\ell$  as the input of the dealer, and simulating  $\text{ValidityPred}_j$  as always outputting 1 for every honest  $P_j$ . Note that the output of the honest parties in the ideal execution is determined by the trusted party.
- **Hyb<sub>2</sub>**: This is the real execution. In particular, the output of the honest parties is determined by the protocol, and not by the functionality.

**Hyb<sub>0</sub> and Hyb<sub>1</sub> are identically-distributed.** We claim that the view of the environment in Hyb<sub>0</sub> and Hyb<sub>1</sub> is identical. The only difference between the two hybrids is the view of the adversary. All parties are deterministic. In Hyb<sub>0</sub> it runs in an execution where the dealer uses  $S_1, \dots, S_\ell$  as input. In Hyb<sub>1</sub> it runs in an execution where the dealer uses  $S'_1, \dots, S'_\ell$ . However, for every  $i \in I, r \in [\ell]$ , it holds that  $S_r(x, y, i) = S'_r(x, y, i)$ ,  $S_r(x, i, z) = S'_r(x, i, z)$ ,  $S_r(i, y, z) = S'_r(i, y, z)$ , and  $L_i(S_1, \dots, S_\ell) = L_i(S'_1, \dots, S'_\ell)$ . From Lemma 7.10, the view of the adversary is distributed identically in those two executions.

**Hyb<sub>1</sub> and Hyb<sub>2</sub> are identically-distributed.** The difference is that in Hyb<sub>1</sub>, the outputs of the honest parties are determined by the functionality, whereas in the real execution, they are determined by the protocol. In both executions, the protocol is executed using the same input for the honest dealer,  $S_1(x, y, z), \dots, S_\ell(x, y, z)$ . As follows from Lemma 7.8 either all parties output OK, or no party outputs OK, and the dealer outputs a set **zeros**. Note that in the proof, we can see that the latter only happens as a function of the behavior of the adversary and does not depend on the polynomials. In addition, if a set **zeros'** is output by the dealer, then from Lemma 7.8, the  $|\text{zeros}' \setminus \text{zeros}| \geq t_a$ , and thus the functionality will store **zeros'** and send that output to the dealer in both hybrids. Otherwise, all parties output OK in both hybrids.  $\square$

In what follows, we prove each one of the properties of the protocol, following Definition 7.2.

**Lemma 7.6** ( $t_s$ -Correctness). *If the dealer is honest and the network is synchronous, at a fixed time  $T = 2\Delta + 2T_{\text{BC}}$ , all the honest parties  $P_j$  output  $f_j(x) = S(x, j), g_j(y) = S(j, y)$ .*

*Proof.* Let  $I$  denote the set of malicious parties. Since the network is synchronous and the dealer is honest, then:

1. By the end of round 1, each honest party  $P_j$  receives polynomials  $f_j(x) = S(x, j)$  and  $g_j(y) = S(j, y)$ .
2. In round 2, each honest party  $P_j$  sends the points  $(a_{ji}, b_{ji}) = (S(j, i), S(i, j))$  to each party  $P_i$ , which honest parties receive by the end of the round.
3. Each honest party  $P_j$  receives consistent points from honest parties  $P_k$ , i.e.  $P_k$  sends  $(a_{kj}, b_{kj}) = (S(j, k), S(k, j)) = (b_{jk}, a_{jk})$ . Following that,  $P_j$  sets  $m_{j,k} = \text{OK}$  for every honest  $P_k$  and broadcasts  $(m_{j,1}, \dots, m_{j,n})$ . All honest parties, including the dealer, receive these broadcasts by the end of the broadcast round. Furthermore, for each honest  $P_j$ , if  $m_{j,k} = (\text{NOK}, a_{ij}, b_{ij})$  then  $(a_{ij}, b_{ij}) = (S(j, i), S(i, j))$ .

We now have that the honest parties form a clique in the graph  $\mathbb{G}$  constructed by the honest dealer in round 4. Since there are at least  $n - t_s$  honest parties, this clique is of size  $n - t_s$ , and  $\deg_{\mathbb{G}}(i) \geq n - t_s$  holds for every honest  $P_i$ . This means that no honest party's index is removed from  $W$  in the first iteration. Following similar logic, the honest parties' indices form a clique in  $\mathbb{G}[W]$  in each subsequent iteration, meaning that no honest parties are removed from  $W$ . Hence, the honest parties form a clique even in the graph  $\mathbb{G}[W]$ , and a valid extended star  $(C, D, G, F)$  must be found. From Lemma 3.5, the set  $C$  contains at least  $n - 2t_s$  indices of honest parties from the clique of honest parties, and the indices of all honest parties are included in  $G, F$ . The rest of the rounds proceed as follows:

4. The dealer broadcasts the found star at the beginning of round 4, and all honest parties receive the message by the end of the round.
5. Since the dealer is honest, each party in  $W$  has sufficiently large degree (otherwise they would have been removed) and  $(C, D, G, F)$  must be a valid  $(n, t_s)$  extended star in each honest party  $P_j$ 's graph  $\mathbb{G}_j[W]$ , as they receive the same OK messages as the dealer up to  $T_{\text{BC}}$  rounds later and thus have the same edges in  $E$ . Note that if some vertex  $j \in [n]$  has a non-zero degree in the dealer's graph, then the dealer received its  $(m_{j,1}, \dots, m_{j,n})$  broadcast, including all OK and NOK messages. This means that no honest party will see two indices  $j, k \in W$  such that  $P_j$  and  $P_k$  broadcast conflicting  $m_{j,k} = (\text{NOK}, a_{jk}, b_{jk})$  and  $m_{k,j} = (\text{NOK}, a_{kj}, b_{kj})$ , as the dealer must have received the same messages and removed at least one in round 4.
6. Each honest party  $P_j$  will see that  $j \in G \cap F$ , record  $g_j$  and  $f_j$  and output  $(f_j(x), g_j(y))$  at time  $2\Delta + 2T_{\text{BC}}$ .

□

**Lemma 7.7.** *Assume the network is synchronous. An honest dealer never adds an honest party to set zeros.*

*Proof.* We prove this by induction on the number of restarts  $r$  performed by an honest dealer. Base case: Let  $r = 1$ . Then  $|\text{zeros}| = 0$ . An honest dealer adds a party to **zeros** only if it is outside the graph  $V' \cup \text{zeros}^*$ , where  $\text{zeros}^* \subseteq \text{zeros}$  and is initially empty. We will show that every honest party belongs to  $V'$ , and therefore is never added to **zeros**. Since the network is synchronous and the dealer is

honest, by time  $\Delta$ , each honest party  $j \notin I$  receives polynomials  $f_i(x, y) = S(x, y, i)$ ,  $g_i(x, z) = S(x, i, z)$ ,  $h_i(y, z) = S(i, y, z)$ ,  $q_i(x, z) = L_i(S_1, \dots, S_r)$ , and set  $\text{zeros}^*$ , where  $I$  is the set of malicious parties. As the dealer is honest,  $\text{ValidityPred}_i((\{f_i^r(x, y), g_i^r(x, z), h_i^r(y, z)\}_{r \in [\ell]}, q_i(x, z))) = 1$  and  $\text{consistent}(\text{share}_{i,i}^r, \text{share}_{i,i}^r) = 1$  for  $r \in [\ell]$ . Consequently, at time  $\Delta$ , each honest party  $j \notin I$  sends the points  $\text{share}_{j,i}^r$  for each  $r \in [\ell]$  to each party  $i \in [n]$  which are received by honest parties by time  $2\Delta$ . Each honest party  $i \notin I$  receives consistent points from honest parties  $j \notin I$ , and sets and broadcasts  $m_{i,j} = \text{OK}$  which is received by all honest parties (including the honest dealer) by time  $2\Delta + T_{\text{BC}}$ . Furthermore, for each  $m_{i,j} = (\text{NOK}, \dots)$  for some  $r \in [\ell]$  broadcast by an honest party  $i \notin I$  corresponding to a corrupted party  $j \in I$ . We now have that the honest parties form a  $n - t_s$  clique in the consistency graph  $\mathbb{G}$  constructed by the honest dealer at time  $2\Delta + T_{\text{BC}}$ . Thus, every honest party has at least  $n - t_s$  neighbors, and it must hold that each honest  $j \notin I$  is included in  $V'$ .

Inductive Step: assume the lemma holds for all  $r < k$ . We prove that the lemma holds for  $r = k$ . Again, an honest dealer adds a party to set  $\text{zeros}$  if it is outside the graph  $V' \cup \text{zeros}^*$ , where  $\text{zeros}^* \subseteq \text{zeros}$ . From inductive hypothesis, set  $\text{zeros}$  contains only malicious parties. We that every honest party belongs to  $V'$ , and therefore is never added to  $\text{zeros}$ . The same argument holds just from the base case, except the dealer sends a non-empty set  $\text{zeros}^*$  to honest parties in Step 1b. As the dealer is honest, he sets the shares for the parties in  $\text{zeros}^*$  correctly and consequently, the check in Step 2(a)iii holds for every honest party. Thus, every honest party  $P_i$  sets  $m_{i,j}$  correctly for every  $j \in \text{zeros}$ . Furthermore, for each  $m_{i,j} = (\text{NOK}, \dots)$  for some  $r \in [\ell]$  broadcast by an honest party  $i \notin I$  corresponding to a corrupted party  $j \in I$ . We now have that the honest parties form a  $n - t_s$  clique in the consistency graph  $\mathbb{G}$  constructed by the honest dealer at time  $2\Delta + T_{\text{BC}}$ , and the proof holds as before.  $\square$

**Lemma 7.8 (Validity).** *Assume the network is synchronous. If the dealer is honest and has input  $\{S_r(x, y, z)\}_{r \in [\ell]}$ , such that  $\text{ValidityPred}_i(\text{share}_i) = 1$  for every honest  $i \in [n]$ , then either the dealer outputs an updated set  $\text{zeros}$  by time  $T_{\text{RelationVerification}}$ , such that additional  $t_a$  corrupted parties are added or each honest party outputs  $\text{OK}$  by a known time  $T_{\text{RelationVerification}}$ . Furthermore, if  $|\text{zeros}| \geq t_s - t_a$ , then every honest party outputs  $\text{OK}$  by a known time  $T_{\text{RelationVerification}}$ .*

*Proof.* Since the network is synchronous and the dealer is honest, by time  $\Delta$ , each honest party  $j \notin I$  receives polynomials  $f_i(x, y) = S(x, y, i)$ ,  $g_i(x, z) = S(x, i, z)$ ,  $h_i(y, z) = S(i, y, z)$ ,  $q_i(x, z) = L_i(\{S_1, \dots, S_r\})$ , and set  $\text{zeros}^*$ , where  $I$  is the set of malicious parties. As the dealer is honest,  $\text{ValidityPred}_i((\{f_i^r(x, y), g_i^r(x, z), h_i^r(y, z)\}_{r \in [\ell]}, q_i(x, z))) = 1$  and  $\text{consistent}(\text{share}_{i,i}^r, \text{share}_{i,i}^r) = 1$  for each  $r \in [\ell]$ . Consequently, at time  $\Delta$ , each honest party  $j \notin (I \cup \text{zeros}^*)$  sends the points  $\text{share}_{j,i}^r$  for each  $r \in [\ell]$  to each party  $i \in [n] \setminus \text{zeros}^*$  which are received by honest parties by time  $2\Delta$ . Each honest party  $i \notin I$  receives consistent points from honest parties  $j \notin I$ , and sets  $m_{i,j} = \text{OK}$  which is received by all honest parties (including the honest dealer) by time  $2\Delta + T_{\text{BC}}$ . As the dealer is honest, he sets the shares for every party  $j \in \text{zeros}^*$  correctly, and thus,

$\text{consistent}(\text{share}_i^r, \text{share}_{j,i}^r) = 1$  for every  $j \in \text{zeros}$ . Furthermore, for each  $m_{i,j} = (\text{NOK}, \dots)$  for some  $r \in [\ell]$  broadcast by an honest party  $i \notin I$  corresponding to a corrupted party  $j \in I$ . We now have that the honest parties form a  $n - t_s$  clique in the consistency graph  $\mathbb{G}$  constructed by the honest dealer at time  $2\Delta + T_{\text{BC}}$ . Thus, every honest party has at least  $n - t_s$  neighbors, and it must hold that each honest  $j \notin I$  is included in  $V'$ . Now, if  $|V' \cup \text{zeros}^*| < n - t_a$ , the dealer picks a set of  $t_a$  parties outside  $V' \cup \text{zeros}^*$  to add to set  $\text{zeros}$  and we are done. Else, assume  $|V' \cup \text{zeros}^*| \geq n - t_a$  (Note, this is guaranteed to happen if set  $|V'| \geq t_s - t_a$ ). The dealer adds an edge between every  $i \in \text{zeros}^*$  and  $j \in V'$ , and sets  $V' = V' \cup \text{zeros}^*$ . Thus, every party in  $V'$  has  $n - t_a$  neighbours in  $V'$ . Consider time  $T$  in which all Good messages between pairs of honest parties were received by the dealer. We will show that the dealer must either broadcast  $(\text{dense}, V', W)$  or  $(\text{star}, V', C, D)$  by time  $T$ . If there are at least  $n - t_s$  nodes in  $G$  with a degree of at least  $n - \min(t_a, \frac{t_a + t_s}{3}) + 1$ , the dealer constructs a graph  $W$  from these nodes and sends  $(\text{dense}, V', W)$  to all parties. If such a graph  $W$  does not exist, then there must be at least one honest party  $j \notin I$  whose degree is less than  $n - \min(t_a, \frac{t_a + t_s}{3}) + 1$ . In the synchronous setting, all missing edges (i.e.,  $\min(t_a, \frac{t_a + t_s}{3})$  edges) are connected to corrupted parties. Therefore, graph  $V'_i$  includes at most  $n' = n - \min(t_a, \frac{t_a + t_s}{3})$  parties and contains a clique of size at least  $n - t_s$  (i.e., all honest parties). Let  $t' = t_s - \min(t_a, \frac{t_a + t_s}{3})$ . According to Lemma 3.4, the Find-Star algorithm then identifies a  $(C, D)$  star with  $|C| \geq n' - 2(t_s - \min(t_a, \frac{t_a + t_s}{3}))$  and  $|D| \geq n' - (t_s - \min(t_a, \frac{t_a + t_s}{3}))$ . Consequently, the dealer sends  $(\text{star}, V', C, D)$  to all parties. In either case, every honest party outputs OK by time  $T_{\text{RelationVerification}}$ .  $\square$

**Lemma 7.9** (Termination). *If one honest party terminates by time  $T$ , then every honest party terminates so as well.*

*Proof.* An honest party outputs OK by time  $T_{\text{RelationVerification}}$  if it receives a message of the form  $(\text{dense}, V', W)$  or  $(\text{star}, V', C, D)$  from the dealer and all the corresponding checks in Step 5 pass. By the consistency property of the broadcast protocol, if one honest party receives such a message from the dealer, then all honest parties will eventually receive the same message. Since the verification checks in Step 5 depend on the dealer's message and messages broadcast during the protocol, all honest parties will eventually receive the same messages by the consistency property of broadcast protocol and perform the same checks on the same data. As a result, all honest parties will output OK by time  $T_{\text{RelationVerification}}$  and terminate.  $\square$

**Lemma 7.10** (Privacy). *Irrespective of the network type, when the dealer is honest, the adversary's view is identically distributed for every two polynomials that agree on the adversary's points.*

*That is, for every set of corrupted parties  $I \subseteq [n]$  of cardinality at most  $t_s$ , for every set of bivariate polynomials  $S_1, \dots, S_\ell, S'_1, \dots, S'_\ell \in \mathbb{F}^{\leq d, t_s}[x, y]$  such that for every  $i \in I$   $S_r(x, y, i) = S'_r(x, y, i)$ ,  $S_r(x, i, z) = S'_r(x, i, z)$ ,  $S_r(i, y, z) = S'_r(i, y, z)$  for every  $r \in [\ell]$ , and  $L_i(S_1, \dots, S_\ell) = L_i(S'_1, \dots, S'_\ell)$ , then  $\text{VIEW}_{S, I} = \text{VIEW}_{S', I}$ , where  $\text{VIEW}_{X, I}$  is a random variable denoting all messages that the*

corrupted parties  $I$  receive in an execution of the protocol where the honest dealer uses  $X$  as input.

*Proof.* Throughout the protocol, the dealer sends every party  $P_i$  its shares  $S_r(x, y, i), S_r(x, i, z), S_r(i, y, z)$  for every  $r \in [\ell]$ , and  $L_i(S_1, \dots, S_\ell)$ , and parties send each other subshares  $\text{share}_{i,j}^r$  as defined above. The subshares sent by  $P_j$  to  $P_i$  lie on  $P_i$ 's received shares, so  $P_i$  learns no new information from these messages. Parties then may send NOK complaining about each other if they see that their subshares are not consistent. Honest parties' subshares are always consistent with each other, and thus honest parties do not complain about each other. If an honest party complains about a corrupted party, it includes the corrupted party's subshares, not revealing any more information either.

This means that for any set of polynomials  $S'_1, \dots, S'_\ell$  such that  $\forall i \in I, r \in [\ell] \ S_r(x, y, i) = S'_r(x, y, i), S_r(x, i, z) = S'_r(x, i, z), S_r(i, x, z) = S'_r(i, x, z)$  and  $L_i(S_1, \dots, S_\ell) = L_i(S'_1, \dots, S'_\ell)$ , the corrupted parties' shares and subshares would be identical, meaning that the adversary sees identical messages in both executions.  $\square$

### Synchrony: The Case of a Corrupted Dealer

**Claim 7.11** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 7.4 perfectly implements Functionality 7.1 in the presence of at most  $t_s$  corrupted parties over a synchronous network, assuming that the dealer is corrupted.*

*Proof.* We describe the simulator  $\mathcal{S}$ :

1. The simulator receives the predicates  $\text{ValidityPred}_j$  for every  $j \notin I$ .
2. The simulator simulates an execution of the honest parties using the received validity predicates with the adversary  $\mathcal{A}$ . Since the honest parties have only have the validity predicates as input, the view of the adversary is identical to the real.
3. Once the first honest party outputs OK, let  $S_1, \dots, S_r$  be the polynomials defined in Lemma 5.9, computed from the views of the honest parties at that time. Send the functionality these polynomials.

The view of the adversary is clearly identical between the real and ideal, as the simulator simulates honest parties with their correct inputs. From Lemma 7.6, either no honest party outputs OK, or all honest parties OK, in which case the polynomials  $S_1, \dots, S_r$  were computed. Therefore, the honest parties output the same values.  $\square$

**Lemma 7.12** (Binding). *Assume the network is synchronous. If an honest party outputs OK by time  $T_{\text{RelationVerification}}$ , then all honest parties output OK at most on round later and there exists unique trivariate polynomials  $\{S'_r(x, y, z)\}_{r \in [\ell]}$  of degree at most  $d$  in each of the variables  $x, y, z$  such that it can be extracted from the views of honest parties. Furthermore, for at least  $2t_s + 1$  honest parties, it holds that  $\text{ValidityPred}_i(\text{share}_i) = 1$ .*

*Proof.* Let  $P_j$  be an honest party that outputs OK. This means it must have either received (dense,  $V', W$ ) or (star,  $V', C, D$ ) and verified Step 5 in Fig 7.4, and verified that  $|V'| \geq n - t_a$  and each party in  $V'$  has at least  $n - t_a$  neighbours in  $V'$ . Consider a set  $H \subseteq V'$ , consisting of honest parties of size  $t_s + \min(t_a, \frac{t_s+t_a}{3}) + 1$ . Let  $\{S_r(x, y, z)\}_{r \in [\ell]}$  be  $\ell$  unique trivariate polynomials of degree at least  $t_s + \min(t_a, \frac{t_s+t_a}{3})$  in  $x, y, z$ , such that  $S_r(x, y, k) = f_k^r(x, y)$  for every  $h \in H$  and  $r \in [\ell]$ , by claim 3.2. We will show that each honest party  $i \in V'$  has shares,  $\text{share}_i^r$  that are consistent with  $S_r(x, y, z)$  for each  $r \in [\ell]$ . As  $P_j$  output OK, it must have verified that no  $i, j \in G'$  such that  $P_i, P_j$  broadcast  $m_{i,j} = (\text{NOK}, f_i^r(j, y), f_i^r(x, j), g_i^r(j, z), g_i^r(x, j), h_i^r(j, z), h_i^r(y, j))$  and  $m_{j,i} = (\text{NOK}, f_j^r(i, y), f_j^r(x, i), g_j^r(i, z), g_j^r(x, i), h_j^r(i, z), h_j^r(y, i))$ , respectively by time  $2\Delta + T_{\text{BC}}$  such that  $(f_j^r(i, y), f_j^r(x, i), g_j^r(i, z), g_j^r(x, i), h_j^r(i, z), h_j^r(y, i)) \neq (h_i^r(y, j), g_i^r(x, j), h_i^r(j, z), f_i^r(x, j), g_i^r(j, z), f_i^r(j, y))$ , or else honest  $P_j$  wouldn't have accepted the graph. Consequently, for every honest party  $i \in V' \setminus H$  and  $h \in H$ , it must be the case that  $S_r(x, i, h) = f_h^r(x, i) = g_i^r(x, h)$ . As set  $|H| \geq t_s + \min(t_a, \frac{t_s+t_a}{3}) + 1$ ,  $S(x, i, z) = g_i^r(x, z)$  for each  $r \in [4]$ . Similarly,  $S(x, h, i) = f_i^r(x, h) = g_h^r(x, i)$  and  $S(i, y, h) = h_i^r(y, h) = f_h^r(i, y)$ . Thus,  $S(x, y, i) = f_i^r(x, y)$  and  $S(i, y, z) = h_i^r(y, z)$  for each  $r \in [\ell]$ . Additionally, note that  $V'$  is of size  $|V'| \geq n - t_a$ . Every honest party  $i \in V'$  has at least  $n - t_a$  neighbours in  $V'$ , and thus, must have broadcast OK. An honest party only broadcast OK if  $\text{ValidityPred}_i(\{S_r(x, y, i), S_r(x, i, z), S_r(i, y, z)\}_{r \in [4]}, L_i(\{S_r(x, y, z)\}_{r \in [\ell]})) = 1$ . Since there are at most  $t_s$  malicious in synchronous, at least  $n - t_a - t_s \geq 2t_s + 1$  honest parties in  $i \in V'$  such that  $\text{ValidityPred}_i(\{S_r(x, y, i), S_r(x, i, z), S_r(i, y, z)\}_{r \in [\ell]}, L_i(\{S_r(x, y, z)\}_{r \in [\ell]})) = 1$ .

In addition, all honest parties receive the same broadcasts as  $P_j$  at most one round later, accept the graph, and output OK.  $\square$

## Proofs for Asynchronous Execution

### Asynchrony: The Case of an Honest Dealer

**Claim 7.13** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 7.4 perfectly implements Functionality 7.1 in the presence of at most  $t_a$  corrupted parties over an asynchronous network, assuming that the dealer is honest.*

*Proof.* When the dealer is honest, then it sends to the functionality its polynomial  $S_1, \dots, S_\ell$  and set zeros. The functionality sends the shares  $(S_r(x, y, i), S_r(x, i, z), S_r(i, y, z))_{r \in [\ell]}, L_i(S_1, \dots, S_\ell)$  to the adversary for every corrupted party  $P_i$ , and possibly sends each party  $P_j$  its output share OK. It then runs the protocol with the adversary, while the simulated honest dealer uses the polynomials  $S'_1(x, y), \dots, S'_\ell$  satisfying  $S'_r(x, y, i) = S_r(x, y, i)$ ,  $S'_r(x, i, z) = S_r(x, i, z)$ ,  $S'_r(i, y, z) = S_r(i, y, z)$  and  $L_i(S'_1, \dots, S'_\ell) = L_i(S_1, \dots, S_\ell)$  for every corrupted party  $P_i$  and  $r \in [\ell]$ . Whenever an honest party  $P_j$  writes to its output tape in this simulated execution, the simulator allows the delivery of the output to the honest party  $P_j$  in the ideal execution. If the simulated dealer outputs a



set  $\text{zeros}'$  in the protocol, the adversary sends that set to the functionality and delivers the response to the dealer.

To show that the executions are perfectly distributed, we relied in the synchronous setting on privacy (Lemma 5.6), which allows the simulator to work with alternative inputs that are consistent with the shares of the corrupted parties and not with the polynomials that the dealer actually used in the execution. We rely on the same Lemma here as well. In the synchronous case, we also relied on Lemma 5.5 for correctness, showing that if the dealer is honest and uses some polynomials then each honest party outputs OK, or the dealer outputs an updated  $\text{zeros}$  set. Here, we rely on the correctness for the asynchronous setting, i.e., Lemma 5.11, which just shows that each party eventually outputs those values.  $\square$

**Lemma 7.14** (Validity). *Assume the network is asynchronous. If the dealer is honest and has inputs  $S_1, \dots, S_\ell$ , such that  $\text{ValidityPred}_i(\text{share}_i) = 1$  for every honest  $i \in [n]$ , and some party outputs shares, then each honest party eventually outputs OK.*

*Proof.* Give the network is asynchronous, there can be at most  $t_a$  malicious parties. As the dealer is honest, each honest party  $j \notin I$  eventually receives polynomials  $f_i(x, y) = S(x, y, i)$ ,  $g_i(x, z) = S(x, i, z)$ ,  $h_i(y, z) = S(i, y, z)$ ,  $q_i(x, z) = L_i(S_1, \dots, S_r)$  for  $r \in [\ell]$ , where  $I$  is the set of malicious parties, and set  $\text{zeros}^*$ . Furthermore,  $\text{ValidityPred}_i(\text{share}_i) = 1$  and  $\text{consistent}(\text{share}_{i,i}^r, \text{share}_{i,i}^r) = 1$  for  $r \in [\ell]$ . Additionally, as the dealer is honest, he sets the shares for every party  $j \in \text{zeros}^*$  correctly, and thus,  $\text{consistent}(\text{share}_i^r, \text{share}_{j,i}^r) = 1$  for every  $j \in \text{zeros}$ . Furthermore, for each  $m_{i,j} = (\text{NOK}, \dots)$  for some  $r \in [\ell]$  broadcast by an honest party  $i \notin I$  corresponding to a corrupted party  $j \in I$ . Assume some honest party  $P_k$  output OK. Then, it must have either received  $(\text{dense}, V', W)$  or  $(\text{star}, V', C, D)$ , and verified the checks in step 5. Consequently, from the consistency of broadcast, each honest receives the same messages as party  $P_k$ , and the checks from step 5 pass, and each honest party eventually outputs OK.  $\square$

**Lemma 7.15** (Termination). *If one honest party terminates, then every honest party eventually terminates.*

*Proof.* An honest party  $P_i$  outputs OK if it receives a message of the form  $(\text{dense}, V', W)$  or  $(\text{star}, V', C, D)$  from the dealer and all the corresponding checks in Step 5 pass. By the consistency property of the broadcast protocol, if one honest party receives such a message from the dealer, then all honest parties will eventually receive the same message. Since the verification checks in Step 5 depend on the dealer's message and messages broadcast during the protocol, all honest parties will eventually receive the same messages by the consistency property of broadcast protocol and perform the same checks on the same data. As a result, all honest parties will output OK eventually and terminate.  $\square$

### Asynchrony: The Case of a Corrupted Dealer



**Claim 7.16** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 7.4 perfectly implements Functionality 7.1 in the presence of at most  $t_a$  corrupted parties over an asynchronous network, assuming that the dealer is corrupted.*

*Proof.* The simulator receives the predicate  $\text{ValidityPred}_j$  for every  $j \notin I$  and runs the protocol while simulating the honest parties with these predicates. Note that the honest parties only have these values as inputs, and therefore the simulation is perfect.

When the first honest party  $P_j$  writes an output to its output tape, OK, then it must be that all honest parties will eventually also output OK, and there exist polynomials  $S_1, \dots, S_r$ . As follows from the proof of Lemma 5.13, from the message  $(W, C, D, G, F)$  that the dealer has broadcasted, it can reconstruct a unique bivariate polynomial  $S(x, y)$  from the shares of the honest parties in  $D$ . It sends  $S(x, y)$  to the trusted party, together with the set  $K$  of all honest parties. The trusted party then sends  $S(x, k), S(k, y)$  to each honest party  $P_k$ , yet those messages are not delivered yet. The simulator continues to simulate the protocol with the adversary. Whenever an honest party  $P_j$  in the simulated execution writes to its output tape, the simulator allows the delivery of the output of  $P_j$  (which is  $S(x, j), S(j, y)$ ) in the ideal model.

As we will see in Lemma 5.13, if one honest party receives an output, then eventually all honest parties receive an output, and all the outputs of the honest parties must lie on the same bivariate polynomial.  $\square$

**Lemma 7.17** (Binding). *Assume the network is asynchronous. If an honest party outputs OK, then all honest parties eventually do and there exists unique trivariate polynomials  $\{S'_r(x, y, z)\}_{r \in [\ell]}$  of degree at most  $d$  in each of the variables  $x, y, z$  such that it can be extracted from the views of honest parties. Furthermore, for at least  $2t_s + 1$  honest parties, it holds that  $\text{ValidityPred}_i(\text{share}_i) = 1$ .*

*Proof.* Let  $P_k$  be the first honest party to output OK, then the dealer must have broadcasted Dense or BigStar messages, and  $P_k$  validated the respective property in its local graph. We distinguish each case separately:

1.  $P_k$  received Dense Graph Message:

$P_k$  verified that  $|V'| \geq n - t_a$  and each party in  $V'$  has  $n - t_a$  neighbour in  $V$  and dense graph  $|W| \geq n - t_s$  and each party in  $W$  has degree at least  $n - \min(t_a, \frac{t_s + t_a}{3}) + 1$ . Consider a set  $H \subseteq W$ , consisting of honest parties of size  $t_s + \min(t_a, \frac{t_s + t_a}{3}) + 1$ . Let  $\{S_r(x, y, z)\}_{r \in [\ell]}$  be four unique trivariate polynomials, such that  $S_r(x, y, k) = f_h^r(x, y)$  for every  $h \in H$  and  $r \in [\ell]$ , where  $f_h^r(x, y)$  are the shares sent by the dealer to each party  $h \in H$ . Such trivariate polynomial exists according to claim 3.2. We will show that each honest party  $w \in W$  has shares,  $\text{share}_w^r$  that are consistent with  $S_r(x, y, z)$  for each  $r \in [\ell]$ . As  $|W| \geq n - t_s$  and each party in  $W$  has degree at least  $n - \min(t_a, \frac{t_s + t_a}{3}) + 1$ , any two honest parties  $w, h \in W$  have an intersection of size at least  $n - 2\min(t_a, \frac{t_s + t_a}{3}) + 2$ . As  $\frac{t_s + t_a}{3} \geq \min(t_a, \frac{t_s + t_a}{3})$ , the intersection is of size  $\frac{7}{3}t_s + \frac{t_a}{3} + 2$ , out of which at least  $\frac{4}{3}t_s + \frac{t_a}{3} + 2$  are honest. We refer to this honest intersection set as set  $K$ . This means for every

$k \in K$ ,  $m_{k,w} = \text{OK}$ ,  $m_{w,k} = \text{OK}$  were broadcast by honest parties  $P_k$  and  $P_w$  respectively. Similarly,  $m_{k,h} = \text{OK}$ ,  $m_{w,h} = \text{OK}$  were broadcast by honest parties  $P_k$  and  $P_h$  respectively. This implies that  $P_h$  and  $P_k$  verified that  $S_r(x, k, h) = f_h(x, k) = g_k(x, h)$ ,  $P_k$  and  $P_w$  verified that  $g_k(w, z) = h_w(k, z)$ . In particular, for some  $z = h \in H$ ,  $S(w, k, h) = g_k(w, h) = h_w(k, h)$ . Consequently, as  $|k| \geq t_s + \min(t_a, \frac{t_s+t_a}{3}) + 1$ ,  $S(w, y, h) = h_w(y, h)$ . As this holds for every  $h \in H$ ,  $S(w, y, z) = h_w(y, z)$  for every  $w \in W$ . Similarly,  $P_h$  and  $P_k$  verified that  $S_r(k, y, h) = f_h(k, y) = h_k(y, h)$ ,  $P_k$  and  $P_w$  verified that  $g_w(k, z) = h_k(w, z)$ . Thus, by similar arguments from earlier,  $S(x, w, z) = g_w(x, z)$  for every  $w \in W$ . And finally,  $P_h$  and  $P_k$  verified that  $S_r(h, k, z) = h_h(k, z) = g_k(h, z)$ ,  $P_k$  and  $P_w$  verified that  $f_w(k, z) = g_k(x, w)$ . Thus, by similar argument from earlier,  $S(x, y, w) = f_w(x, y)$  for every  $w \in W$ .

2.  $P_k$  received BigStar message:

Thus,  $P_k$  verified that  $C$  is of size  $|C| \geq n' - 2t' \geq n - \min(t_a, \frac{t_a+t_s}{3}) - 2(t_s - \min(t_a, \frac{t_a+t_s}{3}))$ . As  $\frac{t_a+t_s}{3} \geq \min(t_a, \frac{t_a+t_s}{3})$ ,  $|C| \geq t_s + t_a + \frac{\min(t_s+t_a)}{3} + 1$ . As the network is asynchronous, there are at most  $t_a$  malicious parties. Thus, the shares of honest parties in clique  $C$  define four unique trivariate polynomials,  $\{S_r(x, y, z)\}_{r \in [\ell]}$ , such that  $S_r(x, y, c) = f_c^r(x, y)$ . We will show that every honest party in  $D$  holds shares that are consistent with these trivariate polynomials. By definition of  $C, D$  star, for every party  $c \in C$  and  $d \in D$ , there exists an edge  $(c, d) \in G$ . Thus, every party  $d \in D$  has an edge to every honest party  $c \in C$ . Thus, every honest  $c$  and  $d$  must have verified that  $S(x, c, d) = g_c(x, d) = f_d(x, c)$ ,  $S(x, d, c) = g_d(x, c) = f_c(x, d)$ , and  $S(d, y, c) = h_d(y, c) = f_c(d, y)$ . As this holds for every honest  $c \in C$ , then,  $S(x, y, d) = f_d(x, y)$ ,  $S(x, d, z) = g_d(x, z)$ , and  $S(d, y, z) = h_d(y, z)$ . Finally, Additionally, note that  $V'$  is of size  $|V'| \geq n - t_a$ . Every honest party  $i \in V'$  must have broadcast OK at least once. An honest party only broadcast OK if  $\text{ValidityPred}_i(\{S_r(x, y, i), S_r(x, i, z), S_r(i, y, z)\}_{r \in [\ell]}, L_i(\{S_r(x, y, z)\}_{r \in [\ell]})) = 1$ . Since there are at most  $t_a$  malicious in asynchronous, at least  $n - 2t_a \geq 2t_s + 1$  honest parties in  $i \in V'$  such that  $\text{ValidityPred}_i(\{S_r(x, y, i), S_r(x, i, z), S_r(i, y, z)\}_{r \in [\ell]}, L_i(\{S_r(x, y, z)\}_{r \in [\ell]})) = 1$ .

In addition, all honest parties receive the same broadcasts as  $P_k$ , accept the graph, and output OK.

□

**Efficiency** In every call to  $\Pi_{\text{RelationVerification}}$ , the dealer sends every party a constant number of bivariate polynomials and a set of indices, for  $O(n^3 \log n)$  communication in total. Parties then send each other subshares consisting of a constant number of univariate polynomials, for  $O(n^3 \log n)$  communication as well. Each party broadcasts a single vector of size  $O(n \log n)$ . Using a broadcast with complexity  $O(n^3 \log n + nL)$  for messages of size  $L$ , we get a total complexity of  $O(n^4 \log n)$  for sharing  $O(n^3)$  secrets.

Similarly to Section 5.3, parties can batch their broadcasts, sending the same size vectors for sharing any number of instances. Participating in  $O(Xn)$  in-

stances of the protocol, sharing  $O(Xn^4)$  secrets, parties send  $O(Xn^4 \log n)$  bits in point-to-point communication, and  $O((n^4 + n^2) \log n)$  bits in the broadcast protocol. In total, the protocol requires  $O(Xn^4 \log n)$  communication when sharing  $O(Xn^4)$  secrets, or  $O(\log n)$  per secret.

## 7.2 Triple Sharing construction

In this section, we show how to realize the product-relation verification functionality using our trivariate sharing primitive (Subsection 7.1).

**Definition 7.18** (Triple Sharing Protocol). *A Triple Sharing Protocol has a designated party acting as the dealer. We say  $\Pi_{\text{TriSharing}}$  is a Triple Sharing protocol over the finite field  $\mathbb{F}$  against an adversary corrupting  $t_s$  parties in a synchronous network and  $t_a$  parties in an asynchronous network if the following holds:*

### 1. Synchronous Network:

- (a)  **$t_s$ -Validity:** *If the dealer is honest, then there exists polynomials  $(A^u(x, y), B^u(x, y), C^u(x, y))_{u \in \{0, \dots, t_a\}}$  of degree  $t_s + t_a$  or less in  $x$  and  $t_s$  or less in  $y$  such that every honest party  $P_k$  outputs shares  $(S^u(x, j), S^u(j, y))$  for every  $S \in \{A, B, C\}$  by time  $T_{\text{TriSharing}}$ .*
- (b)  **$t_s$ -Correctness:** *If there exists an honest party that outputs shares, then there exists polynomials  $(A^u(x, y), B^u(x, y), C^u(x, y))_{u \in \{0, \dots, t_a\}}$  of degree  $t_s + t_a$  or less in  $x$  and  $t_s$  or less in  $y$  such that every honest party  $P_j$  outputs  $(S^u(x, j), S^u(j, y))_{u \in \{0, \dots, t_a\}}$  for every  $S \in \{A, B, C\}$ , and  $A(-k, 0) \cdot B(-k, 0) = C(-k, 0)$  for every  $k \in \{0, \dots, t_a\}$ .*
- (c)  **$t_s$ -Termination:** *If the dealer is corrupted and some honest party outputs shares, all honest parties output shares at most 1 round after that.*

### 2. Asynchronous Network:

- (a)  **$t_a$ -Correctness:** *If there exists an honest party that outputs shares, then there exists polynomials  $(A^u(x, y), B^u(x, y), C^u(x, y))_{u \in \{0, \dots, t_a\}}$  of degree  $t_s + t_a$  or less in  $x$  and  $t_s$  or less in  $y$  such that every honest party  $P_j$  that outputs shares, outputs  $(S^u(x, j), S^u(j, y))_{u \in \{0, \dots, t_a\}}$  for every  $S \in \{A, B, C\}$ , and  $A(-k, 0) \cdot B(-k, 0) = C(-k, 0)$  for every  $k \in \{0, \dots, t_a\}$ .*
- (b)  **$t_a$ -Termination:** *If some honest party outputs shares, all honest parties eventually output shares.*

### 3. Privacy:

*Irrespective of the network type, when the dealer is honest, the adversary's view is identically distributed for every set of polynomials that agrees on the adversary's points.*

*That is, for every set of corrupted parties  $I \subseteq [n]$  of cardinality at most  $t_s$ , for every bivariate polynomials  $(A^u(x, y), B^u(x, y), C^u(x, y), A'^u(x, y), B'^u(x, y), C'^u(x, y))_{u \in \{0, \dots, t_a\}}$ , each in  $\mathbb{F}^{\leq d, t_s}[x, y]$ , such that for every  $u \in \{0, \dots, t_a\}$ ,  $S \in \{A, B, C\}$ ,  $S^u(x, i) = S'^u(x, i)$ ,  $S^u(i, y) = S'^u(i, y)$  for every  $i \in I$ , then  $\text{VIEW}_{(A^u, B^u, C^u), I} = \text{VIEW}_{(A'^u, B'^u, C'^u), I}$ , where  $\text{VIEW}_{X, I}$  is a random variable denoting all messages that the corrupted parties  $I$  receive in an execution of the protocol, where  $X$  consists of the polynomials defined in the validity property.*

**Functionality 7.19: Triple Sharing With dealer**

The functionality is parametrized by the indices of the corrupted parties  $I \subseteq [n]$  and a known dealer  $P_d$ .

**1. Honest Dealer:**

- (a) If the adversary sends a (**setShares**, **shares**) command with **shares** containing a pair of polynomials  $f_{X,i}^w(x), g_{X,i}^w(y)$  for every  $w \in \{0, \dots, t_a\}, i \in I, X \in \{A, B, C\}$ , the adversary verifies that  $\forall w \in \{0, \dots, t_a\}, i, j \in I, S \in \{A, B, C\} f_{S,i}^w(j) = g_{S,j}^w(i)$ . If this holds, it stores **shares**.
- (b) If the adversary sends a (**setZeros**, **zeros**) command with **zeros**  $\subseteq [n]$ , then verify that  $|\text{zeros}| \leq t_s - t_a$ , and if the network is synchronous, verify that **zeros**  $\subseteq I$ . If that is the case, store **zeros**.
- (c) The dealer sends the command (**ready**) to the functionality.
  - If no **set shares** is stored, define it as being  $\emptyset$  and if no **zeros** set is stored, define it as being  $\emptyset$ . If there are shares in **shares** for any party  $P_i$  such that  $i \in \text{zeros}$ , remove their shares from **shares** (or set them to 0).
  - For every  $w \in \{0, \dots, t_a\}$ , the functionality uniformly samples polynomials  $A^w(x, y), B^w(x, y), C^w(x, y)$  of degrees  $t_s + t_a$  in  $x$  and  $t_s$  in  $y$  under the condition that for every  $w \in \{0, \dots, t_a\}$ : (1) for every  $i \in I, S \in \{A, B, C\} S^w(x, i) = f_{S,i}^w(x), S^w(i, y) = g_{S,i}^w(y)$ , (2) for every  $j \in \text{zeros}, w \in \{0, \dots, t_a\}, S \in \{A, B, C\} S^w(x, i) = S^w(i, y) = 0$ , and (3) for every  $w \in \{0, \dots, t_a\}, k \in \{0, \dots, t_a\} A^w(-k, 0) \cdot B^w(-k, 0) = C^w(-k, 0)$ . The functionality stores the polynomials. Proceed to sending outputs.

**2. Corrupted Dealer:**

- (a) If the adversary sends (**setInput**,  $(A^w(x, y), B^w(x, y), C^w(x, y))_{w \in \{0, \dots, t_a\}}$ ), the functionality checks that all polynomials are of degree  $t_s + t_a$  or less in  $x$  and  $t_s$  or less in  $y$ , and that for every  $w \in \{0, \dots, t_a\}, k \in \{0, \dots, t_a\} A^w(-k, 0) \cdot B^w(-k, 0) = C^w(-k, 0)$ . If this holds, the functionality stores the polynomials and proceeds to sending outputs. synchrony

- 3. Output:** The functionality sends every party  $P_j$  such that  $j \in [n] \setminus I$  the values  $(S^w(x, j), S^w(j, x))$  for every  $w \in \{0, \dots, t_a\}, S \in \{A, B, C\}$  (non-simultaneously in synchrony, see Remark 3.10).

Recall that  $d$  is defined above to be  $t_s + \min(t_a, (t_a + t_s)/3)$ , the degree of the trivariate polynomial.

Note that in the above functionality, the polynomials are sampled uniformly in a manner consistent with the adversary's chosen shares. In addition, in asynchrony, the polynomials are sampled uniformly under the condition that parties whose indices are in **zeros** (who are chosen by the adversary) have their shares equal 0. Since in asynchrony, the adversary controls at most  $t_a$  parties and **zeros** is of size  $t_s - t - a$  at the most, this means that the adversary learns at most  $t_s$  shares on each of the polynomials. Similarly to the observation of [45], this

means that the adversary's view is identical to that of an adversary controlling  $t_s$  parties, mainting the secrecy of the secrets stored in  $A^w(-k, 0), B^w(-k, 0)$  and  $C^w(-k, 0)$  for every  $w \in \{0, \dots, t_a\}, k \in \{0, \dots, t - a\}$ .

---

**Protocol 7.20:**  $\Pi_{\text{TriSharing}}$ 


---

**Input.** Let  $U = \{0, \dots, t_a\}^2$ . The dealer maintains a set **zeros**, which is initially set to empty. Each party  $P_i$  maintains a counter **count**, initially set to 1.

**The protocol.**

**Share or Restart Dealer  $P_1$ :**

1.  $P_1$  samples bivariate polynomials  $\{A^w(x, y), B^w(x, y), C^w(x, y)\}_{w \in \{0, \dots, t_a\}}$  of degree  $t_s + t_a$  in  $x$  and  $t_s$  in  $y$ , such that for every  $k \in \{0, \dots, t_a\}, w \in \{0, \dots, t_a\}$ ,  $A^w(-k, 0) \cdot B^w(-k, 0) = C^w(-k, 0)$  under the condition that  $\forall j \in \text{zeros}, S \in \{A, B, C\}, w \in \{0, \dots, t_a\}$   $S^w(x, j) = S^w(j, x) = 0$ .  $P_1$  invokes  $\{\Pi_{\text{CompleteSync}}(S^w(x, y))\}_{S \in \{A, B, C\}, w \in [\text{pack}+1]}$ .
  2.  $P_1$  computes  $E^u(x) = A^u(x) \cdot B^u(x) - C^u(x)$ , and defines the coefficients  $e_1^u, \dots, e_{2t_s}^u$  such that  $E^u(x) = \sum_{i=1}^{2t_s} e_i^u \cdot x^i$  for every  $u \in U$  (implicitly,  $e_0^u = 0$ ).
  3.  $P_1$  chooses  $\ell = \lceil \frac{2t_s(t_a+1)^2}{(d-t_s+1)^3} \rceil$  trivariate polynomials  $S_1(x, y, z), \dots, S_\ell(x, y, z)$  of degree  $d$  in all three variables and packs the coefficients as described in Section 2.4 of [3], dividing the coefficients of the polynomial over  $\ell$  polynomials instead of 4.
  4.  $P_1$  invokes  $\Pi_{\text{TriSharing}}$  on each trivariate polynomial as input. If  $\Pi_{\text{TriSharing}}$  outputs an updated set **zeros'** by time  $T_{\text{RelationVerification}}$  such that **zeros'**  $\setminus$  **zeros**  $\neq \emptyset$ , then update **zeros** = **zeros**  $\cup$  **zeros'**. If after the update **zeros** would have more than  $t_s - t_a$  indices, store an arbitrary subset of **zeros** of size  $t_s - t_a$ , e.g., the minimal indices. Broadcast (Restart) and go to step 1. Else, if it outputs OK, broadcast (Terminate) and terminate.
  5. Each party  $P_i$  waits to receive either (Terminate) or (Restart) from the dealer. If it receives (Terminate) and receives OK from the corresponding  $\Pi_{\text{TriSharing}}$  call, it outputs the shares obtained in step 1. Else, if it receives (Restart), it increments **count** and goes to step 1 only if **count**  $\leq \frac{t_s - t_a}{t_a}$ .
- 

## Proofs for Synchronous Execution

### Synchrony: The Case of an Honest Dealer

**Claim 7.21** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 7.20 perfectly implements Functionality 7.19 in the presence of at most  $t_s$  corrupted parties over a synchronous network, assuming that the dealer is honest.*

*Proof.* The simulator  $\mathcal{S}$  works as follows.

1. Run the protocol honestly with the adversary  $\mathcal{A}$ , simulating the honest parties as described in the protocol with no inputs. Initialize a set  $\text{zeros} = \emptyset$ . Each time that the adversary sends a set  $\text{zeros}'$  to the functionality, update  $\text{zeros} = \text{zeros} \cup \text{zeros}'$ , and keep only the minimal  $t_s - t_a$  indices. If the simulator sees that the parties receive outputs in a given iteration, it stores the shares  $S^w(x, i), S^w(i, y)$  received in that iteration for every corrupted party  $P_i$  and  $w \in \{0, \dots, t_a\}, S \in \{A, B, C\}$ .
2. Send the functionality the commands `setShares` and `setZeros` with the stored shares and zeros sets.
3. Output the output of the adversary.

We claim that in the synchronous setting, the ideal execution is identical to the real execution. Lemma 7.23 shows that for every pair of polynomials with consistent shares for the corrupted parties, the adversary's view is consistent. In addition, given these shares, the functionality uniformly samples a polynomial consistent with the shares and having 0 shares for every index in  $\text{zeros}$ . Lemmas 7.22 and 7.24 show that all honest parties receive shares on the polynomials as output. The dealer does the same in the real protocol, and thus honest parties' outputs are distributed identically.  $\square$

**Lemma 7.22** (Validity). *If the dealer is honest, then there exists polynomials  $(A^u(y), B^u(y), C^u(y))_{u \in \{0, \dots, t_a\}}$  of degree  $t_s$  or less such that every honest party  $P_k$  outputs shares  $(A^u(j), B^u(j), C^u(j))_{u \in \{0, \dots, t_a\}}$ .*

*Proof.* This follows directly from the  $t_s$ -Correctness property of  $\Pi_{\text{CompleteSync}}$ , from the validity property of  $\Pi_{\text{RelationVerification}}$ , and from the fact it takes at most  $\frac{t_s - t_a}{t_a}$  restarts for the honest dealer to add  $t_s - t_a$  corrupted parties to set  $\text{zeros}$ .  $\square$

**Lemma 7.23** (Privacy). *Irrespective of the network type, when the dealer is honest, the adversary's view is identically distributed for every set of polynomials that agrees on the adversary's points.*

*That is, for every set of corrupted parties  $I \subseteq [n]$  of cardinality at most  $t_s$ , for every bivariate polynomials  $(A^u(x, y), B^u(x, y), C^u(x, y), A'^u(x, y), B'^u(x, y), C'^u(x, y))_{u \in \{0, \dots, t_a\}}$ , each in  $\mathbb{F}^{\leq d, t_s}[x, y]$ , such that for every  $u \in \{0, \dots, t_a\}, S \in \{A, B, C\}$ ,  $S^u(x, i) = S'^u(x, i), S^u(i, y) = S'^u(i, y)$  for every  $i \in I$ , then it is the case that  $\text{VIEW}_{(A^u, B^u, C^u), I} = \text{VIEW}_{(A'^u, B'^u, C'^u), I}$ , where  $\text{VIEW}_{X, I}$  is a random variable denoting all messages that the corrupted parties  $I$  receive in an execution of the protocol, where  $X$  consists of the polynomials defined in the validity property.*

*Proof.* The protocol may restart a constant number of times, and each time the protocol restarts new polynomials are sampled independently.

In the final iteration of the protocol, the adversary receives shares on the polynomials  $S_0, \dots, S_{t_a}$  sampled in that iteration for every  $A, B, C$ . From the privacy of  $\Pi_{\text{CompleteSync}}$ , in the call to  $\Pi_{\text{CompleteSync}}$ , it only learns its shares  $S(x, i), S(i, y)$  for each  $S \in \{A, B, C\}$  and for each faulty party  $P_i$ . Similarly, from

privacy of `RelationVerification`, in the call to `RelationVerification`, it only learns the shares  $S_r(x, y, i), S_r(x, i, z), S_r(i, y, z)$  and  $L_i(S_1, \dots, S_r)$  for each  $r \in [\ell]$  and faulty party  $P_i$ .

For any set of polynomials  $A'^u, B'^u, C'^u$  with the same adversarial shares, the adversary receives the exact same shares from  $\Pi_{\text{CompleteSync}}$ . As is shown in [3], the number of trivariate polynomials consistent with the adversary's view is identical for any set of shares the adversary received. Therefore, using any set of polynomials  $A'^u, B'^u, C'^u$  with the same adversarial shares yields the exact same view.  $\square$

**Lemma 7.24** ( $t_s$ -Termination). *If some honest party outputs shares, all honest parties output shares at most 1 round after that.*

*Proof.* This follows from termination of  $\Pi_{\text{RelationVerification}}$ , and  $t_s$ -commitment and  $t_s$ -correctness of  $\Pi_{\text{CompleteSync}}$ .  $\square$

### Synchrony: The Case of a Corrupted Dealer

**Claim 7.25** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 7.20 perfectly implements Functionality 7.19 in the presence of at most  $t_s$  corrupted parties over a synchronous network, assuming that the dealer is corrupted.*

*Proof.* We describe the simulator  $\mathcal{S}$ :

1. The simulator simulates an execution of the honest parties (who have no input), with the adversary  $\mathcal{A}$ . Since the honest parties have no input, the view of the adversary is identical to the real.
2. Once the first honest party outputs a value, let the polynomials defined in Lemma 7.26 be  $(A^w(x, y), B^w(x, y), C^w(x, y))_{w \in \{0, \dots, t_a\}}$ . Send these polynomials to the functionality in a `setInput` command.

The view of the adversary is clearly identical between the real and ideal, as the simulator simulates honest parties that have no inputs. We now show that the output of the environment in the ideal execution is identical to its output in the real. This follows directly from Lemmas 7.26 and 7.24.  $\square$

**Lemma 7.26** ( $t_s$ -Correctness). *If there exists an honest party that outputs shares, then there exists polynomials  $(A^u(y), B^u(y), C^u(y))_{u \in \{0, \dots, t_a\}}$  of degree  $t_s$  or less such that every honest party  $P_j$  outputs  $(A^u(j), B^u(j), C^u(j))_{u \in \{0, \dots, t_a\}}$ .*

*Proof.* This follows from the  $t_s$ -Correctness and  $t_s$ -Commitment properties of  $\Pi_{\text{CompleteSync}}$  and from the termination of  $\Pi_{\text{RelationVerification}}$ .  $\square$

### Proofs for Asynchronous Execution



### Asynchrony: The Case of an Honest Dealer

**Claim 7.27** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 7.20 perfectly implements Functionality 7.19 in the presence of at most  $t_a$  corrupted parties over an asynchronous network, assuming that the dealer is honest.*

*Proof.* The simulator  $\mathcal{S}$  works identically to the simulator in Lemma 7.21, except it now delivers the honest parties' outputs when they are delivered in the simulated asynchronous execution. In particular, it simulates all honest parties, stores and updates a set **zeros**, and finally stores shares it received from the functionality. It then sends the functionality the shares and zeros, and delivers the output to the honest parties according to the asynchronous execution.

In the proof of Lemma 7.21 we relied on Lemmas 7.23, 7.22, and 7.24, whereas now we rely on Lemmas 7.23, 7.28, and 7.29.  $\square$

**Lemma 7.28** ( $t_a$ -Correctness). *If there exists an honest party that outputs shares, then there exists polynomials  $(A^u(y), B^u(y), C^u(y))_{u \in \{0, \dots, t_a\}}$  of degree  $t_s$  or less such that every honest party  $P_j$  outputs  $(A^u(j), B^u(j), C^u(j))_{u \in \{0, \dots, t_a\}}$ .*

*Proof.* This follows from the  $t_s$ -Correctness and  $t_s$ -Commitment properties of  $\{ \Pi_{\text{CompleteSync}}, \text{ and } t_a\text{-termination of } \Pi_{\text{RelationVerification}} \}$ .  $\square$

**Lemma 7.29** ( $t_a$ -Termination). *If some honest party outputs shares, all honest parties eventually output shares.*

*Proof.* This follows from termination of  $\Pi_{\text{RelationVerification}}$ , and  $t_a$ -commitment and  $t_a$ -correctness of  $\Pi_{\text{CompleteSync}}$ .  $\square$

### Asynchrony: The Case of a Corrupted Dealer

**Claim 7.30** (Simulation). *Assume that  $n > 3t_s + t_a$ . Protocol 7.20 perfectly implements Functionality 7.19 in the presence of at most  $t_a$  corrupted parties over an asynchronous network, assuming that the dealer is corrupted.*

*Proof.* The simulator  $\mathcal{S}$  works identically to the simulator in Lemma 7.25, except it now delivers the honest parties' outputs when they are delivered in the simulated asynchronous execution. In particular, it simulates all honest parties, and if any party computes an output, it computes polynomials  $(A^w(x, y), B^w(x, y), C^w(x, y))_{w \in \{0, \dots, t_a\}}$  from the views of the honest parties, and provides these polynomials to the functionality. It then delivers the outputs to the honest parties according to the asynchronous execution.

In the proof of Lemma 7.25 we relied on Lemmas 7.26 and 7.24, whereas now we rely on Lemmas 7.28 and 7.29.  $\square$

**Efficiency** The protocol consists of  $O(n)$  calls to  $\Pi_{\text{CompleteSync}}$  and  $O(1)$  calls to  $\Pi_{\text{RelationVerification}}$  in each restart. Parties restart  $\frac{t_s - t_a}{t_a} = O(1)$  times in total. As discussed in Section 6.3, this means that the total communication in the calls to  $\Pi_{\text{CompleteSync}}$  is  $O(n^4 \log n)$ . In addition, the constant number of calls



to  $\Pi_{\text{RelationVerification}}$  require  $O(n^4 \log n)$  communication in total as well. In total,  $O(n^2)$  triplet in the protocol, and thus the cost per triplet is  $O(n^2 \log n)$

Note that when sharing fewer than  $O(n^2)$  secrets, the broadcast costs dominate the costs of the internal protocol with the rest of the cost lowering accordingly. The broadcast costs are  $O(n^4 \log n + Ln^2)$  for broadcasting messages of size  $L$ , with  $L$  being  $O(n \log n)$  in these protocols. Sending all of the information for  $n$  instances with *different* dealers in one broadcast allows us to retain a cost of  $O(n^4 \log n + L'n^2)$  with  $L' = O(n^2 \log n)$  being the length of all  $n$  vectors combined. Therefore, when sharing fewer secrets, we still retain a total cost of  $O(n^4 \log n)$  for  $n$  concurrent executions of the protocol.

## 8 Perfectly Secure Network Agnostic MPC

In this section, we give an overview of the building blocks required for the complete MPC protocol, which follows the common construction in literature as in [2, 3, 32]. We refer the reader to [32] for detailed explanation. First, we give overview of some building blocks needed for the complete MPC protocol and then, we give explain how to finally get our NA (network agnostic) MPC based on these building blocks.

### 8.1 Secret Reconstruction

Since the outcome of our VSS is secrets in Shamir-shared format, we discuss how such sharing can be reconstructed efficiently. We use two standard ways of reconstruction: (1) private reconstruction, and (2) batched public reconstruction.

**Private Reconstruction.** We describe how to privately reconstruct a degree- $t_s$  shared secret at a designated party  $P$ . In this procedure, every party sends its share to  $P$ , who attempts to recover the underlying secret. The party  $P$  collects  $2t_s + 1$  shares that are consistent with a single degree- $t_s$  polynomial. Note, as  $n > 3t_s + t_a$ , it is guaranteed that  $P$  would receive such shares eventually under any network conditions. To identify this polynomial,  $P$  repeatedly applies online error correction (OEC) (see the full version of the paper for more detail). Once such a polynomial is obtained, it is guaranteed to be the correct degree- $t_s$  polynomial, since it coincides with the shares of at least  $t_s + 1$  honest parties. However, this is inefficient as it incurs  $O(\log n)$  communication.

**Batched Public Reconstruction.** Reconstructing  $t_a + 1$  Shamir-shared secrets would be equivalent to performing  $(t_a + 1)n$  private reconstructions, which incurs  $O(n^3 \log n)$  communication. To reduce this overhead, a batched reconstruction technique (from [36]) allows the recovery of  $t_a + 1$  shared values with only  $O(n^2 \log n)$  bits of communication, yielding an amortized cost of  $O(n \log n)$  per secret. The core idea is that instead of reconstructing each secret separately, the parties bundle  $t_a + 1$  sharings together and encode them into  $n$  fresh shares using a linear error-correcting code (e.g., Reed-Solomon), which tolerates up to  $t_s$  adversarial errors. Each party then locally reconstructs just one of these new shares via private reconstruction, and by combining their results, all parties can jointly recover the original  $t_a + 1$  values.

### 8.2 MPC Protocol

The MPC protocol uses the packed VSS (Section 6), the verifiable triple sharing (Section 7) and the rest of the building blocks which are taken from [32]. Our MPC protocol has two phases: (i) Preparing the Beaver triples and input sharing, and (ii) Circuit evaluation using the batched Beaver multiplication.

### 8.3 Input sharing and Preparing Beaver Triples

This section is further divided into the following three subsections:

**Input and Verifiable Sharing.** In this phase, using the verifiable triple sharing protocol (see the full version of the paper for more details), each party acting as a dealer is made to Shamir-share the required number of triples  $(a, b, c)$  such that  $c = ab$  holds. Each party also uses the packed VSS described in Section 6 to share its inputs. The total number of triples each party must share is determined by the size of the computation circuit.

**Agreement on Core Set.** Given the possibility of the protocol running in the asynchronous communication model, parties cannot wait indefinitely for all others to complete their input and triple sharing. Malicious parties may delay or completely skip their instances, potentially stalling the protocol. To mitigate this, the honest parties agree on a common subset of dealers, referred to as the *Core*, consisting of at least  $n - t_s$  parties. This set is guaranteed to have both the input and triple sharing instances eventually terminate for all parties. Once the triple sharing from all dealers in the Core is complete, the parties proceed to execute the triple extraction protocol. Notably, our ACS ensures that all honest parties are in the Core if the network is synchronous.

**Triple Extraction.** The final component is a triple extraction protocol. The triple extraction protocol, consumes a single verified multiplication triple,  $(\langle a_i \rangle, \langle b_i \rangle, \langle c_i \rangle)$ , held by each party  $P_i \in \text{Core}$  from the previous phase. From this, the parties derive  $h + 1 - t_s$  fresh triples unknown to all participants, where  $h = \lfloor (|\text{Core}| - 1)/2 \rfloor$ . For ease of notation, let  $m = |\text{Core}|$  and assume  $\text{Core} = \{P_1, \dots, P_m\}$ . The communication complexity of the protocol is  $\mathcal{O}(n^2)$  point-to-point messages. At a high level, the idea is to transform the  $m$  initial shared triples  $(\langle a_i \rangle, \langle b_i \rangle, \langle c_i \rangle)$  for  $i \in [m]$  into correlated triples  $(\langle x_i \rangle, \langle y_i \rangle, \langle z_i \rangle)$  such that the sets  $\{x_i, y_i, z_i\}_{i \in [m]}$  interpolate three polynomials  $X(\cdot), Y(\cdot), Z(\cdot)$  of degrees  $h, h$ , and  $2h$ , satisfying the relation  $X(\cdot) \cdot Y(\cdot) = Z(\cdot)$ . Specifically, for each  $i \in [m]$ , we have  $X(i) = x_i, Y(i) = y_i$ , and  $Z(i) = z_i$ , with  $1, \dots, m$  being publicly known, distinct field elements. By construction, the adversary learns  $(x_i, y_i, z_i)$  only if  $P_i$  is compromised. Therefore, the adversary's knowledge is limited to at most  $t_s$  points on each of the polynomials, preserving a degree of freedom of  $h + 1 - t_s$  in  $X(\cdot), Y(\cdot)$  (and hence  $Z(\cdot)$ ). The final output consists of evaluations of these polynomials at  $h + 1 - t_s$  publicly known positions  $\beta_1, \dots, \beta_{h+1-t_s}$ , which serve as the extracted multiplication triples.

The procedure is as follows. For indices  $i \in \{1, \dots, h + 1\}$ , the assignments  $x_i = a_i, y_i = b_i, z_i = c_i$  are made directly. For indices  $i \in \{h + 2, \dots, m\}$ , the values  $\langle x_i \rangle$  and  $\langle y_i \rangle$  are obtained *non-interactively* as linear combinations of  $\{x_j, y_j\}_{j \in [h+1]}$ . Recall that  $z_i$  is supposed to equal  $x_i \cdot y_i$  for every  $i \in \{h + 2, \dots, m\}$ . Therefore, parties compute a sharing of  $z_i$  using the sharing of  $x_i$  and  $y_i$ . This can be done via Beaver's method, using  $\langle x_i \rangle, \langle y_i \rangle$ , and the triple  $(\langle a_i \rangle, \langle b_i \rangle, \langle c_i \rangle)$ . If  $P_i$  is corrupted, then the values  $(x_i, y_i, z_i)$  are revealed to the adversary, as expected. Note that, in total, at most  $t_s$  triples are provided by corrupted parties, and thus in total the adversary learns at most  $t_s$  values

$x_i, y_i, z_i$ , either by them being directly assigned to be the values  $a_i, b_i, c_i$  for  $i \in \{1, \dots, h+1\}$ , or by information being leaked to it after its triplet was used to compute  $z_i$  for  $i \in \{h+2, \dots, m\}$ . Overall, the triple extraction step reduces to a batch of  $\mathcal{O}(h)$  Beaver multiplications, requiring  $\mathcal{O}((nh + n^2) \log n)$  bits of communication under batched public reconstruction.

#### 8.4 Circuit Evaluation

The second phase of the MPC uses the shares and precomputed degree- $t$  shared multiplication triples to evaluate the circuit. For the linear gates, parties locally apply the linear operation on their respective shares of the inputs. However, the circuit multiplication gates are evaluated via batched Beaver multiplication. This follows the well-known randomization method introduced by Beaver [16], which allows computing the product  $xy$  using a random triple  $\langle a \rangle$ ,  $\langle b \rangle$ , and  $\langle c \rangle$ , and only requires two public reconstructions. Essentially, each party first locally computes the differences  $d = \langle x \rangle - \langle a \rangle$  and  $e = \langle y \rangle - \langle b \rangle$ , which are then publicly reconstructed. Since the product  $z = xy$  can be expanded as  $(d+a)(e+b) = de + db + ea + ab$ , the parties can locally reconstruct  $\langle xy \rangle$  using the formula:

$$\langle xy \rangle = de + d\langle b \rangle + e\langle a \rangle + \langle c \rangle$$

This uses the reconstructed values  $d$  and  $e$ , along with the pre-shared triple. To improve efficiency, the protocol processes a batch of  $l$  multiplications at once, requiring  $2l$  public reconstructions. These are grouped into batches of size  $t+1$ , which ensures that each reconstruction has an amortized communication cost of  $\mathcal{O}(n \log n)$  bits. As a result, the total communication complexity of the batched multiplication protocol over  $l$  multiplications is  $\mathcal{O}((n^2 + nl) \log n)$ .

#### 8.5 Efficiency

Parties provide inputs using a single call to the complete VSS protocol, generate triplets with dealers, and call an ACS protocol in the synchronous phase. Following that, parties combine the triplets with dealers to triples without dealers, and continue with computing the circuit. The input sharing and ACS require  $\mathcal{O}(n^4 \log n)$  communication. In total,  $C$  triples need to be generated. Each dealer shares triples at a rate of  $\mathcal{O}(n^2 \log n)$  per triple (plus  $\mathcal{O}(n^4 \log n)$  across all sharings), and combining them at rate 1 by taking  $\mathcal{O}(n)$  triples with dealers and generating  $\mathcal{O}(n)$  triples without dealers. This means that in total,  $C$  triples need to be shared by dealers, for a total cost of  $\mathcal{O}(Cn^2 \log n)$ . Finally, the circuit is evaluated by batch reconstructing triples at each layer. In such a reconstruction,  $\mathcal{O}(n)$  triples can be reconstructed with  $\mathcal{O}(n^2 \log n)$  communication, but with at least one reconstruction of cost  $\mathcal{O}(n^2 \log n)$  at each layer of the circuit. In total, this adds a cost of  $\mathcal{O}((Dn^2 + Cn) \log n)$ . All protocols require  $\mathcal{O}(1)$  expected rounds, except the evaluation, which must be computed layer-by-layer sequentially, requiring  $\mathcal{O}(1)$  time per layer. In total, the protocol requires  $\mathcal{O}(D)$  expected time.

## References

1. Abraham, I., Asharov, G., Chandramouli, A.: Simple is COOL: graded dispersal and its applications for byzantine fault tolerance. In: Meka, R. (ed.) 16th Innovations in Theoretical Computer Science Conference, ITCS 2025, January 7-10, 2025, Columbia University, New York, NY, USA. LIPIcs, vol. 325, pp. 1:1–1:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2025). <https://doi.org/10.4230/LIPICS.ITCS.2025.1>, <https://doi.org/10.4230/LIPICS.ITCS.2025.1> 17, 26
2. Abraham, I., Asharov, G., Patil, S., Patra, A.: Detect, pack and batch: Perfectly-secure MPC with linear communication and constant expected time. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14005, pp. 251–281. Springer (2023). [https://doi.org/10.1007/978-3-031-30617-4\\_9](https://doi.org/10.1007/978-3-031-30617-4_9), [https://doi.org/10.1007/978-3-031-30617-4\\_9](https://doi.org/10.1007/978-3-031-30617-4_9) 4, 6, 9, 17, 18, 31, 74
3. Abraham, I., Asharov, G., Patil, S., Patra, A.: Perfect asynchronous MPC with linear communication overhead. In: Joye, M., Leander, G. (eds.) Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part V. Lecture Notes in Computer Science, vol. 14655, pp. 280–309. Springer (2024). [https://doi.org/10.1007/978-3-031-58740-5\\_10](https://doi.org/10.1007/978-3-031-58740-5_10), [https://doi.org/10.1007/978-3-031-58740-5\\_10](https://doi.org/10.1007/978-3-031-58740-5_10) 4, 6, 9, 17, 18, 20, 52, 54, 69, 71, 74
4. Abraham, I., Asharov, G., Patra, A., Stern, G.: Asynchronous agreement on a core set in constant expected time and more efficient asynchronous VSS and MPC. In: Boyle, E., Mahmoody, M. (eds.) Theory of Cryptography - 22nd International Conference, TCC 2024, Milan, Italy, December 2-6, 2024, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 15367, pp. 451–482. Springer (2024). [https://doi.org/10.1007/978-3-031-78023-3\\_15](https://doi.org/10.1007/978-3-031-78023-3_15), [https://doi.org/10.1007/978-3-031-78023-3\\_15](https://doi.org/10.1007/978-3-031-78023-3_15) 4, 6, 7, 9, 27
5. Abraham, I., Asharov, G., Yanai, A.: Efficient perfectly secure computation with optimal resilience. In: Nissim, K., Waters, B. (eds.) Theory of Cryptography - 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8-11, 2021, Proceedings, Part II. Lecture Notes in Computer Science, vol. 13043, pp. 66–96. Springer (2021). [https://doi.org/10.1007/978-3-030-90453-1\\_3](https://doi.org/10.1007/978-3-030-90453-1_3), [https://doi.org/10.1007/978-3-030-90453-1\\_3](https://doi.org/10.1007/978-3-030-90453-1_3) 4, 6
6. Abraham, I., Dolev, D., Halpern, J.Y.: An almost-surely terminating polynomial protocol for asynchronous byzantine agreement with optimal resilience. In: Bazzi, R.A., Patt-Shamir, B. (eds.) Proceedings of the Twenty-Seventh Annual ACM Symposium on Principles of Distributed Computing, PODC 2008, Toronto, Canada, August 18-21, 2008. pp. 405–414. ACM (2008). <https://doi.org/10.1145/1400751.1400804>, <https://doi.org/10.1145/1400751.1400804> 28
7. Abraham, I., Dolev, D., Stern, G.: Revisiting asynchronous fault tolerant computation with optimal resilience. Distributed Computing **35**(4), 333–355 (2022) 4
8. Alexandru, A.B., Blum, E., Katz, J., Loss, J.: State machine replication under changing network conditions. In: Agrawal, S., Lin, D. (eds.) Advances in

- Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I. Lecture Notes in Computer Science, vol. 13791, pp. 681–710. Springer (2022). [https://doi.org/10.1007/978-3-031-22963-3\\_23](https://doi.org/10.1007/978-3-031-22963-3_23), [https://doi.org/10.1007/978-3-031-22963-3\\_23](https://doi.org/10.1007/978-3-031-22963-3_23) 5, 7
9. Alhaddad, N., Das, S., Duan, S., Ren, L., Varia, M., Xiang, Z., Zhang, H.: Balanced byzantine reliable broadcast with near-optimal communication and improved computation. In: Milani, A., Woelfel, P. (eds.) PODC '22: ACM Symposium on Principles of Distributed Computing, Salerno, Italy, July 25 - 29, 2022. pp. 399–417. ACM (2022). <https://doi.org/10.1145/3519270.3538475>, <https://doi.org/10.1145/3519270.3538475> 26
  10. Appan, A., Chandramouli, A., Choudhury, A.: Perfectly-secure synchronous MPC with asynchronous fallback guarantees. In: Milani, A., Woelfel, P. (eds.) PODC '22: ACM Symposium on Principles of Distributed Computing, Salerno, Italy, July 25 - 29, 2022. pp. 92–102. ACM (2022). <https://doi.org/10.1145/3519270.3538417>, <https://doi.org/10.1145/3519270.3538417> 5, 6, 7, 8
  11. Appan, A., Chandramouli, A., Choudhury, A.: Perfectly-secure synchronous MPC with asynchronous fallback guarantees. *IEEE Trans. Inf. Theory* **69**(8), 5386–5425 (2023). <https://doi.org/10.1109/TIT.2023.3264444>, <https://doi.org/10.1109/TIT.2023.3264444> 5, 8
  12. Appan, A., Choudhury, A.: Network agnostic MPC with statistical security. In: Rothblum, G.N., Wee, H. (eds.) *Theory of Cryptography - 21st International Conference, TCC 2023, Taipei, Taiwan, November 29 - December 2, 2023, Proceedings, Part II. Lecture Notes in Computer Science*, vol. 14370, pp. 63–93. Springer (2023). [https://doi.org/10.1007/978-3-031-48618-0\\_3](https://doi.org/10.1007/978-3-031-48618-0_3), [https://doi.org/10.1007/978-3-031-48618-0\\_3](https://doi.org/10.1007/978-3-031-48618-0_3) 5, 7, 8
  13. Asharov, G., Lindell, Y., Rabin, T.: Perfectly-secure multiplication for any  $t < n/3$ . In: *Advances in Cryptology - CRYPTO 2011* (2011) 4, 6
  14. Bacho, R., Collins, D., Liu-Zhang, C., Loss, J.: Network-agnostic security comes (almost) for free in DKG and MPC. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 14081, pp. 71–106. Springer (2023). [https://doi.org/10.1007/978-3-031-38557-5\\_3](https://doi.org/10.1007/978-3-031-38557-5_3), [https://doi.org/10.1007/978-3-031-38557-5\\_3](https://doi.org/10.1007/978-3-031-38557-5_3) 5, 6, 7
  15. Bangalore, L., Choudhury, A., Patra, A.: The power of shunning: Efficient asynchronous byzantine agreement revisited. *J. ACM* **67**(3), 14:1–14:59 (2020). <https://doi.org/10.1145/3388788>, <https://doi.org/10.1145/3388788> 28
  16. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: *Annual International Cryptology Conference* (1991) 9, 76
  17. Beerliová-Trubíniová, Z., Hirt, M.: Simple and efficient perfectly-secure asynchronous mpc. In: *Advances in Cryptology - ASIACRYPT 2007*. pp. 376–392. Berlin, Heidelberg (2007) 4, 6
  18. Beerliová-Trubíniová, Z., Hirt, M.: Perfectly-secure mpc with linear communication complexity. In: *Proceedings of the 5th Conference on Theory of Cryptography*. p. 213–230. TCC'08, Springer-Verlag, Berlin, Heidelberg (2008) 4, 6
  19. Ben-Or, M., Canetti, R., Goldreich, O.: Asynchronous secure computation. In: *ACM symposium on Theory of computing* (1993) 4, 6, 11
  20. Ben-Or, M., Canetti, R., Goldreich, O.: Asynchronous secure computation. In: *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*. p. 52–61. STOC '93, Association for Computing Machinery, New York, NY,

- USA (1993). <https://doi.org/10.1145/167088.167109>, <https://doi.org/10.1145/167088.167109> 14, 21
21. Ben-Or, M., Goldwasser, S., Wigderson, A.: Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In: Annual ACM Symposium on Theory of Computing (1988) 4, 6, 13
  22. Ben-Or, M., Kelmer, B., Rabin, T.: Asynchronous secure computations with optimal resilience (extended abstract). In: Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing. p. 183–192. PODC '94, Association for Computing Machinery, New York, NY, USA (1994) 4, 27
  23. Blum, E., Katz, J., Loss, J.: Synchronous consensus with optimal asynchronous fallback guarantees. In: Hofheinz, D., Rosen, A. (eds.) Theory of Cryptography - 17th International Conference, TCC 2019, Nuremberg, Germany, December 1-5, 2019, Proceedings, Part I. Lecture Notes in Computer Science, vol. 11891, pp. 131–150. Springer (2019). [https://doi.org/10.1007/978-3-030-36030-6\\_6](https://doi.org/10.1007/978-3-030-36030-6_6), [https://doi.org/10.1007/978-3-030-36030-6\\_6](https://doi.org/10.1007/978-3-030-36030-6_6) 5, 7
  24. Blum, E., Katz, J., Loss, J.: Tardigrade: An atomic broadcast protocol for arbitrary network conditions. In: Tibouchi, M., Wang, H. (eds.) Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part II. Lecture Notes in Computer Science, vol. 13091, pp. 547–572. Springer (2021). [https://doi.org/10.1007/978-3-030-92075-3\\_19](https://doi.org/10.1007/978-3-030-92075-3_19), [https://doi.org/10.1007/978-3-030-92075-3\\_19](https://doi.org/10.1007/978-3-030-92075-3_19) 5, 7
  25. Blum, E., Zhang, C.L., Loss, J.: Always have a backup plan: Fully secure synchronous MPC with asynchronous fallback. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12171, pp. 707–731. Springer (2020). [https://doi.org/10.1007/978-3-030-56880-1\\_25](https://doi.org/10.1007/978-3-030-56880-1_25), [https://doi.org/10.1007/978-3-030-56880-1\\_25](https://doi.org/10.1007/978-3-030-56880-1_25) 5, 7
  26. Bracha, G.: Asynchronous byzantine agreement protocols. Inf. Comput. **75**(2), 130–143 (1987). [https://doi.org/10.1016/0890-5401\(87\)90054-X](https://doi.org/10.1016/0890-5401(87)90054-X), [https://doi.org/10.1016/0890-5401\(87\)90054-X](https://doi.org/10.1016/0890-5401(87)90054-X) 12, 26
  27. Canetti, R.: Asynchronous secure computation. Technion - Computer Science Department - Technical Report **CS0755** (1993) 14
  28. Canetti, R.: Studies in secure multiparty computation and applications. Scientific Council of The Weizmann Institute of Science (1996) 19, 21
  29. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: FOCS (2001) 23
  30. Canetti, R., Chen, M.: Universally composable succinct vector commitments and applications. IACR Cryptol. ePrint Arch. p. 1103 (2025), <https://eprint.iacr.org/2025/1103> 24
  31. Chaum, D., Crépeau, C., Damgård, I.: Multiparty unconditionally secure protocols (extended abstract). In: 20th Annual ACM Symposium on Theory of Computing (1988) 4, 6
  32. Choudhury, A., Patra, A.: An efficient framework for unconditionally secure multiparty computation. IEEE Transactions on Information Theory **63**(1), 428–468 (2017). <https://doi.org/10.1109/TIT.2016.2614685> 74
  33. Cohen, R., Coretti, S., Garay, J.A., Zikas, V.: Probabilistic termination and composability of cryptographic protocols. J. Cryptol. **32**(3), 690–741 (2019). <https://doi.org/10.1007/S00145-018-9279-Y>, <https://doi.org/10.1007/S00145-018-9279-Y> 25, 26



34. Cohen, R., Forghani, P., Garay, J.A., Patel, R., Zikas, V.: Concurrent asynchronous byzantine agreement in expected-constant rounds, revisited. In: Rothblum, G.N., Wee, H. (eds.) *Theory of Cryptography - 21st International Conference, TCC 2023, Taipei, Taiwan, November 29 - December 2, 2023, Proceedings, Part IV. Lecture Notes in Computer Science*, vol. 14372, pp. 422–451. Springer (2023). [https://doi.org/10.1007/978-3-031-48624-1\\_16](https://doi.org/10.1007/978-3-031-48624-1_16), [https://doi.org/10.1007/978-3-031-48624-1\\_16](https://doi.org/10.1007/978-3-031-48624-1_16) 11, 28
35. Cramer, R., Damgård, I., Maurer, U.: General secure multi-party computation from any linear secret-sharing scheme. In: *International Conference on the Theory and Applications of Cryptographic Techniques (2000)* 4, 6
36. Damgård, I., Nielsen, J.B.: Scalable and unconditionally secure multiparty computation. In: *Annual International Cryptology Conference*. pp. 572–590. Springer (2007) 74
37. Deligios, G., Hirt, M., Liu-Zhang, C.: Round-efficient byzantine agreement and multi-party computation with asynchronous fallback. In: Nissim, K., Waters, B. (eds.) *Theory of Cryptography - 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8–11, 2021, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 13042, pp. 623–653. Springer (2021). [https://doi.org/10.1007/978-3-030-90459-3\\_21](https://doi.org/10.1007/978-3-030-90459-3_21), [https://doi.org/10.1007/978-3-030-90459-3\\_21](https://doi.org/10.1007/978-3-030-90459-3_21) 5, 7
38. Dodis, Y., Micali, S.: Parallel reducibility for information-theoretically secure computation. In: Bellare, M. (ed.) *Advances in Cryptology - CRYPTO 2000, 20th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20–24, 2000, Proceedings. Lecture Notes in Computer Science*, vol. 1880, pp. 74–92. Springer (2000). [https://doi.org/10.1007/3-540-44598-6\\_5](https://doi.org/10.1007/3-540-44598-6_5), [https://doi.org/10.1007/3-540-44598-6\\_5](https://doi.org/10.1007/3-540-44598-6_5) 9
39. Feldman, P., Micali, S.: Optimal algorithms for byzantine agreement. In: *20th Annual ACM Symposium on Theory of Computing (1988)* 6
40. Fischer, M.J., Lynch, N.A., Paterson, M.S.: Impossibility of distributed consensus with one faulty process. *Journal of the ACM* **32**(2), 374–382 (April 1985). <https://doi.org/10.1145/3149.214121>, <https://doi.org/10.1145/2F3149.214121> 21
41. Gennaro, R., Rabin, M.O., Rabin, T.: Simplified vss and fast-track multiparty computations with applications to threshold cryptography. In: *ACM symposium on Principles of distributed computing (1998)* 4, 6
42. Goyal, V., Liu, Y., Song, Y.: Communication-efficient unconditional mpc with guaranteed output delivery. In: *CRYPTO (2019)* 6
43. Katz, J., Maurer, U., Tackmann, B., Zikas, V.: Universally composable synchronous computation. In: Sahai, A. (ed.) *Theory of Cryptography - 10th Theory of Cryptography Conference, TCC 2013, Tokyo, Japan, March 3–6, 2013. Proceedings. Lecture Notes in Computer Science*, vol. 7785, pp. 477–498. Springer (2013). [https://doi.org/10.1007/978-3-642-36594-2\\_27](https://doi.org/10.1007/978-3-642-36594-2_27), [https://doi.org/10.1007/978-3-642-36594-2\\_27](https://doi.org/10.1007/978-3-642-36594-2_27) 24
44. Nayak, K., Ren, L., Shi, E., Vaidya, N.H., Xiang, Z.: Improved extension protocols for byzantine broadcast and agreement. *arXiv preprint arXiv:2002.11321* (2020) 15
45. Patil, S., Patra, A.: Perfectly-secure network-agnostic MPC with optimal resiliency. In: Balliu, A., Kuhn, F. (eds.) *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2025, Hotel Las Brisas Huatulco, Huatulco, Mexico, June 16–20, 2025*. pp. 121–130. ACM (2025). <https://doi.org/10.1145/3732772.3733500>, <https://doi.org/10.1145/3732772.3733500> 6, 7, 8, 68

- 46. Patra, A., Choudhary, A., Rangan, C.P.: Communication efficient perfectly secure VSS and MPC in asynchronous networks with optimal resilience. In: Progress in Cryptology - AFRICACRYPT 2010. vol. 6055, pp. 184–202. Springer (2010) 4, 6
- 47. Patra, A., Choudhury, A., Pandu Rangan, C.: Efficient asynchronous verifiable secret sharing and multiparty computation. Journal of Cryptology **28**(1), 49–109 (2015) 4, 6
- 48. Prabhu, B., Srinathan, K., Rangan, C.P.: Asynchronous unconditionally secure computation: An efficiency improvement. In: Progress in Cryptology — INDOCRYPT 2002. pp. 93–107. Berlin, Heidelberg (2002) 4, 6
- 49. Srinathan, K., Pandu Rangan, C.: Efficient asynchronous secure multiparty distributed computation. In: INDOCRYPT 2000. pp. 117–129 (2000) 4, 6